Two Games on a Dynkin Diagram

Specification for an explorable website on Mozes' numbers game, Wildberger's mutation game, Satake diagrams, and the Lie theory that connects them

Project owner: John Janik, Ph.D.

Version 1.1, 9 October 2026. Specification for handover to Claude Code, with sources (section 11).

Contents

0. Read this first **4**
0.1 What this package contains . . . 4
0.2 How Claude Code should use this document . . . 4
0.3 Provenance labels . . . 5
0.4 Three findings to know before reading further . . . 5
0.5 Map of the document . . . 6

1. Purpose, audience and scope **6**
1.1 Purpose . . . 6
1.2 Audience . . . 6
1.3 Learning outcomes . . . 7
1.4 The storyline . . . 7
1.5 Scope . . . 8
1.6 What success looks like . . . 8
1.7 Names . . . 8

2. The mathematical core **9**
2.1 The board . . . 9
2.2 The mutation game . . . 9
2.3 The numbers game . . . 10
2.4 The duality: one reflection, two coordinate systems . . . 10
2.5 The invariant form and root lengths . . . 11
2.6 What the positions are . . . 12
2.7 Facts about play on a finite-type board . . . 12
2.8 The boards of finite type . . . 14
2.9 Translating other authors' conventions . . . 15
2.10 Wildberger's two texts: names, numbering and misprints . . . 16
2.11 The Mutation Conjecture . . . 17
2.12 Affine boards . . . 17
2.13 Folding . . . 20
2.14 Geometry: Euclidean coordinates, polytopes, the Coxeter plane . . . 21
2.15 Satake diagrams . . . 24
2.16 Neighbouring games, and what "mutation" does not mean . . . 27
2.17 Golden boards: H_2, H_3, H_4 . . . 28
2.18 From the games to Lie algebra matrices . . . 29

3. Curriculum **30**
M0. Start here: two games, one board . . . 30
M1. The mutation game . . . 30
M2. The form behind the game . . . 31
M3. Finite, affine, wild . . . 32
M4. The numbers game and the duality . . . 33
M5. The Weyl group inside the games . . . 34
M6. Multiple edges: B, C, F, G and folding . . . 34
M7. Geometry: mirrors, polytopes and the E8 picture . . . 35
M8. From games to Lie algebras . . . 36
M9. Real forms and Satake diagrams . . . 37
M10. Atlas, sandbox, history and sources . . . 37

4. Catalogue of interactive tools **38**
4.0 Rules common to every tool [D] . . . 38
T1. Game Board (phase 1; used by every module) . . . 40
T2. Orbit Garden (phase 1) . . . 40
T3. Form Finder (phase 2) . . . 41
T4. Classification Lab (phase 2) . . . 42
T5. Numbers Game Lab (phase 3) . . . 42
T6. Kaleidoscope (phase 3) . . . 43
T7. Symbolic Game (phase 3) . . . 43
T8. Weyl Explorer (phase 3) . . . 43
T9. Kostant's Game (phase 1) . . . 44
T10. Geometry Studio (phase 4) . . . 44
T11. Folding Bench (phase 4) . . . 44
T12. Representation Builder (phase 5) . . . 45
T13. Satake Studio (phase 6) . . . 45
T14. Atlas (phase 6) . . . 46
T15. Affine Playground (phase 2) . . . 46
T16. Sandbox (phase 2) . . . 46

5. Engine specification **46**
5.1 Principles . . . 47
5.2 Types . . . 49
5.3 Boards . . . 50
5.4 The games and everything built on them . . . 50
5.5 Notes on algorithms . . . 53
5.6 Caps, and what happens off the finite boards . . . 54
5.7 Tests . . . 56

6. Data and drawing conventions **58**
6.1 Node positions . . . 58
6.2 Roads . . . 58
6.3 Values on nodes . . . 59
6.4 The Satake catalogue . . . 60
6.5 Names of real forms . . . 61
6.6 State, links and file formats . . . 61
6.7 LaTeX export . . . 63

7. Architecture **63**
7.1 Stack [D] . . . 63
7.2 Recent changes the builder must know about . . . 65
7.3 Repository layout and pages . . . 66
7.4 State, links, embedding, and life without JavaScript . . . 68

7.5 Accessibility requirements . . . 68
7.6 Performance budget [D] . . . 72
7.7 Testing . . . 73
7.8 Deployment and durability . . . 74
7.9 Licensing and credit [D; open decision in section 10] . . . 74

8. Design direction **74**
8.1 Twelve principles . . . 74
8.2 Anatomy of a lesson page . . . 76
8.3 Visual language [D] . . . 76
8.4 Writing rules . . . 76
8.5 Patterns for predictions and challenges . . . 77

9. Build plan **77**
9.1 Phases and gates . . . 77
9.2 Definition of done . . . 79
9.3 Working agreements for Claude Code . . . 79
9.4 Playtest script (owner's part of gate G3, and each later release) . . . 80
9.5 Risks . . . 80

10. Open decisions for the project owner **81**

Appendix A. Worked examples **82**
A.1 The three rank 2 boards . . . 82
A.2 The generic game on A_3 . . . 84
A.3 Weyl's dimension formula as two games . . . 84
A.4 Kostant's game . . . 84
A.5 Three small Satake diagrams . . . 84
A.6 The pentagon . . . 85

Appendix B. Satake tables for comparison with Helgason **85**
B.1 Exceptional boards [V] . . . 85
B.2 Classical boards of low rank, as test vectors [V] . . . 86

Appendix C. Dictionary **88**

Appendix D. Corrections to the project note "algebraic data" **89**

Appendix E. What a person should check before publishing **90**

Appendix F. Prior art **91**

Appendix G. Schema of `golden_data.json` **93**

Appendix H. Starter `CLAUDE.md` **95**

11. Sources **97**
11.0 How to read this section . . . 97
11.1 The mutation game (Wildberger) and work derived from it . . . 97
11.2 The numbers game (Mozes) and its literature . . . 100
11.3 Related games and structures . . . 104
11.4 Real forms and Satake diagrams . . . 106
11.5 Standard Lie theory, Coxeter groups, polytopes and the A-D-E pattern . . . 108
11.6 Explorable explanations: design practice and evidence . . . 110
11.7 Related interactive work surveyed (prior art) . . . 111
11.8 Technology and standards . . . 111

11.9 Corrections to citations that circulated in the project's earlier notes 111

0. Read this first

0.1 What this package contains

File	Role
`SPEC.md` (this file)	What to build: the mathematics with fixed conventions, the curriculum, every interactive tool, the engine, the architecture, the build plan, and the open decisions.
`SOURCES.md`	Section 11: the annotated bibliography and research record. Every citation key used here, such as [W20] or [Don08], is resolved there, with access status and with what was and was not confirmed.
`reference/lie_games.py`	A small reference implementation ("the oracle") of both games, root systems, Weyl data, classification of boards, folding, Wythoff polytopes, Satake diagrams with their catalogue, and the representation constructions. Python standard library only. Exact arithmetic throughout, except one clearly marked last section for Euclidean coordinates.
`reference/verify.py`	Runs every check behind this specification (6649 checks, all passing on 9 October 2026, in about 75 seconds) and writes the golden data. Standard library only; run with Python 3.11, 3.12 and 3.13, which write the same file byte for byte. It exits with a non-zero status if any check fails.
`reference/golden_data.json`	The golden data that the site's TypeScript engine must reproduce. Appendix G describes its schema.
`reference/bench_orbit.mjs`	The timing measurement behind the rule of section 5.1 (E-9) on what may run on the main thread. Node 22, no dependencies. It is a measurement, and no part of the contract.
`SPEC.pdf`	A typeset copy of `SPEC.md` and `SOURCES.md` for reading. The Markdown files are the source of truth.

0.2 How Claude Code should use this document

1. Read sections 0, 1 and 2 completely before writing any code. Section 2 fixes the conventions. They were reconciled across five source conventions and machine-checked; **do not change them**, and do not "simplify" a formula from memory.
2. Treat `reference/golden_data.json` as the acceptance oracle for the engine (section 5). When a mathematical statement in this document and the oracle seem to disagree, the oracle wins and the discrepancy is reported to the project owner.
3. Build in the phases of section 9. Each phase has a gate with an automatic part and a part for the project owner. The automatic part must pass before the next phase starts.
4. Write lesson text from sections 2 and 3 in the site's own words. Do not copy sentences, figures or table layouts from the papers. Cite by the keys of `SOURCES.md`.

5. Anything labelled [U] (unverified) must not be printed on the site as fact until a person has checked it. Appendix E is the checklist. Section 7.3 describes the component that marks such statements in previews and replaces them by a cautious fallback in a release.
6. Section 7.2 lists recent breaking changes in the toolchain (Astro 7, KaTeX 0.18, TypeScript 7, Vite 8) that postdate most training data. Read it before scaffolding.
7. Section 10 lists the decisions that belong to the project owner. Where one of them blocks work, use the stated default and record that the default was used.
8. Appendix H is a starter `CLAUDE.md` that restates the rules above in the form Claude Code loads automatically.

0.3 Provenance labels

Every mathematical or bibliographic claim that the site will rely on carries one of these labels, here and in `SOURCES.md`.

Label	Meaning
[S]	Sourced: read in the cited source itself during this research.
[S2]	Second-hand: reported by a reliable later source that was read; the original was not.
[V]	Verified by computation: checked by `reference/verify.py`. A claim marked [V] has a named check in that script, and the range of boards checked is stated with the claim or is "every type of rank at most 8".
[D]	Design decision or proposal of this specification. Open to revision by the project owner.
[U]	Unverified: from memory or inference. Check before publishing.

Statements with no label in section 2 are definitions. A label [V] says that the statement holds on the boards checked; where the statement is a theorem for all boards, the source of the theorem is named beside it.

0.4 Three findings to know before reading further

1. **The two games are one reflection written in two coordinate systems, and the Cartan matrix is the translator.** In the mutation game a move changes one number. In the numbers game the same move negates one number and changes its neighbours. A position p of the first becomes the position $a = Cp$ of the second, and moves correspond: $CS_i = R_iC$ (section 2.4) [V]. The graph never changes in either game. The project note "algebraic data" describes the mutation game incorrectly in three main respects; Appendix D gives the corrections.
2. **Nothing like this site exists.** The interactive record of the two games consists of a 2004 Java applet, a 2021 demonstration for simple graphs and a 2026 Python package with static plots. No interactive tool for Satake diagrams was found at all (Appendix F). Nothing joins the games to roots, Weyl groups, polytopes, representations and real forms in one place.
3. **The mathematics in this document was checked by machine and reviewed three times, and the reading of sources has stated limits.** Every type of rank at most 8 passes all checks, including the full Satake classification obtained by exhaustive enumeration and the printed root lists of Wildberger's two texts; the face counts of Wythoff polytopes were compared with exact convex hulls for 48 polytopes of dimensions 3 and 4. Three complete drafts were then each read by two independent reviewers, one for the mathematics and one for whether a builder could work from it; version 1.1 incorporates the findings of all three rounds. In the third round the mathematical reviewer

recomputed sections 2.7 to 2.15, all 167 Satake records and the appendices with independent code and reported no false statement there. On the source side, Mozes' paper, Helgason's table and Kac's book could not be opened by the research tools, and the web-search allowance ran out before three questions were settled. Appendix E lists what a person should check, in order of importance.

0.5 Map of the document

Section	Content
1	Purpose, audience, learning outcomes, the storyline, scope
2	The mathematical core: definitions, fixed conventions, verified facts, translation between authors
3	Curriculum: eleven modules, each with goals, narrative beats, the figure for each beat, and sources
4	Catalogue of sixteen interactive tools with behaviour and acceptance criteria
5	Engine specification: types, functions with the phase in which each is built, algorithms, exactness, limits, tests
6	Data and drawing conventions, the Satake catalogue, the state and link format
7	Architecture: stack with pinned versions, repository layout, state and URLs, accessibility, performance, testing, hosting, licensing
8	Design direction and writing rules
9	Phased build plan with gates, definition of done, working agreements, playtest script, risks
10	Open decisions for the project owner
Appendices	A worked examples; B Satake tables for comparison with Helgason; C dictionary of terms; D corrections to the project note; E human checklist; F prior art; G golden data schema; H starter `CLAUDE.md`
11 (`SOURCES.md`)	Annotated bibliography and research record

1. Purpose, audience and scope

1.1 Purpose

The site teaches the mathematical intuition that connects two elementary games to Lie theory. A reader plays Wildberger's mutation game and Mozes' numbers game on a Dynkin diagram, watches the root system, the weights, the Weyl group, the root polytopes and the simplest representations grow out of their own moves, and then decorates the same board with black nodes and arrows to meet the real forms through Satake diagrams. The games need only integer addition, so the entry cost is low; the destination is the structure theory that usually takes a graduate course to reach.

1.2 Audience

Reader	What they bring	What they should leave with
Primary: a mathematically curious reader with first-year linear algebra (undergraduate, physicist, engineer, programmer)	Vectors, matrices, the idea of a reflection	Outcomes LO1 to LO8 below, and the ability to read a Dynkin or Satake diagram
Secondary: a graduate student or researcher who knows Lie theory	Root systems, Weyl groups, highest weights	A fast computational playground, the precise dictionary between the two games and standard theory, and citable links to positions
Tertiary: teachers and expositors	Either of the above	Embeddable tools and exportable figures

The primary reader sets the level of the prose. Each module has a clearly marked "For readers who know Lie theory" paragraph that states the standard name of what was just played [D].

1.3 Learning outcomes

After working through the site a reader can:

- **LO1** play both games on any board, and state each rule as a formula.
- **LO2** explain why the two games are the same reflection in two coordinate systems, and convert a position of one into a position of the other with the Cartan matrix.
- **LO3** find all roots of a small board by play, recognise positive and negative roots, and find the highest root.
- **LO4** write down the invariant form of a board and use it to decide whether the game is finite, and connect this to the list $A_n, B_n, C_n, D_n, E_6, E_7, E_8, F_4, G_2$.
- **LO5** explain why every complete numbers game from a generic start has the same length and the same end, and read the Weyl group, reduced words, the exponents and the order of the group off the play.
- **LO6** read a multiple edge with an arrow as long and short roots, and fold a simply laced board to a multiply laced one.
- **LO7** build the vertices and edges of a root polytope or a Wythoff polytope from an orbit, change to Euclidean coordinates, and explain the Coxeter-plane picture of E_8.
- **LO8** explain how a reflection splits into raising and lowering steps, and build the matrices of a minuscule representation from a game graph.
- **LO9** read a Satake diagram as a decorated board, compute its involution, restricted roots and multiplicities by play, and name the real form.
- **LO10** (for the secondary reader) translate between the conventions of Wildberger, Mozes and Eriksson, Donnelly, Bourbaki, Kac and SageMath.

1.4 The storyline

The whole site hangs on seven statements. Each module develops one of them. [D]

1. A Dynkin diagram is a compact description of a set of mirrors meeting at a point: one mirror per node, with the edges recording the angles between mirrors and the arrows recording a difference in scale.
2. Reflect a point in one of those mirrors. If the point is recorded by its coordinates along the mirror normals, only one coordinate changes: this is a move of the **mutation game**. If the point is recorded by its signed distances to the mirrors, the distance to the chosen mirror changes sign and the neighbouring distances absorb it: this is a move of the **numbers game**.
3. The Cartan matrix converts the first record into the second. Its entries are the edge counts of the board. It is the one piece of algebraic data both games encode.
4. Playing is walking in the Weyl group. Positions of the mutation game are roots; positions of the numbers game are weights; legal play spells reduced words; the numbers fired along the way are the

heights of the coroots, and from them come the exponents, the order of the group and Weyl's dimension formula.

5. The board alone decides whether the game closes up. It does so exactly when a quadratic form attached to the board is positive definite, which happens exactly for the Dynkin diagrams. The boards on the boundary are the affine diagrams, where one population is frozen and one weighted sum is conserved.
6. A simple Lie algebra appears when each reflection is cut into single steps up and down. In the smallest representations every step is a game move, and the game graph is the representation.
7. A real form is the same algebra seen through an involution. On the board this is a choice of black nodes and arrows, a Satake diagram. The involution can itself be played; under it the roots cast shadows with multiplicities, and the shadows form a smaller board with its own game.

1.5 Scope

In scope for version 1 (phases 0 to 6 of section 9): both games on all finite-type boards of rank at most 8, on affine boards (twisted ones included) and on reader-drawn boards of up to 12 nodes, including boards with a one-way road as counterexamples; the linked two-view game board; orbits, roots and root posets; the invariant form and the classification lab; the Weyl group, reduced words, heaps and Kostant's game; folding; Euclidean coordinates, root polytopes, Wythoff polytopes in dimensions 3 and 4 with projections from higher rank, the Coxeter plane and the E_8 picture; minuscule representations built from the game; the Satake studio with the full catalogue for rank at most 8; an atlas of data; shareable links and figure export.

Planned extensions, specified here but built after version 1 (phase 7): the golden boards H_2, H_3, H_4 and the golden foldings $A_4 \to H_2$, $D_6 \to H_3$, $E_8 \to H_4$ (section 2.17) [V]; the 7-dimensional representation of G_2 and the vector representations of type B from single steps (section 2.18) [V]; generalized Satake diagrams; Vogan diagrams as a side-by-side view; embeddable web components.

Out of scope: general representations with weight multiplicities above one (other than the cases named above); Lie groups as manifolds; Kac-Moody algebras beyond the affine boards as game boards; cluster algebras, except for one page explaining that Fomin-Zelevinsky mutation is a different operation (section 2.16); user accounts, server-side computation, analytics beyond privacy-respecting page counts.

1.6 What success looks like

- **Correctness**: the TypeScript engine reproduces `golden_data.json` under the comparison rules of section 5.7 (integers exactly, the few floating-point values to a stated tolerance), and the property tests of that section pass.
- **Comprehension**: in a playtest, a reader with linear algebra and no Lie theory can, after modules M0 to M4, (a) predict the result of a move in either game, (b) convert a population to its numbers, (c) explain in their own words why a complete game on A_3 always takes six moves. Section 9.4 gives the playtest script.
- **Reach**: every page reads completely without interaction and without JavaScript (section 7.4 says what each tool shows in that case); every tool is operable by keyboard and announced to screen readers; every page meets the performance budget of section 7.6 on a mid-range phone.
- **Durability**: no runtime dependency on any server or plug-in; versions pinned; the built site archived at each release.

1.7 Names

Working title: *Two Games on a Dynkin Diagram* [D]; the final name is an open decision (section 10). In prose and interface the two views of a board are called **Populations** (the mutation game; roots) and **Numbers** (the numbers game; weights). Wildberger's story of Martians and anti-Martians on Mars, and of Venus for the dual game, is retold with credit as the narrative for these two views ([W20] section 2; [W-MS] pp. 1 and 30). Engine code uses the neutral words `population`, `position`, `mutate` and `fire`.

2. The mathematical core

This section is the single source of truth for conventions. In prose and formulas nodes are numbered from 1, with the Bourbaki labels of section 2.8; the added node of an affine board is numbered 0. The engine counts positions from 0 internally, as the oracle does, and everything outside the engine (data files, links, the screen) uses the labels (section 5.1, E-7). Vectors are columns and matrices act on the left. C_{ij} means row i, column j.

2.1 The board

A **board** is a directed multigraph on nodes $1, \dots, n$ with no loops. Its **adjacency matrix** is

$$A_{ij} = \text{number of edges from } i \text{ to } j \quad (i \neq j), \qquad A_{ii} = 0.$$

Unless stated otherwise a board is **bidirected**: $A_{ij} > 0$ exactly when $A_{ji} > 0$, so every road has at least one road back. A simple graph is the special case $A_{ij} = A_{ji} \in \{0, 1\}$: one undirected edge counts as one road each way ([W20] section 2). A board on which some road has no road back is a **one-way board**. Both games can be played on a one-way board, since the formulas below make sense for any road counts, but none of the theory of this section applies to it; section 2.11 shows what goes wrong. Two nodes are **joined** when at least one road runs between them, in either direction, and a board is **connected** when its nodes cannot be split into two groups with no road between them.

The **Cartan matrix** of the board is

$$C = 2I - A^{\mathsf{T}}, \qquad C_{ii} = 2, \qquad C_{ij} = -A_{ji} \ \ (i \neq j).$$

So $-C_{ij}$ is the number of roads from j **into** i. For a bidirected board, C is a generalized Cartan matrix in the sense of Kac, in the convention $C_{ij} = \langle \alpha_i^\vee, \alpha_j \rangle$ that SageMath also uses, and every generalized Cartan matrix is the matrix of exactly one bidirected board. Bourbaki and Humphreys print the transpose (section 2.9). The engine sorts matrices into three kinds before anything else (function `board_kind` of the oracle): `gcm` for a bidirected board, `one-way` for a board with a one-way road, and `not-a-board` when a diagonal entry is not 2 or an off-diagonal entry is positive.

Wildberger writes "an edge of type (m, n) from x_i to x_j" for $m = A_{ij}$ roads from x_i to x_j and $n = A_{ji}$ back ([W20] section 4). The pairs that occur on finite-type boards are $(1, 1)$, $(2, 1)$ and $(3, 1)$. In a drawing, $(1, 1)$ is a single line; $(2, 1)$ and $(3, 1)$ are a double and a triple line with an arrowhead pointing **from** i **to** j, the direction in which the extra roads run. Section 2.5 shows that this is the direction from the longer root to the shorter one, so the drawing is the usual Dynkin diagram.

2.2 The mutation game

A **population** is a vector $p \in \mathbb{Z}^n$: $p_i > 0$ means p_i Martians in city i, $p_i < 0$ means anti-Martians. A **mutation at** i changes only the entry p_i:

$$p_i \ \longmapsto \ -p_i + \sum_{j \neq i} A_{ji}\, p_j \ = \ p_i - \sum_j C_{ij}\, p_j.$$

In words: the inhabitants of city i change type, and every other city sends one copy of its population down each road that leads into i. As a matrix, $p \mapsto S_i p$ with

$$S_i = I - e_i e_i^{\mathsf{T}} C$$

(the identity with row i replaced by e_i^{T} minus row i of C). The board is never changed by a mutation.

Facts ([W20] Theorem 5.1 [S]): $S_i^2 = I$; $S_iS_j = S_jS_i$ when i and j are not joined; S_iS_j has order 3, 4, 6 when $A_{ij}A_{ji} = 1, 2, 3$ [all three V for every pair of nodes of every type of rank at most 8]; and S_iS_j has infinite order when $A_{ij}A_{ji} \geq 4$, and also when the roads between i and j run one way only. (For four boards with $A_{ij}A_{ji} \geq 4$ the script confirms that the trace of the 2×2 block is $A_{ij}A_{ji} - 2 \geq 2$ and that no power up to the 200th is the identity [V].)

The unit vector e_i, one Martian in city i, is a **simple root population**. A **root population** is anything reachable from a simple root population by mutations. R denotes the set of root populations of a board, and a root population is **positive** when all its entries are ≥ 0.

Lie theory reading. p stands for $\sum p_i\alpha_i$ in the root lattice Q, and S_i is the simple reflection $s_i(\beta) = \beta - \langle\alpha_i^\vee, \beta\rangle\alpha_i$ written in simple-root coordinates. R is the set of (real) roots and the group generated by the S_i is the Weyl group W.

2.3 The numbers game

A **position** is a vector $a \in \mathbb{Z}^n$, one number on each node. **Firing node** i negates a_i and feeds the neighbours:

$$a_i \longmapsto -a_i, \qquad a_j \longmapsto a_j + A_{ij}\,a_i \;=\; a_j - C_{ji}\,a_i \quad (j \neq i).$$

In words: node i sends its number down every road that **leaves** it, then changes sign. As a matrix, $a \mapsto R_i a$ with

$$R_i = I - C\,e_i e_i^{\mathsf{T}}$$

(the identity with column i replaced by e_i minus column i of C).

Legal play (the game proper): a node may be fired only when its number is **positive**, and the game ends when no number is positive. **Free play** fires any node whatever its sign; this is the group action. The site offers both and always says which is active.

The sign of the legal move is a convention. This specification follows Proctor, Donnelly and Wildberger and fires positive numbers. Mozes, Eriksson, the 1986 Olympiad problem and Gashi, Schedler and Speyer fire negative numbers; negating every number translates one convention into the other because each firing is linear (section 2.9).

Lie theory reading. a is the list of Dynkin labels of the weight $\lambda = \sum a_i\omega_i$, that is $a_i = \langle\lambda, \alpha_i^\vee\rangle$. Firing is the same simple reflection, $s_i\lambda = \lambda - a_i\alpha_i$, written in fundamental-weight coordinates, because $\alpha_i = \sum_j C_{ji}\,\omega_j$ (column i of C). Legal play moves from the dominant chamber towards the antidominant one.

2.4 The duality: one reflection, two coordinate systems

(a) The translator. Send a population p to the position $a = Cp$. Then for every node i

$$C\,S_i \;=\; R_i\,C \qquad \text{[V]},$$

so mutating city i and then translating gives the same result as translating and then firing node i. Entry by entry,

$$a_i = 2p_i - \sum_{j \neq i} A_{ji}p_j, \qquad \text{and a mutation at } i \text{ replaces } p_i \text{ by } p_i - a_i.$$

The number on node i is the amount by which a mutation would lower city i. A positive number marks a mutation that lowers, zero a mutation that changes nothing, a negative number a mutation that

raises. This sentence is the bridge between the two views and must appear, with the same wording, wherever the two views are shown together [D].

(b) The transpose. $R_i(C) = S_i(C^{\mathsf{T}})^{\mathsf{T}}$ [V]: the numbers game on a board is the transpose of the mutation game on the board with every road reversed. On a simple graph C is symmetric and $R_i = S_i^{\mathsf{T}}$. On a multiply laced board the reversed board is the dual diagram ($B_n \leftrightarrow C_n$; F_4 and G_2 with long and short exchanged).

(c) The games do not coincide, even on simply laced boards. S_i changes one entry; R_i changes the entry and its neighbours. They are intertwined by C, which is a different statement. Over the integers C maps $\mathbb{Z}^n$ onto a sublattice of index $\det C$, so the numbers game has more positions than the mutation game: populations fill the root lattice Q, positions fill the weight lattice P, and P/Q has $\det C$ elements. On E_8, F_4 and G_2, where $\det C = 1$, every position of the numbers game is the translation of exactly one integer population [V].

(d) Geometry. With the invariant form of section 2.5, the entries of p are the coefficients of a point along the mirror normals α_i, and

$$a_i = \langle \lambda, \alpha_i^\vee \rangle = \frac{2(\lambda, \alpha_i)}{(\alpha_i, \alpha_i)} = \frac{2}{|\alpha_i|} \times (\text{signed distance from the point to mirror } i).$$

So a population lists coordinates along the normals and a position lists scaled distances to the mirrors. One reflection, recorded two ways. The Cartan matrix is the matrix of pairings between normals and mirrors, which is why it is the translator.

(e) Wildberger's formulation. The manuscript defines the dual population of p as $q = p\tau$ with $q(x) = (p\, s_x - p)(x)$, "the change in p at x resulting from a mutation at x", notes that τ has matrix $A - 2I$, and proves $p\, s_x \tau = p\, \tau r_x$ ([W-MS] Proposition 0.3, pp. 31-32 [S]). In the notation here $q = -Cp = -a$: his dual population records the gain, this specification records the drop. The site uses $a = Cp$ so that the numbers are the Dynkin labels of the weight. Because every move is linear, the two choices give the same games.

2.5 The invariant form and root lengths

A board is **balanced**, and its Cartan matrix **symmetrizable** (Kac), when there are positive rational numbers $d_1, \dots, d_n$ with

$$d_i\, C_{ij} = d_j\, C_{ji} \quad \text{for all } i, j, \qquad \text{equivalently} \qquad d_i\, A_{ji} = d_j\, A_{ij}.$$

On a bidirected board such d exist exactly when, around every cycle, the product of the road counts in one direction equals the product in the other; every bidirected tree is balanced; the d_i are unique up to a common factor on each connected component ([W20] Lemma 8.3, Corollary 8.5 [S]; Kac's Exercise 2.1 [S2]). The word is Wildberger's, with one difference: he defines "balanced" for any directed multigraph by the condition on cycles, and proves it equivalent to the existence of such weights for bidirected graphs. A single one-way road is balanced in his sense, being a tree, and has no positive weights; in this specification "balanced" always means that the weights exist. The engine normalises $\min_i d_i = 1$ on each component.

The **invariant form** is

$$(p, q) = p^{\mathsf{T}} B\, q, \qquad B = DC, \quad B_{ij} = d_i C_{ij},$$

a symmetric matrix with $(\alpha_i, \alpha_i) = 2d_i$. Every mutation preserves it, $(S_i p, S_i q) = (p, q)$ [V], and on a connected board it is the only invariant bilinear form up to a constant ([W20] Theorems 8.4 and 8.7 [S]). A one-way board has no invariant form with all $d_i > 0$ and is never balanced in the sense just fixed. A mutation is the reflection $S_i p = p - \frac{2(p, e_i)}{(e_i, e_i)} e_i$ for this form. Wildberger's balanced weights α_i are the numbers $(\delta_{x_i}, \delta_{x_i}) = 2d_i$, up to scale ([W20] Lemma 8.9).

Consequences used throughout:

- **Long and short.** $d_i/d_j = A_{ij}/A_{ji}$, so more roads run from the longer root to the shorter. With the normalisation above, short roots have $(\beta,\beta) = 2$ and long roots have 4 (types B, C, F) or 6 (type G).
- **Simple graphs.** $d_i = 1$ and $(p,p) = 2\sum_i p_i^2 - 2\sum_{\text{edges } ij} p_i p_j$, the Tits form of the graph ([W20] section 7).
- **Coroots.** For a root $\beta = \sum b_i\alpha_i$, the coroot $\beta^\vee = 2\beta/(\beta,\beta)$ has coordinates $b_i d_i \big/ \frac{1}{2}(\beta,\beta)$ in the simple coroots. These are integers, and the coroots are exactly the root populations of the reversed board C^T [V].
- **Numbers as pairings.** $a = Cp$ says $a_i = \langle p, \alpha_i^\vee \rangle$.

2.6 What the positions are

Object	On the board	Coordinates
Root lattice Q	all populations	$\mathbb{Z}^n$, basis the simple roots $\alpha_i = e_i$
Weight lattice P	all positions of the numbers game	$\mathbb{Z}^n$, basis the fundamental weights $\omega_i = e_i$
Inclusion $Q \subset P$	$p \mapsto a = Cp$	index $\det C$
A weight as a population	$p = C^{-1}a$	rational, with denominators dividing $\det C$
ρ	the position with 1 on every node	$(1, \dots, 1)$
Dominant position	no negative number	$a_i \geq 0$
Weyl group W	the group generated by the moves	acts on Q by the S_i and on P by the R_i
Length of $w \in W$	least number of moves producing w	
Longest element w_0	the element reached by any complete legal game from ρ	sends ρ to $-\rho$
$-w_0$	a symmetry of the board	a permutation of the nodes (section 2.8)
Highest root θ	the root population of largest form value that no mutation raises	unique [V]

2.7 Facts about play on a finite-type board

"Finite type" means one of the boards of section 2.8. N is the number of positive roots, h the Coxeter number. Every statement below was checked by `verify.py` for all types of rank at most 8 unless a narrower range is stated; the sources named are where the statement, or the theorem behind it, is found.

- **F1 Signs.** Every root population is all ≥ 0 or all ≤ 0, there are $2N$ of them, and $N = nh/2$. ([W03a] Proposition 3.3 for simple graphs, stated with the remark that no direct combinatorial proof is known to the author; see section 2.11.)
- **F2 Norm two.** On a simply laced board the positive roots are exactly the non-negative integer populations with $(p,p) = 2$ (checked inside the box below the highest root; the general statement is [W03a] Proposition 3.5 [S]).
- **F3 Strong convergence.** From any start, any two legal plays of the numbers game end at the same position after the same number of moves, and fire the same multiset of numbers. The end position has no positive number. (Checked on random starts in eight types. The definition is [Eri95b] Definition 1.1; the theorem is attributed to Mozes and Eriksson [S2]; [Don08] Theorem 2.1 [S].)
- **F4 The generic game.** From ρ, or from any start with all numbers positive, every legal play has exactly N moves and ends at w_0 applied to the start: the negatives of the starting numbers, permuted

by $-w_0$. From ρ the end is $-\rho$. (Checked from ρ and from six random all-positive starts on every type.)

- **F5 Reduced words.** The complete legal plays from an all-positive start are exactly the reduced words of w_0; the positions reachable by legal play correspond one to one with the elements of W; a node carries a positive number exactly when firing it lengthens the element; and the least number of moves that reaches a position equals the number of positive roots β with $\langle a, \beta^\vee \rangle < 0$, the mirrors that separate the position from the start. (The count of reachable positions was checked for $|W| \leq 60000$, the two statements about lengths for $|W| \leq 2000$. [BB05] section 4.3 [S2]; [Don08] Theorem 2.8 [S]; [Hum90] section 1.7 [S].)
- **F6 Fired numbers are coroot pairings.** Play a complete legal game with indeterminates $x_1, \ldots, x_n$ as the start, using the word of any complete game from ρ. The linear forms fired are the positive root populations of the reversed board, each exactly once. So from a start λ with all numbers positive, the numbers fired are the values $\langle \lambda, \beta^\vee \rangle$, one for each positive coroot $\beta^\vee$; from ρ they are the heights of the positive coroots. From a dominant start with zeros, the same holds for the coroots with a nonzero value ([Don08] Remark 2.13, Theorem 5.2 and section 3 [S]; checked from random all-positive starts and from every fundamental weight whose orbit has at most 20000 positions).
- **F7 Exponents.** Let k_j be the number of times the value j is fired in the game from ρ. Then $k_1 \geq k_2 \geq \ldots$, and the conjugate partition is the list of exponents $m_1 \leq \ldots \leq m_n$. Further $N = \sum m_i$, $h = m_n + 1$ and $|W| = \prod(m_i + 1)$. (For roots this is the theorem of Shapiro, Steinberg, Kostant and Macdonald: [Ste59] 9.1 [S], [Kos59] [S2]. The game gives the version for coroots; see the caution in `SOURCES.md` 11.5.)
- **F8 The wave.** On a board of rank at least 2, colour the nodes with two colours so that joined nodes differ. From ρ, fire all nodes of one colour, then all of the other, and repeat; one **round** is the firing of one colour class. Every firing is legal and the game ends after exactly h rounds, whichever colour starts (on A_3: 4 rounds, 6 moves). On A_1 one colour class is empty and the statement degenerates to a single move. ([FZ03b] Lemma 2.1 [S]; [Ste59] 6.3 [S].)
- **F9 Weyl's dimension formula from two games.** Record the word of any complete game from ρ and the product D of the numbers fired. Replay the same word from $\lambda + \rho$ and let M be the product of the numbers fired. Then $\dim V(\lambda) = M/D$. (Checked on 30 representations, from $\dim 3$ for A_2 to 3875 for E_8.)
- **F10 Starts with zeros.** From a fundamental weight ω_i the game has $N - N_J$ moves, where N_J is the number of positive roots of the board with node i removed; the positions reachable by legal play are the whole orbit, which has $|W|/|W_J|$ elements; the game ends at $w_0\omega_i$. ([Don08] Proposition 3.2 and corollaries [S], for any dominant start. The length and the end were checked for every node of every type; the statement about the whole orbit for the orbits of at most 20000 positions, which leaves out nodes 3 to 6 of E_8.)
- **F11 Minuscule nodes.** The following are equivalent for a node i: every position in the orbit of ω_i has all its numbers in $\{-1, 0, 1\}$; the entry at node i of the highest root **of the reversed board** is 1. Such nodes are **minuscule**. Their number is $\det C - 1$. A third equivalent condition, which needs no enumeration: every number fired in the game from ω_i is 1. (The first condition was checked by enumeration for orbits of at most 20000 positions, the other two for every node. In Lie-theoretic terms the second condition reads: $\alpha_i^\vee$ has coefficient 1 in the highest root of the dual root system. That root is the coroot of the highest *short* root, and it differs in general from $\theta^\vee$, the coroot of the highest root, whose coefficients are the comarks of section 2.12: for B_3 the highest root of the reversed board is $(2, 2, 1)$, while $\theta^\vee = (1, 2, 1)$. The golden key is `highest_root_of_reversed_board`.)
- **F12 How many games.** The number of complete legal games from ρ is the number of reduced words of w_0: 2, 16, 768, 292864 for A_2 to A_5 (OEIS A005118 [S]; the theorem is Stanley's, [Sta84], not read); 2, 42, 24024 for B_2 to B_4, and the same for C (OEIS A039622 [S]; [BH95] Proposition 3.3 [S]); 2316 for D_4, 12985968 for D_5, 2144892 for F_4, 2 for G_2 (computed; no published value found). All of these values are asserted by the script [V]. It also confirms that the count for A_n equals the number of standard Young tableaux of the staircase shape $(n, n-1, \ldots, 1)$ for $n \leq 7$, and that the count for B_n and C_n equals the number of standard Young tableaux of the $n \times n$ square for $n \leq 6$ [V]. `golden_data.json` has the counts for every type with $|W| \leq 60000$.
- **F13 Kostant's game.** Start from one Martian in city i and mutate only where the mutation raises

the population. Every population met is a positive root. On a simply laced board every such move raises a city by exactly 1, every play has exactly $h-2$ moves, and all plays end at the highest root. On B, C, F, G the play ends at the highest root when i is a long node and at the highest short root when i is a short node. (Sources and attribution in `SOURCES.md` 11.3.)

- **F14 The highest root and h.** The entries of the highest root sum to $h-1$ [V]. (A classical fact; the usual reference is [Ste59], and the item number there was not confirmed [U].)

2.8 The boards of finite type

Node numbering is Bourbaki's [S2; identical to SageMath's, which was run]:

- A_n: path $1-2-\cdots-n$.
- B_n ($n \geq 2$): path $1-\cdots-n$; two roads from $n-1$ to n and one back; node n is short. $C_{n,n-1} = -2$.
- C_n ($n \geq 2$): path $1-\cdots-n$; two roads from n to $n-1$ and one back; node n is long. $C_{n-1,n} = -2$.
- D_n ($n \geq 4$): path $1-\cdots-(n-1)$, and node n joined to node $n-2$.
- E_6, E_7, E_8: path $1-3-4-5-6(-7(-8))$, and node 2 joined to node 4.
- F_4: path $1-2-3-4$; two roads from 2 to 3 and one back; nodes 1, 2 long and 3, 4 short. $C_{32} = -2$.
- G_2: nodes 1, 2; three roads from 2 to 1 and one back; node 1 short, node 2 long. $C_{12} = -3$.

Reference matrices (all confirmed against SageMath):

$$B_2 = \begin{pmatrix} 2 & -1 \\ -2 & 2 \end{pmatrix}, \quad C_2 = \begin{pmatrix} 2 & -2 \\ -1 & 2 \end{pmatrix}, \quad G_2 = \begin{pmatrix} 2 & -3 \\ -1 & 2 \end{pmatrix}, \quad F_4 = \begin{pmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -2 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{pmatrix}.$$

The full data for A_1 to A_8, B_2 to B_8, C_2 to C_8, D_4 to D_8, E_6, E_7, E_8, F_4, G_2 is in `golden_data.json`. A selection [V]:

Type	N	h	$\lvert W\rvert$	exponents	$\det C$	highest root	highest short root	minuscule nodes	$-w_0$ on nodes
A_1	1	2	2	1	2	(1)		1	identity
A_2	3	3	6	1, 2	3	(1, 1)		1, 2	(2, 1)
A_3	6	4	24	1, 2, 3	4	(1, 1, 1)		1, 2, 3	(3, 2, 1)
A_4	10	5	120	1, 2, 3, 4	5	(1, 1, 1, 1)		1, 2, 3, 4	(4, 3, 2, 1)
A_5	15	6	720	1, 2, 3, 4, 5	6	(1, 1, 1, 1, 1)		1, 2, 3, 4, 5	(5, 4, 3, 2, 1)
B_2	4	4	8	1, 3	2	(1, 2)	(1, 1)	2	identity
B_3	9	6	48	1, 3, 5	2	(1, 2, 2)	(1, 1, 1)	3	identity
B_4	16	8	384	1, 3, 5, 7	2	(1, 2, 2, 2)	(1, 1, 1, 1)	4	identity
C_2	4	4	8	1, 3	2	(2, 1)	(1, 1)	1	identity
C_3	9	6	48	1, 3, 5	2	(2, 2, 1)	(1, 2, 1)	1	identity
C_4	16	8	384	1, 3, 5, 7	2	(2, 2, 2, 1)	(1, 2, 2, 1)	1	identity
D_4	12	6	192	1, 3, 3, 5	4	(1, 2, 1, 1)		1, 3, 4	identity
D_5	20	8	1920	1, 3, 4, 5, 7	4	(1, 2, 2, 1, 1)		1, 4, 5	(1, 2, 3, 5, 4)
D_6	30	10	23040	1, 3, 5, 5, 7, 9	4	(1, 2, 2, 2, 1, 1)		1, 5, 6	identity
E_6	36	12	51840	1, 4, 5, 7, 8, 11	3	(1, 2, 2, 3, 2, 1)		1, 6	(6, 2, 5, 4, 3, 1)
E_7	63	18	2903040	1, 5, 7, 9, 11, 13, 17	2	(2, 2, 3, 4, 3, 2, 1)		7	identity
E_8	120	30	696729600	1, 7, 11, 13, 17, 19, 23, 29	1	(2, 3, 4, 6, 5, 4, 3, 2)		none	identity
F_4	24	12	1152	1, 5, 7, 11	1	(2, 3, 4, 2)	(1, 2, 3, 2)	none	identity
G_2	6	6	12	1, 5	1	(3, 2)	(2, 1)	none	identity

N is the number of positive roots and h the Coxeter number. In the column "$-w_0$ on nodes" the entry $(\sigma(1), \ldots, \sigma(n))$ lists the images of the nodes; it is the reversal on A_n, the exchange of the last two nodes on

D_n for odd n, the flip on E_6 and the identity on every other type [V]. The numbers of complete games are in F12 and in the JSON.

Names in rank 2. B_2 and C_2 are the same board with the two nodes exchanged. The oracle and the golden data call a board recognised up to renumbering `B2`, never `C2`, and call its affine board `C2~`, which is how the standard lists name them. The site shows "$B_2 = C_2$" where a single name could mislead [D].

2.9 Translating other authors' conventions

The table gives, for each source, how to turn its objects into the objects of this specification. It is the basis of the "conventions" switch in the Atlas (tool T14) and of outcome LO10.

Source	Its rule, in its own notation	Legal move	Translation to this specification
This specification	mutation $p_i \mapsto -p_i + \sum_{j \neq i} A_{ji} p_j$; firing $a_j \mapsto a_j + A_{ij} a_i$, $a_i \mapsto -a_i$	fire positive	$C_{ij} = \langle \alpha_i^\vee, \alpha_j \rangle$, $A_{ij} = -C_{ji}$
Wildberger, mutation ([W20] section 5; [W-MS] p. 10; [W03c] section 2) [S]	$(p\, s_x)(x) = -p(x) + \sum_{z \neq x} A(z,x) p(z)$, row vectors, maps written on the right	any	identical; $A(z,x) = A_{zx}$
Wildberger, dual mutation ([W-MS] p. 30; [W03c] section 2) [S]	$(q\, r_x)(x) = -q(x)$, $(q\, r_x)(y) = q(y) + A(x,y) q(x)$; positions called weights	any; the "numbers game" allows only positive firings	identical; $a = q$
Wildberger, the map τ ([W-MS] p. 31) [S]	$q = p\tau$, $q(x) = (p\, s_x - p)(x)$, matrix $A - 2I$		$q = -Cp = -a$ (opposite sign to section 2.4)
Olympiad 1986, Problem 3 [S]	integers on a pentagon; a negative y becomes $-y$ and is added to both neighbours	fire negative	$a = -x$; board the 5-cycle
Mozes [Moz90] [S2]	integer weights from a symmetrizable generalized Cartan matrix	fire negative	$a = -x$; which of C, C^T is his matrix is [U]
K. Eriksson ([Eri95b] section 10) [S]	neighbour t of the fired node s gains $k_{st} x_s$, real weights $k_{st} > 0$	fire negative	$a = -x$, $C_{ts} = -k_{st}$
Björner and Brenti [BB05] [S2]	neighbour s' gains $k_{s,s'} p_s$; defines positive and negative moves	theorem stated for positive play	$a = p$, $C_{s's} = -k_{s,s'}$
Proctor [Pro84] [S2]	Weyl reflections on weights in fundamental-weight coordinates	fire positive	identical
Proctor ([Pro99b] section 15) [S]	restricted game: only nodes labelled -1 are fired, from a start with no positive label	fire -1 only	negate: from a dominant start, fire only nodes equal to $+1$ [U, this document's reading]
Donnelly [Don08], Donnelly and Eriksson [DE08] [S]	$\lambda_j \mapsto \lambda_j - M_{ij} \lambda_i$, "amplitudes" M_{ij}	fire positive	$a = \lambda$, $C = M^\mathsf{T}$
Gashi, Schedler, Speyer [GS11], [GSS12] [S]	$f_i(v)_j = v_j - c_{ij} v_i$	fire negative	$a = -v$, $C = c^\mathsf{T}$; their $\delta \cdot v$ is minus the level of section 2.12
Benkart, Klivans, Reiner [BKR16] [S]	$u'_j = u_j - c_{ij} u_i$	fire negative	$a = -u$, $C = c^\mathsf{T}$ (the same comparison of formulas as in the row above)
Kostant's game ([Ele16], [Che17]) [S]	value c_i replaced by $-c_i + \sum_{j \sim i} c_j$ when $c_i < \frac{1}{2} \sum_{j \sim i} c_j$	"unhappy" vertices only	the mutation game, moves allowed where $a_i < 0$; happy is $a_i = 0$; manic or excited is $a_i > 0$
SageMath; Kac [Kac90]	Cartan matrix $a_{ij} = \langle \alpha_i^\vee, \alpha_j \rangle$		same matrix. Kac numbers the nodes of E_6, E_7, E_8 and G_2 differently ([S2] for E_6, [U] for the rest)
Bourbaki [Bou68]; Humphreys [Hum72]	Cartan matrix $n(\alpha_i, \alpha_j) = \langle \alpha_i, \alpha_j^\vee \rangle$		transpose of C [S2]; same node numbering

Two cautions for anyone comparing with the literature. First, group elements are attached to play sequences in two opposite orders: Donnelly puts the last move on the left, Björner and Brenti the first, so "left" and "right" descents swap between them. The engine records plays as lists of nodes and never needs to choose. Second, the numbers game with real edge weights $k_{ij}k_{ji} = 4\cos^2(\pi/m)$ covers the non-crystallographic Coxeter graphs H_3, H_4, $I_2(m)$; for those boards see section 2.17.

2.10 Wildberger's two texts: names, numbering and misprints

The site uses Bourbaki's names and numbering. A reader who follows Wildberger's paper, manuscript or lectures needs these translations. They were obtained by typing the printed lists of positive roots into `verify.py` and comparing them with the lists the oracle generates [V for every row; golden key `wildberger_lists`].

In [W20] section 9 and [W-MS] pp. 23-29	Is, in Bourbaki's terms	Lists compared
"B_2", "C_2"	B_2, C_2 with the same numbering	both
"B_n", $n \geq 3$: two roads from vertex 1 to vertex 2	C_n with the node order reversed (his vertex k is node $n+1-k$)	$n = 3, 4$
"C_n", $n \geq 3$: two roads from vertex 2 to vertex 1	B_n with the node order reversed	$n = 3, 4$
"F_4" (highest root printed as 2432)	F_4 with the node order reversed; his vertices 3, 4 are long	all 24 roots
"G_2"	G_2, same numbering (vertex 1 short)	all 6 roots
"D_n": path $1, \ldots, n-1$ with vertex 0 joined to vertex 2	D_n with his vertex k equal to node $n-k$ and his 0 equal to node n	$n = 4, 5$
"E_6", "E_7": path $1, 2, \ldots$ with vertex 0 joined to vertex 3	E_n with his path $1, 2, 3, \ldots$ equal to nodes $1, 3, 4, \ldots$ and his 0 equal to node 2	E_6 in both texts, E_7 in [W20]

So for rank at least 3 the letters B and C are exchanged relative to Bourbaki. The same numbering of D and E graphs, with an extra vertex 0 joined to vertex 2 or 3, is used by the 2004 applet on Wildberger's pages (Appendix F). The `mutation-game` Python package states that it follows Wildberger's convention for B and C, but its B_3 is Bourbaki's B_3 (Appendix F).

Misprints that the site must not reproduce [V: with these corrections, and only with them, every printed list equals the computed one]:

Where	Printed	Should read	In
"C_4" list	1211	2211	both texts
"G_2" list	12	21	both texts
Example 8.6 of [W20], p. 20 of [W-MS]	$\beta_{41} = 60$ and the term $-60p_4p_1$	β_{42} and $-60p_4p_2$: the fourth edge joins x_4 to x_2, and only the corrected form is invariant	both texts
"A_4" list	0010 a second time	0011	manuscript only
"D_5" list (p. 27)	1 over 0111 and 1 over 1111, each printed twice	0 over 0111 and 0 over 1111 in place of the first copies	manuscript only

Where	Printed	Should read	In
"E_6" list (p. 28)	0 over 00010, 1 over 01111 and 1 over 11110, each printed twice	0 over 01000, 0 over 01111 and 1 over 01211	manuscript only
"E_6" diagram (p. 28)	vertex 0 drawn above vertex 2	above vertex 3, as the root lists on the same page require	manuscript only [S, seen on the page image]

The published paper corrects the A_4, D_5 and E_6 lists. The introduction of [W03c] prints the dimension of F_4 as 56 for 52 [S].

2.11 The Mutation Conjecture

[W20] (p. 57, unnumbered) states: for any multigraph, every root population consists entirely of Martians or entirely of anti-Martians. The manuscript poses the same statement for simple graphs as "The Mutation Problem" ([W-MS] p. 2), and [W03a] records it as a proposition for simple graphs with the remark that the author knows no direct combinatorial proof. [W20] adds that it "can possibly also be derived from Coxeter group analysis", crediting discussions with Bob Howlett, and asks for a graph-theoretic or combinatorial resolution.

What the site should say:

- **Read literally, with one-way roads allowed, the statement is false.** [W20] defines a directed multigraph as any multiset of directed edges (section 4), so a single road from city 1 to city 2 with no road back is a multigraph in its sense. On that board the plays $(1,0) \to (1,1) \to (-1,1)$ (mutate 2, then 1) reach a mixed population, and the set of root populations is infinite [V]. Sections 5, 6 and 9 of the paper, and the theorems of its section 8, assume a connected bidirected multigraph; section 8 opens with a general directed multigraph and remarks that a one-way edge forces a weight to be zero [S]. Reading the conjecture in the bidirected setting is this specification's interpretation [D].
- For a **bidirected** board the statement is true. A root population is a real root of the Kac-Moody algebra of the generalized Cartan matrix C, and every root of such an algebra is a non-negative or a non-positive combination of simple roots. This holds for every generalized Cartan matrix, symmetrizable or not. [U as to the exact place in [Kac90]; the statement is standard. Checked by the script for every finite type, for the first 2500 or so root populations of every untwisted affine board of rank at most 8 and of ten twisted affine boards, and for H_2, H_3, H_4 [V].]
- What remains open, in Wildberger's sense, is a proof that uses only the game: a combinatorial reason why Martians and anti-Martians never mix.

A useful by-product of the Kac-Moody reading is an exact test for root populations on any bidirected board (oracle function `is_root_population`): a positive root population other than a single Martian has at least one city whose number is positive; a mutation at any such city lowers it and gives a positive root population again; so lower greedily, and the population is a root population exactly when a single Martian is reached without any city going negative and without getting stuck. The test is exact for every generalized Cartan matrix, balanced or not [V against breadth-first search: on every non-negative population of small total on seven boards, three of them without an invariant form, and on the affine boards above. The two facts used are standard; [U] for a reference, Appendix E item 4].

This is the natural "open problem" page of the site (module M3): the reader hunts for a mixed population on boards of their own design, the engine reports that none exists on bidirected boards, and the one-way road shows why the hypothesis matters.

2.12 Affine boards

An **affine board** is a connected bidirected board whose Cartan matrix is singular with all proper principal minors positive. Equivalently its adjacency matrix has spectral radius exactly 2. [W20] Theorem 4.1 classifies

the connected bidirected boards of spectral radius 2 (his List A, seventeen entries, each drawn with its Perron-Frobenius population α_0) and below 2 (List B, the finite types) [S]. Kac's tables of affine diagrams have sixteen entries; judged by the printed Perron-Frobenius labels, List A is the same list with the four-armed star $\tilde{D}_4$ entered separately from the family $\tilde{D}_n$, but the two lists were not compared edge by edge [U].

How the site tells finite, affine and neither apart (oracle function `cartan_class`; section 5.5 gives the algorithm). A board that is not balanced is neither. For a balanced connected board let $m_1, \dots, m_n$ be the leading principal minors of the symmetric matrix $B = DC$. The board is of **finite type** when all of them are positive, **affine** when $m_1, \dots, m_{n-1} > 0$ and $m_n = 0$, and **indefinite** otherwise. [V: on the 32 finite types, on every affine board with at most nine nodes, on 400 random connected boards with two to five nodes and on the connected two-way test boards of the golden data (one of them with ten nodes) this agrees with the definition by all principal minors. The criterion by principal minors, and the fact that finite and affine boards are always balanced, are Kac's; [U] for the references.]

Names. The untwisted affine board of a finite type X_n is written $\tilde{X}_n$ (`A4~` in data; Kac: $X_n^{(1)}$). The other affine boards are the **twisted** ones, with Kac's names $A_{2n}^{(2)}$, $A_{2n-1}^{(2)}$, $D_{n+1}^{(2)}$, $E_6^{(2)}$, $D_4^{(3)}$ (`A2^(2)`, `D4^(3)` in data). Apart from the family $A_{2n}^{(2)}$ they are the untwisted boards of B_n, C_n, F_4, G_2 with every road reversed [V: every board in the oracle's affine catalogue with at most nine nodes is affine, is recognised under its own name, and differs from the others].

For an affine board there are two null vectors, and they play different roles [V for the untwisted affine board of every type of rank at most 8 and for ten twisted boards]:

- **Marks**: the positive integer vector δ with $C\delta = 0$. As a population, δ is fixed by every mutation: every city is exactly balanced by its neighbours. This is Wildberger's Perron-Frobenius population. One Martian in a suitable city, added to δ, gives a root population again; then further copies of δ can be added, so the root populations are unbounded. [W20] Proposition 6.1 and List A2 give, for each entry of List A, a starting city and an explicit mutation word [S]. Computed refinement [V]: on an untwisted affine board $e_x + \delta$ is a root population for **every** city x; on a twisted board this holds for some cities only, and the others need 2δ or 3δ (table below).
- **Comarks**: the positive integer vector c with $c^{\mathsf{T}}C = 0$. The **level** $\ell(a) = \sum_i c_i a_i$ of a position is unchanged by every firing.

On simply laced boards marks and comarks coincide. On the others they differ, and reversing all roads exchanges them [V].

Fate of the numbers game on an affine board, with positive firing:

Level	What happens
$\ell(a) < 0$	every legal play ends, after the same number of moves, at the same position
$\ell(a) = 0$, $a \neq 0$	the game never ends and visits only finitely many positions (it loops)
$\ell(a) > 0$	the game never ends and the numbers grow without bound

Status: for simply laced affine boards this is [GSS12] Theorem 1.2, credited to Eriksson, with the signs reversed because they fire negative numbers [S]. For the other affine boards, twisted ones included, it is this specification's formulation, supported by random play on the 42 boards above [V]: every sampled start of negative level ended, with the same length and end in two random orders of play; no sampled start of positive level had ended or repeated a position after 1500 moves; and from the numbers $a = Cp$ of a root population (which always have level zero) the game returned to an earlier position (on the untwisted boards within $2N$ moves, N being the number of positive roots of the finite board). A reviewer of this specification supplied an argument that covers every affine board. Each legal firing lowers by one the number of positive coroots

that pair positively with the position, so play ends exactly when that number is finite; this is the criterion that $-a$ lies in the Tits cone ([Don08] Theorem 2.15, attributed there to Eriksson [S]). For an affine matrix the Tits cone is the open half-space of positive level together with the multiples of δ, on which every number is zero [U: believed to be Kac's Proposition 5.8, from the reviewer's memory]. So play ends exactly when the level is negative or the position is zero. At level zero the positions reached form an orbit of the finite Weyl group, which is finite, so a position repeats. At positive level the invariant form forbids a repeat. No source was read that states the table for the non-simply-laced boards; until Appendix E item 6 is settled, the site words it for those boards as computed, with this argument beside it. Play detects two of the three fates exactly: a game that has ended, and a position that has come back. That a game of positive level never ends cannot be seen by playing; the engine reports it as "still running at the cap", and the tool adds the sentence about the level.

The 5-cycle is the affine board $\tilde{A}_4$ [V], all marks and comarks equal to 1, and its numbers game is the 1986 Olympiad problem: "the sum is positive" in the Olympiad's convention is "the level is negative" here. The Olympiad problem prescribes no particular numbers. The site's example start is $(-1, 2, -3, 4, -1)$ in the Olympiad's convention, which is $(1, -2, 3, -4, 1)$ in this specification's convention; it ends after 14 moves whatever the order of play [V: all 1230 complete games were enumerated; golden key `pentagon_example`].

Building affine boards from finite ones. Add a node 0 with $\alpha_0 = \delta - \theta$: $C_{0j} = -\langle \theta^\vee, \alpha_j \rangle$ and $C_{j0} = -\langle \alpha_j^\vee, \theta \rangle$. With θ the highest root this gives the untwisted affine board; with θ the highest short root of a board of type B, C, F, G it gives a twisted one ($D^{(2)}_{n+1}$ from B_n, $A^{(2)}_{2n-1}$ from C_n with $n \geq 3$, $E^{(2)}_6$ from F_4, $D^{(3)}_4$ from G_2) [V for rank at most 8]. In the language of the games: play Kostant's game (F13) from any city, let a be the numbers of the final population β, and add a new city that sends a_j roads to city j and receives $\langle \beta^\vee, \alpha_j \rangle$ roads back. On a simply laced board the two counts are equal, they are 0 or 1 (2 on A_1), and "join a new city to the cities that end up with a positive number" is the whole recipe [V for every type and every starting city; golden key `kostant`].

Marks and comarks of some untwisted boards, node 0 first [V]:

Affine board	marks (node 0 first): the frozen population δ	comarks (node 0 first): weights of the conserved level
$\tilde{A}_1$	(1, 1)	(1, 1)
$\tilde{A}_4$	(1, 1, 1, 1, 1)	(1, 1, 1, 1, 1)
$\tilde{B}_3$	(1, 1, 2, 2)	(1, 1, 2, 1)
$\tilde{C}_3$	(1, 2, 2, 1)	(1, 1, 1, 1)
$\tilde{D}_4$	(1, 1, 2, 1, 1)	(1, 1, 2, 1, 1)
$\tilde{E}_6$	(1, 1, 2, 2, 3, 2, 1)	(1, 1, 2, 2, 3, 2, 1)
$\tilde{E}_7$	(1, 2, 2, 3, 4, 3, 2, 1)	(1, 2, 2, 3, 4, 3, 2, 1)
$\tilde{E}_8$	(1, 2, 3, 4, 6, 5, 4, 3, 2)	(1, 2, 3, 4, 6, 5, 4, 3, 2)
$\tilde{F}_4$	(1, 2, 3, 4, 2)	(1, 2, 3, 2, 1)
$\tilde{G}_2$	(1, 3, 2)	(1, 1, 2)

Twisted boards in the golden data [V]. Vectors are in the order of the rows of each matrix; for a board that is the reverse of an untwisted board, that is the order of the untwisted board, node 0 first:

Twisted board	Cartan matrix	marks	comarks	weights d	least k with $e_x + k\delta$ a root population, by city
$A^{(2)}_2$	$\begin{pmatrix} 2 & -4 \\ -1 & 2 \end{pmatrix}$	(2, 1)	(1, 2)	(1, 4)	(1, 2)

Twisted board	Cartan matrix	marks	comarks	weights d	least k with $e_x + k\delta$ a root population, by city
$A_4^{(2)}$	$\begin{pmatrix} 2 & -2 & 0 \\ -1 & 2 & -2 \\ 0 & -1 & 2 \end{pmatrix}$	(2, 2, 1)	(1, 2, 2)	(1, 2, 4)	(1, 1, 2)
$A_6^{(2)}$	path with a double road at each end, both pointing the same way	(2, 2, 2, 1)	(1, 2, 2, 2)	(1, 2, 2, 4)	(1, 1, 1, 2)
$D_3^{(2)}$	$\begin{pmatrix} 2 & -2 & 0 \\ -1 & 2 & -1 \\ 0 & -2 & 2 \end{pmatrix}$	(1, 1, 1)	(1, 2, 1)	(1, 2, 1)	(1, 2, 1)
$D_4^{(2)}$	reverse of $\tilde{C}_3$	(1, 1, 1, 1)	(1, 2, 2, 1)	(1, 2, 2, 1)	(1, 2, 2, 1)
$D_5^{(2)}$	reverse of $\tilde{C}_4$	(1, 1, 1, 1, 1)	(1, 2, 2, 2, 1)	(1, 2, 2, 2, 1)	(1, 2, 2, 2, 1)
$A_5^{(2)}$	reverse of $\tilde{B}_3$	(1, 1, 2, 1)	(1, 1, 2, 2)	(1, 1, 1, 2)	(1, 1, 1, 2)
$A_7^{(2)}$	reverse of $\tilde{B}_4$	(1, 1, 2, 2, 1)	(1, 1, 2, 2, 2)	(1, 1, 1, 1, 2)	(1, 1, 1, 1, 2)
$E_6^{(2)}$	reverse of $\tilde{F}_4$	(1, 2, 3, 2, 1)	(1, 2, 3, 4, 2)	(1, 1, 1, 2, 2)	(1, 1, 1, 2, 2)
$D_4^{(3)}$	$\begin{pmatrix} 2 & 0 & -1 \\ 0 & 2 & -1 \\ -1 & -3 & 2 \end{pmatrix}$	(1, 1, 2)	(1, 3, 2)	(1, 3, 1)	(1, 3, 1)

On an affine board Kostant's game never ends, and the level of every root population, read through $a = Cp$, is zero [V].

2.13 Folding

Let σ be a symmetry of a simply laced board (a permutation of the nodes preserving the edges). Two different constructions are called "folding" in the literature, and they give dual answers. The site must name which one it shows.

(a) Folding the board (orbit sums). Require that no two nodes in the same σ-orbit are joined. The **folded board** has one node per orbit, and

$$A^{\text{fold}}_{O'O} = \text{number of edges from the orbit } O' \text{ into one fixed node of } O.$$

A population or a position that is constant on orbits is called **symmetric**. Acting on all nodes of an orbit, in any order, is a **symmetric move**. Then [V, 200 random symmetric plays per row]:

> symmetric play of the mutation game on the big board is the mutation game on the folded board, and symmetric play of the numbers game on the big board is the numbers game on the folded board, with the same numbers.

In Lie theory this is the folding of Lusztig and Stembridge: the simple roots of the folded system are the orbit sums, and its Weyl group is the subgroup of W commuting with σ ([Ste08] [S]; [Lus93] [S2]; the "orbit Lie algebra" of [FSS96] [S]). Larger orbits become longer roots.

(b) Restricting the roots (orbit averages). Replace every root by its average over the group generated by σ. The averages form a root system of the **dual** type, in which larger orbits give shorter roots; when the board folds, the Cartan matrix of the averaged simple roots is the transpose of the folded Cartan matrix [V], which is one more appearance of section 2.4(b).

The rows computed by the script [V; golden key `folding`]:

Big board, symmetry	Orbits, in the Bourbaki order of the folded board	(a) folded board	(b) averaged roots
A_3, flip	{1,3}, {2}	B_2	B_2 (quasi-split form `su(2,2)`)
A_5, flip	{1,5}, {2,4}, {3}	B_3	C_3 (quasi-split form `su(3,3)`)
A_7, flip	{1,7}, {2,6}, {3,5}, {4}	B_4	C_4 (quasi-split form `su(4,4)`)
D_4, swap of nodes 3 and 4	{1}, {2}, {3,4}	C_3	B_3 (quasi-split form `so(3,5)`)
D_5, swap of nodes 4 and 5	{1}, {2}, {3}, {4,5}	C_4	B_4 (quasi-split form `so(4,6)`)
D_6, swap of nodes 5 and 6	{1}, {2}, {3}, {4}, {5,6}	C_5	B_5 (quasi-split form `so(5,7)`)
E_6, flip $1 \leftrightarrow 6$, $3 \leftrightarrow 5$	{1,6}, {3,5}, {4}, {2}	F_4	F_4 (quasi-split form `e6(2)`)
D_4, rotation $1 \to 3 \to 4 \to 1$	{2}, {1,3,4}	G_2	G_2
A_4, flip	{1,4}, {2,3}	does not fold: an orbit contains an edge	BC_2 (quasi-split form `su(2,3)`)

The general pattern, of which these rows are instances: A_{2n-1} folds to B_n and averages to C_n; D_{n+1} folds to C_n and averages to B_n; E_6 gives F_4 both ways, with long and short exchanged; D_4 with its symmetry of order 3 gives G_2 both ways, with long and short exchanged; A_{2n} does not fold, and its averaged roots form the non-reduced system BC_n. (In rank 2 the names B_2 and C_2 denote the same system, which is why the first row shows B_2 twice.)

Two readings of construction (b):

- For the first four patterns the averaged roots are the root system of the subalgebra fixed by the corresponding symmetry of the Lie algebra ([FSS96] [S]). For A_{2n} this fails: the fixed subalgebra is $\mathfrak{so}(2n+1)$, of type B_n, which uses only part of BC_n [U for a reference; standard].
- For a symmetry of order 2, construction (b) is exactly the Satake diagram with no black nodes and arrows given by σ (section 2.15): the averaged roots are the restricted roots of the quasi-split real form named in the table [V].

SageMath's "folded Cartan types" follow (b) [S]; Wikipedia's "Dynkin diagram" article describes (a) in one sentence and (b) in the next without saying so [S].

A third kind of folding, with the golden ratio, is in section 2.17.

2.14 Geometry: Euclidean coordinates, polytopes, the Coxeter plane

Euclidean coordinates (finite type). Populations live in $\mathbb{Z}^n$ with the invariant form, which is positive definite but not the dot product. To draw, convert to coordinates in which the form is the dot product. One recipe gives the Coxeter plane at the same time [V for every type of rank 2 to 8; oracle function `euclidean_basis`]:

1. $G = D^{1/2}\, C\, D^{-1/2}$ with $D = \mathrm{diag}(d_i)$. G is symmetric and has the eigenvalues of C.
2. Diagonalise: $G = V \Lambda V^{\mathsf{T}}$, eigenvalues in increasing order. They are $2 - 2\cos(\pi m_j/h)$ for the exponents m_j.
3. $y = E\, p$ with $E = \Lambda^{1/2} V^{\mathsf{T}} D^{1/2}$. Then $E^{\mathsf{T}} E = B$, so $y \cdot y' = (p, p')$.
4. A position a of the numbers game is drawn at $y = E\, C^{-1} a$.

Any other factorisation $B = E^{\mathsf{T}}E$ is an equally valid picture, rotated. The simplest is the triangular one (Cholesky; oracle function `cholesky_basis`), which needs no eigenvalues: it puts α_1 along the first axis and α_2 in the plane of the first two axes, at the angle $\pi - \pi/m$ to α_1 with $m = 3, 4, 6$ for $A_{12}A_{21} = 1, 2, 3$, and at a right angle when nodes 1 and 2 are not joined [V]. The kaleidoscope (T6) uses it. Floating point enters only in these two recipes; everything before step 1 is exact.

The Coxeter plane is spanned by the first and last coordinates of y (smallest and largest eigenvalue, exponents 1 and $h-1$). The product of all mutations of one colour class followed by all of the other rotates this plane by exactly $2\pi/h$; no root projects to the origin; and the roots fall on concentric rings, **each holding a multiple of h roots** [V for every type of rank 2 to 8]. The multiple is not always 1, and different roots can project to the same point:

Board	h	ring sizes, from the innermost ring outwards	distinct points
A_2	3	6	6 of 6
A_3	4	8, 4	8 of 12
A_4	5	10, 10	20 of 20
B_2	4	4, 4	8 of 8
B_3	6	12, 6	12 of 18
C_3	6	6, 6, 6	18 of 18
D_4	6	18, 6	12 of 24
D_5	8	8, 16, 8, 8	32 of 40
E_6	12	24, 12, 24, 12	48 of 72
E_7	18	18, 18, 18, 18, 18, 18, 18	126 of 126
E_8	30	30, 30, 30, 30, 30, 30, 30, 30	240 of 240
F_4	12	12, 12, 12, 12	48 of 48
G_2	6	6, 6	12 of 12

Every ring has exactly h roots only for B_2, B_4, B_5, B_7, B_8, C_2, C_3, C_4, C_5, C_6, C_7, C_8, E_7, E_8, F_4, G_2. Different roots share a projected point for A_3, A_5, A_7, B_3, B_6, D_4, D_5, D_6, D_7, D_8, E_6, so a picture of those types shows fewer dots than roots, and the tool must say so. For E_8 there are 8 rings of 30 and all 240 points are distinct [V]. The factor $\Lambda^{1/2}$ matters: projecting with the bare eigenvectors gives a distorted picture in which the rotation is not a rotation. (Source for the rotation: [Cas17] Proposition 3.7 and Theorem 3.11 [S]; [Cox51] [S]. The ring data are in the golden key `types[T].coxeter_plane`.)

Orbit polytopes. Take a dominant position λ with numbers $a_i \geq 0$, and call a node **ringed** when $a_i > 0$. The convex hull of the orbit of λ is a polytope whose combinatorics depend only on which nodes are ringed. This is Wythoff's construction in the language of weights ([Cox35], [CKPS95] [S2]). Everything needed to draw it comes from the board, with integer arithmetic and no convex-hull computation:

- **Vertices**: the orbit of a under free play of the numbers game. There are $|W|/|W_U|$ of them, U the set of unringed nodes.
- **Edges**: the pairs $\{\mu, \nu\}$ obtained from the base edges $\{\lambda, R_i\lambda\}$, i ringed, by applying one group element to both ends [V: this set equals the set of edges of the exact convex hull in the 48 cases described below, and on simply laced boards with equal rings the edges are exactly the pairs of vertices at minimum distance].
- **Faces of every dimension**: a **face type** is a set K of nodes such that every connected component of K contains a ringed node. Its faces have dimension $|K|$, each is the orbit polytope of the sub-board K, and their number is

$$\frac{|W|}{|W_K| \cdot |W_{K'}|}, \qquad K' = \{\text{unringed nodes that are neither in } K \text{ nor joined to } K\}.$$

  [V in two ways. The counts agree with the published face counts of the polytopes in the table below, which were typed into the script as recalled and not read from a source ([U] for the four largest, 1_{22},

2_{31}, 2_{41} and 1_{42}, whose values are less widely quoted). And for 48 polytopes the script builds the convex hull exactly, by brute force over hyperplanes in integer arithmetic, and finds the same number of faces in every dimension: all seven choices of rings on A_3, B_3, C_3, and every choice of rings on A_4, B_4, C_4, D_4, F_4 that gives at most 32 vertices. The rule is of the kind described in [CKPS95] and should be compared with that paper before it is attributed to anyone.]

- **Edge lengths**: the edge from λ to $R_i\lambda$ has squared length $a_i^2(\alpha_i, \alpha_i) = 2d_i a_i^2$ [V]. A **uniform** polytope needs $a_i|\alpha_i|$ to be the same for all ringed nodes. When all ringed nodes have the same length, equal numbers do it: on B_3 the orbit of $(1, 1, 0)$, with rings on the two long nodes, is a uniform truncated octahedron. When rings sit on both long and short nodes the ratio must be irrational (the uniform truncated cube is B_3 with numbers $(0, 1, \sqrt{2})$), so the integer orbit $(0, 1, 1)$ has the right combinatorics and edges of two lengths [V]. The site draws integer orbits exactly and offers "make uniform" as a display option computed in floating point [D].

Root polytopes. The long roots form one orbit, that of the highest root θ, and the short roots another. The convex hull of all the roots is the orbit polytope of θ: on boards of type B, C, F every short root is the midpoint of two long roots, and on G_2 every short root is one third of the sum of two long roots, so the short roots lie inside the hull of the long ones [V]. "The root polytope" on the site therefore means the orbit of θ, with the short roots available as a second set of points. As positions of the numbers game [V for every type; golden keys `numbers_of_highest_root`, `numbers_of_highest_short_root`]:

Board	numbers of θ	numbers of the highest short root
A_n	1 on nodes 1 and n (for A_1: 2 on node 1)	
B_n, $n \geq 3$	1 on node 2	1 on node 1
B_2	2 on node 2	1 on node 1
C_n	2 on node 1	1 on node 2
D_n, E_6	1 on node 2	
E_7	1 on node 1	
E_8	1 on node 8	
F_4	1 on node 1	1 on node 4
G_2	1 on node 2	1 on node 1

Examples with face counts (vertices, edges, 2-faces, and so on) [V]:

Board	numbers	polytope	face counts
A_3	(1, 0, 0)	tetrahedron	(4, 6, 4)
A_3	(0, 1, 0)	octahedron	(6, 12, 8)
A_3	(1, 0, 1)	cuboctahedron	(12, 24, 14)
A_3	(1, 1, 0)	truncated tetrahedron	(12, 18, 8)
A_3	(1, 1, 1)	truncated octahedron (permutohedron)	(24, 36, 14)
B_3	(1, 0, 0)	octahedron	(6, 12, 8)
B_3	(0, 0, 1)	cube	(8, 12, 6)
B_3	(0, 1, 0)	cuboctahedron	(12, 24, 14)
B_3	(1, 1, 0)	truncated octahedron	(24, 36, 14)
B_3	(0, 1, 1)	truncated cube	(24, 36, 14)
B_3	(1, 0, 1)	rhombicuboctahedron	(24, 48, 26)
B_3	(1, 1, 1)	truncated cuboctahedron	(48, 72, 26)
A_4	(1, 0, 0, 0)	5-cell	(5, 10, 10, 5)
B_4	(0, 0, 0, 1)	tesseract	(16, 32, 24, 8)
B_4	(1, 0, 0, 0)	16-cell	(8, 24, 32, 16)
D_4	(1, 0, 0, 0)	16-cell	(8, 24, 32, 16)
D_4	(0, 1, 0, 0)	24-cell (D4 root polytope)	(24, 96, 96, 24)

Board	numbers	polytope	face counts
F_4	(1, 0, 0, 0)	24-cell	(24, 96, 96, 24)
F_4	(0, 0, 0, 1)	24-cell	(24, 96, 96, 24)
E_6	(1, 0, 0, 0, 0, 0)	2_21	(27, 216, 720, 1080, 648, 99)
E_6	(0, 1, 0, 0, 0, 0)	1_22 (E6 root polytope)	(72, 720, 2160, 2160, 702, 54)
E_7	(0, 0, 0, 0, 0, 0, 1)	3_21	(56, 756, 4032, 10080, 12096, 6048, 702)
E_7	(1, 0, 0, 0, 0, 0, 0)	2_31 (E7 root polytope)	(126, 2016, 10080, 20160, 16128, 4788, 632)
E_8	(0, 0, 0, 0, 0, 0, 0, 1)	4_21 (E8 root polytope)	(240, 6720, 60480, 241920, 483840, 483840, 207360, 19440)
E_8	(1, 0, 0, 0, 0, 0, 0, 0)	2_41	(2160, 69120, 483840, 1209600, 1209600, 544320, 144960, 17520)
E_8	(0, 1, 0, 0, 0, 0, 0, 0)	1_42	(17280, 483840, 2419200, 3628800, 2298240, 725760, 106080, 2400)

Two graphs on the same vertices. On an orbit there are two natural sets of edges and they are different. The **move graph** joins μ to $R_i\mu$ for every node i: it records what the game does. The **polytope graph** joins $w\lambda$ to $wR_i\lambda$ for ringed i: it records which vertices are neighbours in space. One comes from multiplying by a generator on one side, the other from multiplying on the other side. On the hexagon of A_2 from $(1, 1)$ each graph has six edges, four are shared and two differ [V]. The Geometry Studio (T10) shows both and lets the reader switch [D].

Facts about E_8 used on the site [V unless marked]: 240 roots, all of norm 2; each root has inner product 1 with exactly 56 others, -1 with 56, 0 with 126 and -2 with its negative, and the 6720 edges of the root polytope 4_{21} join the pairs with inner product 1. Because $\det C = 1$ every weight is an integer population; the fundamental weights are the columns of C^{-1}:

node	population of ω_i	norm	orbit size	polytope
8	$(2, 3, 4, 6, 5, 4, 3, 2)$, the highest root	2	240	4_{21}
1	$(4, 5, 7, 10, 8, 6, 4, 2)$	4	2160	2_{41}
7	$(4, 6, 8, 12, 10, 8, 6, 3)$	6	6720	rectified 4_{21} [U for the name; a reviewer confirmed it from memory, no source was read]
2	$(5, 8, 10, 15, 12, 9, 6, 3)$	8	17280	1_{42}

The orbit sizes of $\omega_1, \ldots, \omega_8$ are 2160, 17280, 69120, 483840, 241920, 60480, 6720, 240. The numbers of lattice vectors of norm 2, 4, 6, 8 are 240, 2160, 6720, 17520 (OEIS A004009 [S]); the last is $17280 + 240$, the orbit of ω_2 together with the doubled roots.

2.15 Satake diagrams

The decorated board. A **Satake diagram** on a finite-type board is a pair (X, τ): a set X of **black** nodes (the rest are **white**) and a symmetry τ of the board with $\tau^2 = 1$ and $\tau(X) = X$. White nodes exchanged by τ are joined by an arrow in the drawing. From the pair, define an involution on populations:

$$\theta = -\,w_X \circ \tau,$$

where τ permutes the entries of a population and w_X is the longest element of the black sub-board, applied as mutations. θ can be played: (1) move each city's population along the arrows of τ; (2) play the word

of w_X, which the reader obtains by playing the numbers game on the black sub-board from all ones and recording the nodes fired (F4); (3) change every Martian into an anti-Martian and back. The engine function is `theta` (section 5.4).

Which pairs are real forms. Let τ be a symmetry of the board. A pair (X, τ) is **admissible** ([Kol14] Definition 2.3 [S]) when

1. $\tau^2 = 1$ and $\tau(X) = X$;
2. on the black nodes, τ agrees with $-w_X$: $w_X(\alpha_i) = -\alpha_{\tau(i)}$ for $i \in X$;
3. for every white node j fixed by τ, the number $\langle \alpha_j, \rho_X^\vee \rangle = \sum_{\beta \in \Phi_X^+} \frac{(\alpha_j, \beta)}{(\beta, \beta)}$ is an integer.

The admissible pairs, up to symmetries of the board, are exactly the real forms of the complex simple Lie algebra of the board, the all-black pair being the compact form ([Kol14], remark after Definition 2.3, citing Araki's table [S]). This was confirmed by enumerating every pair on every board of rank at most 8 and comparing with the catalogue of section 6.4 [V]. The catalogue has 167 records for 166 classes of admissible pairs: on D_4 the records $\mathfrak{so}(2,6)$ and $\mathfrak{so}^*(8)$ are the same class, exchanged by a symmetry of the board (the swap of nodes 1 and 4, for instance) [V]. The numbers of real forms, compact form included: A_1 2, A_2 3, A_3 5, A_4 4, A_5 6, A_6 5, A_7 7, A_8 6; B_n $n+1$; C_2 3, C_3 3, C_4 4, C_5 4, C_6 5, C_7 5, C_8 6; D_4 5, D_5 7, D_6 8, D_7 9, D_8 10; E_6 5, E_7 4, E_8 3; F_4 3; G_2 2. (D_4 has 5 and not 6 because triality identifies $\mathfrak{so}(2,6)$ with $\mathfrak{so}^*(8)$.)

The three conditions are judged independently of one another (oracle function `admissibility`), so that the Studio can show three lights. Examples [V; golden key `satake_examples`]:

Board	black nodes	τ	condition 1	condition 2	condition 3	verdict
A_3	1, 3	identity	holds	holds	holds	admissible: `su*(4)`
A_3	1	identity	holds	holds	**fails**	not admissible
A_3	1, 2	identity	holds	**fails**	holds	not admissible
A_3	2	$1 \leftrightarrow 3$	holds	holds	holds	admissible: `su(1,3)`
A_3	1	$1 \leftrightarrow 3$	**fails**	**fails**	**fails**	not admissible
A_2	1	identity	holds	holds	**fails**	not admissible
C_3	3	identity	holds	holds	**fails**	not admissible
D_4	3, 4	identity	holds	holds	holds	admissible: `so(2,6)` = `so*(8)`
D_4	1, 2, 3, 4	identity	holds	holds	holds	admissible: `so(8)`
D_5	1, 2, 3, 4, 5	identity	holds	**fails**	holds	not admissible
G_2	2	identity	holds	holds	**fails**	not admissible
G_2	1	identity	holds	holds	**fails**	not admissible

What a failed condition 3 means. Take A_2 with node 1 black and no arrow. Conditions 1 and 2 hold, and $\theta = -s_1$ is an involution that permutes the roots [V]. Yet no involution of the Lie algebra $\mathfrak{sl}(3)$ acts as

this θ on the Cartan subalgebra and fixes the root vectors of the black node: the natural candidate, built from the Chevalley involution and a representative of w_X, has order 4 [V, by a 3×3 matrix computation], and the formula of the next paragraphs would give $\dim \mathfrak{k} = 5$, which no real form of $\mathfrak{sl}(3)$ has [V]. So the picture is not the Satake diagram of any real form. The same permutation of the roots does occur for a real form, here $\mathfrak{sl}(3,\mathbb{R})$, on its maximally compact Cartan subalgebra, where the fixed root is imaginary and non-compact. Vogan diagrams are built on such Cartan subalgebras (the Vogan diagram of $\mathfrak{sl}(3,\mathbb{R})$ shows the two nodes exchanged and nothing painted), and the situation is outside the Satake picture [U as a statement about the literature, Appendix E item 15; a reviewer of this specification confirmed the substance from memory]. The engine function `satake` refuses a pair that is not admissible instead of returning numbers for it.

What the diagram does not show. By condition 2, τ may move black nodes, and it may be non-trivial with no arrow drawn at all. Printed tables and the CoReLG software record τ only on white nodes. The cases [V]: τ is the non-trivial symmetry, with no arrow drawn, for $\mathfrak{so}(p,2n-p)$ with $p \le n-2$ and $n-p$ odd, and for the compact forms of A_n ($n \ge 2$), D_n (n odd) and E_6; τ moves black nodes in addition to the drawn arrows for $\mathfrak{su}(p,q)$ with $q-p \ge 3$ and for $\mathfrak{e}_{6(-14)}$. The engine stores the full τ; the drawing shows arrows between white nodes only; the Studio has a "show τ on black nodes" toggle [D].

Shadows: restricted roots. For a root α put

$$\bar{\alpha} = \tfrac{1}{2}(\alpha - \theta\alpha).$$

The roots with $\bar{\alpha} = 0$ are exactly the roots of the black sub-board. The nonzero $\bar{\alpha}$ are the **restricted roots**; they form a root system Σ, possibly the non-reduced BC_r. The **multiplicity** of a restricted root is the number of roots with that shadow ([DK19] [S]). It depends only on the length of the restricted root. (Both statements are asserted inside the oracle function `satake` for every record it computes [V].) In the engine, shadows are kept as the integer vectors $2\bar{\alpha}$ (oracle function `shadow2`).

Read off the board [V for all 167 records of the catalogue]:

- real rank r = number of τ-orbits of white nodes (checked as the dimension of the span of the shadows);
- simple restricted roots = shadows of the white simple roots, one per τ-orbit; for each, the pair $(m_\lambda, m_{2\lambda})$ of multiplicities of the simple restricted root λ and of its double (zero unless the system is BC_r, where exactly one simple restricted root has a double) [V; golden field `restricted_simple_multiplicities`];
- $\dim \mathfrak{k} = (n-r) + \frac{1}{2}(|\Phi| + |\Phi_X|)$, $\dim \mathfrak{p} = r + \frac{1}{2}(|\Phi| - |\Phi_X|)$, where Φ_X is the set of roots of the black sub-board;
- character $= \dim \mathfrak{p} - \dim \mathfrak{k} = 2r - n - |\Phi_X|$. This closed form is derived here from two printed identities and was not found stated in a source, so the site presents it as a derived formula [D].

Special shapes: all white with no arrows is the split form ($\theta = -1$, every multiplicity 1); all black is the compact form ($\theta = 1$ on roots, no restricted roots); no black nodes with arrows is a quasi-split form and its shadows are the averaged roots of section 2.13(b).

The restricted game. For a τ-orbit O of white nodes define the composite move

$$\tilde{s}_O \;=\; w_{X\cup O} \cdot w_X,$$

played on the big board as the word of w_X followed by the word of the longest element of the sub-board $X \cup O$. Then [V for the 135 catalogue records of positive real rank]: $\tilde{s}_O$ commutes with θ, and it acts on shadows as the reflection in the simple restricted root of O. So the shadows carry a mutation game of their own, on the **restricted board** whose Cartan matrix is $\bar{C}_{OO'} = 2(\bar{\alpha}_O, \bar{\alpha}_{O'})/(\bar{\alpha}_O, \bar{\alpha}_O)$, and each move of that small game is a fixed sequence of moves of the big game. For BC_r the restricted board is B_r (and A_1 for $r = 1$), with the doubled roots recorded as a second multiplicity. The reduced length of $\tilde{s}_O$ is $\ell(w_{X\cup O}) - \ell(w_X)$. (Source: [DK19] equation (2.8) and Theorem 2.4, credited there to Lusztig [S].)

Example [V, by a named check]: for $\mathfrak{e}_{6(-26)}$, $X = \{2,3,4,5\}$ and $\tau = 1$. The word of w_X has 12 letters (the longest word of D_4), each of the two composite moves has reduced length 8, the restricted board is A_2, and every restricted root has multiplicity 8.

The restricted board comes out of the computation with its nodes in the order of the smallest node of each orbit, which need not be the Bourbaki order of its type. The golden field `restricted_orbits_in_bourbaki_order` lists the orbits in Bourbaki order, and the site draws the restricted board in that order.

Other encodings, for a sidebar: Vogan diagrams (built from a maximally compact Cartan subalgebra; [Kna02] VI.8) and the Tits index. The site should say that the same undecorated Dynkin diagram means the split form as a Satake diagram and the compact form as a Vogan diagram [S2].

Generalized Satake diagrams ([RV20]) replace condition 3 by: the connected component of $X \cup \{j\}$ containing a τ-fixed white node j is not of type A_2. They are an optional advanced switch. Their shadows need not form a root system: on G_2 with the long node 2 black the shadows are $\pm\lambda, \pm 2\lambda, \pm 3\lambda$ with multiplicities 2, 1, 2, and with the short node 1 black they are $\pm\lambda, \pm 2\lambda$ with multiplicities 4, 1, which looks like BC_1 although $\mathfrak{g}_2$ has no real form of real rank 1 [V; golden key `generalized_satake_example`]. The type-recognition code must therefore refuse pairs that fail condition 3.

2.16 Neighbouring games, and what "mutation" does not mean

Kostant's game is the mutation game with a ratchet (F13). Its three moods are the three signs of the number a_i of section 2.4: unhappy $a_i < 0$, happy $a_i = 0$, manic or excited $a_i > 0$. On a simply laced board of finite type its state graph, over all starting nodes, is the Hasse diagram of the root poset. Off the finite types a raising move can raise a city by any positive amount, even on a simple graph (on the triangle, $(1,1,0) \to (1,1,2)$).

Chip-firing is a different game. A node fires when its pile reaches a threshold (its degree), loses a fixed amount, and the total is conserved; firings commute. In the numbers game a node fires according to the sign of its number, passes on an amount proportional to that number, and is negated; moves at joined nodes do not commute, and on a finite-type board no weighted sum of the numbers is conserved (what is conserved is the form value of the position). The two agree, after the substitution $v = 1 - u$, in the minuscule situation where every number fired equals 1 ([BKR16] Remark 4.11 [S]).

Quiver representations. Put a vector space on each node of an oriented simply laced board and a linear map on each edge. At a node where all arrows point in, a reflection functor replaces the space; the dimensions change by exactly the mutation rule, and the arrows at that node turn around ([BGP73] Theorem 1.1 [S]). Gabriel's theorem: the indecomposable representations of an *A-D-E* quiver correspond to the positive root populations. A quiver arrow is an orientation; Wildberger's unequal road counts are lengths. The site must keep the two meanings of "arrow" apart.

Cluster mutation is a different operation with the same name. This needs one clearly placed page, because the project note "algebraic data" and at least one piece of existing software documentation blur the two.

	Wildberger's mutation	Fomin-Zelevinsky mutation
What you hold	one integer per city	one variable per vertex, and the quiver
What a move changes	one integer	one variable **and the quiver**
The diagram	never changes	always changes: the arrows at the vertex are reversed, and arrows between its neighbours appear or cancel unless the vertex is a source or a sink
Rule	new = −old + sum over incoming roads	old × new = product over in-neighbours + product over out-neighbours
The moves generate	the Weyl group	no group; an exchange graph

	Wildberger's mutation	Fomin-Zelevinsky mutation
Finite when	the board is a Dynkin diagram	some quiver in the mutation class is an oriented Dynkin diagram
What gets enumerated	all $2N$ roots	the $N+n$ almost positive roots

(Sources: [FZ02] Definition 4.2; [FZ03a] Theorems 1.4 and 1.9 [S].) The smallest illustration: mutating the quiver $1 \to 2 \to 3$ at vertex 2 creates an arrow from 1 to 3 and turns the path into a triangle, while the board of the mutation game on A_3 is the same path forever.

2.17 Golden boards: H_2, H_3, H_4

The project brief says "more generally a Coxeter graph". The finite connected Coxeter graphs that are not Dynkin diagrams are H_3, H_4 and the polygons $I_2(m)$ with $m = 5$ or $m \geq 7$ ($I_2(3)$, $I_2(4)$, $I_2(6)$ are A_2, B_2, G_2; $I_2(5)$ is also called H_2). The numbers game extends to them with real road weights whose products are $4\cos^2(\pi/m)$ ([Eri95b]; [Don08] section 2 [S]). For the label 5 the weight is the golden ratio φ, and all arithmetic stays exact in $\mathbb{Z}[\varphi]$, the numbers $a + b\varphi$ with integers a, b and $\varphi^2 = \varphi + 1$.

Boards. H_n ($n = 2, 3, 4$; $H_2 = I_2(5)$) is the path $1-2-\cdots-n$ with symmetric matrix C: 2 on the diagonal, -1 on ordinary edges, $-\varphi$ in both directions on the last edge. Both games are defined by the formulas of sections 2.2 and 2.3 with this C [D]. Facts [V]:

Board	root populations	order of the group	length of the game from all ones
H_2	10	10	5
H_3	30	120	15
H_4	120	14400	60

Every root population is all ≥ 0 or all ≤ 0 (signs taken as real numbers), $CS_i = R_iC$ holds, and the game from all ones ends at all minus ones.

Golden folding. There is a third kind of folding, in which a pair of nodes of a simply laced board becomes one node carrying a number of $\mathbb{Z}[\varphi]$. For each pair one node is "plain" and one is "golden", and

$$\text{value on the } H \text{ board } = \text{ (value at the plain node)} + \varphi \cdot \text{(value at the golden node)},$$

for populations and for positions alike. The two nodes of a pair are never joined, and acting on both is one move of the H game [V for both games]:

Big board	H board	pairs (plain, golden), in the order of the H nodes
A_4	H_2	$(1,3)$, $(4,2)$
D_6	H_3	$(1,5)$, $(2,4)$, $(6,3)$
E_8	H_4	$(8,1)$, $(7,3)$, $(6,4)$, $(2,5)$

Under the fold the roots of the big board go onto the roots of the H board together with their multiples by φ: $240 = 120 + 120$ for E_8 [V]. Seen in Euclidean coordinates this is the projection of the 240 roots of E_8 onto the 4-dimensional space belonging to the exponents 1, 11, 19, 29, where they form two 600-cells whose radii are in the ratio φ [V]. An exhaustive search over all ways of pairing non-adjacent nodes finds exactly one golden folding of A_4, one of E_8, and two of D_6 that differ by the symmetry of the board [V].

The construction is classical (it is usually credited to Lusztig, to Shcherbak and to Moody and Patera [U; no source was read for it in this research]).

Arithmetic for the engine: $(a+b\varphi)(c+d\varphi) = (ac+bd)+(ad+bc+bd)\varphi$, and the sign of $a+b\varphi$ is the sign of $(2a+b)+b\sqrt{5}$, decided exactly by comparing $(2a+b)^2$ with $5b^2$ when the two terms have opposite signs. Other polygons $I_2(m)$ are a floating-point display feature only [D].

2.18 From the games to Lie algebra matrices

The complex simple Lie algebra of a board is generated by elements E_i, F_i, H_i, one triple per node, subject to the Chevalley-Serre relations

$$[H_i, H_j] = 0, \quad [H_i, E_j] = C_{ij}E_j, \quad [H_i, F_j] = -C_{ij}F_j, \quad [E_i, F_j] = \delta_{ij}H_i,$$
$$\mathrm{ad}(E_i)^{1-C_{ij}}E_j = 0, \qquad \mathrm{ad}(F_i)^{1-C_{ij}}F_j = 0 \quad (i \neq j).$$

The numbers game shows the weights of a representation and how the Weyl group moves them. Three facts connect the game to the generators.

Strings. In any finite-dimensional representation the weights through μ in the direction α_i form an unbroken string $\mu - r\alpha_i, \ldots, \mu + q\alpha_i$ with $r - q = a_i$, the number on node i. Firing node i is the reflection of the string end over end. F_i and E_i are single steps down and up the string. In Dynkin labels a single step subtracts or adds column i of C. (Standard; [Hum72] sections 9.4 and 21.3 [U for the section numbers].)

Minuscule representations: the game graph is the representation. Let ω_i be minuscule (F11). Take one basis vector for each position in the orbit of ω_i, and define on a position μ:

- H_k multiplies by the number μ_k;
- F_k sends μ to the fired position $R_k\mu$ when $\mu_k = 1$, and to 0 otherwise;
- E_k sends μ to $R_k\mu$ when $\mu_k = -1$, and to 0 otherwise.

These matrices of zeros and ones satisfy all the Chevalley-Serre relations [V for 18 representations: five minuscule nodes of A_1 to A_4; the spin representations of B_2, B_3, B_4; the vector representations of C_2, C_3; all three minuscule nodes of D_4; nodes 1 and 5 of D_5; the two 27s of E_6 and the 56 of E_7]. The firing move itself is the permutation matrix $E_k + F_k + P_k^0$, where P_k^0 projects onto the positions with $\mu_k = 0$. The group element $\exp(E_k)\exp(-F_k)\exp(E_k)$ equals $E_k - F_k + P_k^0$: the same matrix with one sign changed in each pair, and its square is $(-1)^{H_k}$ [V]. So in a minuscule representation a game move is a Lie group element with its signs removed. For a node that is not minuscule the recipe fails (checked on ω_2 of D_4 [V]).

This is the content of Wildberger's construction for simply laced algebras, in which the Lie algebra is built on the ideals of a minuscule heap ([W03a]; his Theorem 4.1 is stated without proof, and the proofs are in the later literature, [Gre13]). E_8, F_4 and G_2 have no minuscule node, which is why E_8 is the case his method leaves out.

One zero weight. Some representations have as weights one orbit together with the zero weight, once: the vector representation of B_n (dimension $2n+1$), the 7-dimensional representation of G_2, the adjoint of A_1. For these, take the positions of the orbit of the highest weight and one extra basis vector for the zero position, let H_k multiply by μ_k, and let E_k and F_k move one step (add or subtract column k of C) with coefficient 1, except that the two steps that **leave** the zero position have coefficient 2. The Chevalley-Serre relations hold [V for A_1, B_2, B_3, B_4 and G_2]. This gives the site a route to G_2 that stays inside the game, in the spirit of [W03c], whose own construction uses a different basis with entries between -2 and 2.

Out of reach of these two recipes: every other representation. That includes representations whose weights form several orbits, even when no weight is repeated, and representations with a repeated weight, such as the 26 of F_4 (the subject of [vE]) and the adjoint representations in rank at least 2, including the 248 of E_8. The site shows their weights (for the adjoint: roots and zeros) and says plainly that the matrices need more than the game [D].

3. Curriculum

Eleven modules. M0 to M5 form the core path and are written for the primary reader. M6 to M9 can be read in any order after M5. M10 is reference. Every module follows the page anatomy of section 8.2: a question, a concrete board to touch, guided steps with predictions, the general statement, a "for readers who know Lie theory" paragraph, challenges, and a sandbox link. Tool numbers refer to section 4. The "beats" are the order of ideas; each beat is one or two paragraphs with one figure, and the table after the beats says what that figure is. In those tables "T1" and so on name a tool in the stated configuration; "p" is a start given as a population and "a" a start given as numbers (section 6.6); "static" is a drawing with no controls. The configurations are a starting point for the lesson author and may be refined [D]; the numbers in them are asserted by section 19 of `verify.py` [V].

The phase in which each module is written (section 9): M0, M1 in phase 1; M2, M3 in phase 2; M4, M5 in phase 3; M6, M7 in phase 4; M8 in phase 5; M9, M10 in phase 6.

M0. Start here: two games, one board

Question: what can you learn from the simplest possible moves on a small graph? (About five minutes. Outcomes LO1, LO2 in miniature.)

Beats:

1. Three cities in a row, the board A_3, with one Martian in the middle city. The reader mutates city 1 and sees it become 1. The rule in one sentence.
2. A prediction: "city 2 now holds 1 and its neighbours hold 1 and 0. What will it hold after it mutates?" The answer $-1 + 1 + 0 = 0$ is revealed after the reader commits.
3. The same board with the Numbers view switched on. The reader fires a node and sees the number flip while its neighbours absorb it.
4. The reveal: the two boards are linked. Each number is the amount by which a mutation would lower that city. One move, two records.
5. A map of the site as the seven statements of section 1.4, each linking to its module.

Beat	Figure
1	T1, A_3, populations, free play, p $= (0, 1, 0)$; goal "mutate city 1"; result $(1, 1, 0)$
2	T1, A_3, populations, p $= (1, 1, 0)$, prediction (a number) for city 2; answer 0
3	T1, A_3, numbers, free play, a $= (0, 1, 0)$; firing node 2 gives $(1, -1, 1)$
4	T1, A_3, linked, free play, p $= (0, 1, 0)$, which shows the numbers $(-1, 2, -1)$; mutating city 2 lowers it by 2
5	static: the seven statements as a list of links

Sources: [W20] section 2 for the story; section 2.4 for the link.

M1. The mutation game

Question: starting from one Martian, which populations can ever occur? (LO1, LO3.)

Beats:

1. The Mars story and the rule on a simple graph. Worked play on A_2: from $(1, 0)$ the reader finds exactly six populations.

2. Root populations, simple root populations, and the first observation: Martians and anti-Martians never share a board. The reader is asked to try to break this on A_3 and D_4.
3. Mutations that raise, lower, or do nothing. The reader plays Kostant's game (T9): raise whenever possible, and see where it stops. On A_3 the end is $(1,1,1)$ after 2 moves; on D_4 it is $(1,2,1,1)$ after 4; on E_8 it is $(2,3,4,6,5,4,3,2)$ after 28, from any starting city [V].
4. The Orbit Garden (T2) grows the whole set: 12 root populations on A_3, 24 on D_4, 240 on E_8. Arranged by total population, the positive ones form the root poset.
5. A board where the garden never stops growing (a triangle, or a star with four arms), as a cliffhanger for M3.

Beat	Figure
1	T1, A_2, populations, free play, p $= (1,0)$, with T2 beside it in "follow" mode counting the populations found (6)
2	T1, A_3 then D_4, populations, free play, p a single Martian; goal "make a city negative while another is positive" (cannot be met; the caption says so after 20 moves)
3	T9 on A_3 (p $= (1,0,0)$), then D_4, then E_8
4	T2, "explore" mode, on A_3; a board chooser offers D_4 and E_8; then "positive roots only"
5	T2, "explore" mode, on the triangle (a preset custom board), 40 layers: 243 root populations and still growing

For readers who know Lie theory: populations are elements of the root lattice in simple-root coordinates, a mutation is a simple reflection, and the root populations are the roots. Challenges: find all 12 root populations of A_3; reach the highest root of E_6 by raising moves (every such play takes exactly 10 moves); find a population that no single mutation changes (on a finite board there is none except 0; on the triangle there are $(1,1,1)$ and its multiples). Sources: [W20] sections 2, 5, 9; [W03a] section 3; F1, F13.

M2. The form behind the game

Question: is anything conserved when a city mutates? (LO4, first half.)

Beats:

1. An "energy meter" (T3) shows $2\sum p_i^2 - 2\sum_{\text{roads}} p_i p_j$ while the reader plays on A_3. It never moves. Every root population reads 2.
2. The reader builds the form on a new board by adjusting weights until the meter holds still, and so discovers that the recipe is forced.
3. The matrix of the form is $2I - A$: the Cartan matrix of a simple graph. A mutation is a reflection for this form.
4. Lengths and angles: two joined cities are vectors of the same length at 120°; unjoined cities are perpendicular. The picture of A_2 with its six roots.
5. On a simply laced board of finite type the root populations are exactly the integer populations of form value 2 (F2). The reader tests candidates. (The statement is for finite type. On the triangle with a tail of two more cities, $(1,1,1,0,1)$ has form value 2 and is not a root population [V].)

Beat	Figure
1	T3 meter with T1, A_3, populations, free play, p $= (1, 0, 0)$
2	T3 builder on A_2: three sliders (two city weights, one road weight); passes exactly when all three are equal
3	static: the matrix $2I - A$ for A_3 beside the board
4	static (phase 2); replaced by T6 on A_2 once that tool exists (phase 3)
5	T3 "small values" on A_3: the reader types populations; the tool reports the form value and whether it is a root population

For readers who know Lie theory: this is the Tits form of the graph, equivalently the Killing form restricted to the root lattice up to scale. Sources: [W20] section 7 (Proposition 7.1), [W03a] Lemma 3.2 and Proposition 3.5.

M3. Finite, affine, wild

Question: on which boards does the game close up? (LO4.)

Beats:

1. The Classification Lab (T4): the reader draws graphs and watches the count of root populations. Paths, forks and three special trees close up; everything else runs away.
2. The reason: the game is finite exactly when the form is positive definite, exactly when the spectral radius of the board is below 2. The list of such simple graphs is A_n, D_n, E_6, E_7, E_8 ([W20] Theorems 4.1 and 8.8 [S]).
3. The boundary cases, spectral radius exactly 2. Each has a **frozen population** that no mutation changes: twice each city equals the sum of its neighbours. The reader finds it on a cycle (all ones) and on the star with four arms $(2; 1, 1, 1, 1)$.
4. From a frozen population to runaway growth: one Martian in a suitable city plus the frozen population is again a root population ([W20] Proposition 6.1; on the boards of this lesson any city will do), so the roots never end.
5. How to manufacture the boundary boards: play Kostant's game on a finite board and join a new city to the cities that end with a positive number (section 2.12). On D_4 that is city 2, and the new board is the star with four arms.
6. The Mutation Conjecture page (section 2.11): the open question, the counterexample with a one-way road, and a hunter that searches the reader's own boards.
7. A short note on why the same list of graphs appears across mathematics, with links ([HHSV77]).

Beat	Figure
1	T4 with the path A_3 loaded and presets for D_4, E_6, the triangle, the star with four arms
2	T4 verdict panel on the same boards (class, balance, spectral radius), with T3 "small values" beside it for the form values
3	T15 "frozen population" on the square (the 4-cycle), then on the star with four arms; populations view; every city is happy

Beat	Figure
4	T1, star with four arms, populations, free play, p = frozen population plus one Martian; T2 beside it, capped
5	T9 on D_4 with the caption about the new city; a button "add the city" opens T4 with the star
6	T4 with the conjecture hunter switched on; presets: the one-way road, a reader-drawn board
7	static

For readers who know Lie theory: finite, affine and indefinite generalized Cartan matrices; marks; imaginary roots. Sources: [W20] sections 4, 6, 8; [DE08]; [Pro93].

M4. The numbers game and the duality

Question: what does the same move look like from the other side? (LO1, LO2, start of LO5.)

Beats:

1. The rule: a node sends its number down its roads and changes sign. Legal play fires positive numbers. On A_2 from $(1, 1)$ the game lasts three moves (Appendix A).
2. A prediction on A_3 from $(1, 1, 1)$: "will a different order of moves give a different ending, or a different number of moves?" The reader plays twice. Six moves, end $(-1, -1, -1)$, every time (F3, F4).
3. The linked board (T1): populations above, numbers below, joined by $a = Cp$. The sentence of section 2.4(a). The reader verifies it on three moves.
4. The kaleidoscope (T6) on A_2: two mirrors, a point, its two sets of coordinates. Firing is reflecting.
5. Not every position is a population: on A_2 the position $(1, 0)$ would need $\frac{2}{3}$ and $\frac{1}{3}$ of a Martian. The numbers game has $\det C$ times as many positions (section 2.4(c)). The orbit of $(1, 0)$ has three positions and every number in it is -1, 0 or 1: a first minuscule orbit.
6. Where the game came from: the pentagon problem of the 1986 Olympiad, as a puzzle to try now.
7. Why the pentagon game ends, and when such a game does not: on the boundary boards of M3 a weighted sum of the numbers, the **level**, never changes, and its sign decides between three fates (section 2.12).

Beat	Figure
1	T1, A_2, numbers, legal play, a = $(1, 1)$
2	T1, A_3, numbers, legal play, a = $(1, 1, 1)$, prediction (a choice); then T5 "two games compared"
3	T1, A_3, linked, free play, p = $(1, 1, 0)$, which shows the numbers $(1, 1, -1)$
4	T6 on A_2, point at a = $(1, 1)$
5	T1, A_2, linked, a = $(1, 0)$: the populations board shows $\frac{2}{3}$, $\frac{1}{3}$; T2 "follow" mode beside it (3 positions)
6	T15 "pentagon", in the Olympiad's convention, with the example start of section 2.12
7	T15 "three fates" on the pentagon: the reader changes one number and watches the level meter and the outcome

For readers who know Lie theory: positions are weights in fundamental-weight coordinates, legal play descends from the dominant chamber, and P/Q is the fundamental group of the adjoint group. Sources:

[Moz90] (not read; see `SOURCES.md`), [Eri95b], [Don08], [GSS12], [W-MS] pp. 30-32, [W03c] section 2.

M5. The Weyl group inside the games

Question: what is being counted when a game always lasts the same number of moves? (LO5.)

Beats:

1. Positions are group elements. From $(1, 1, 1)$ on A_3 legal play reaches 24 positions, one for each element of the group, which here is the group of permutations of four things (T8).
2. A play is a word; a legal play is a shortest word (F5). The longest element, and why the generic game has N moves: each move uses up one positive root.
3. How many different games? 2, 16, 768 for A_2, A_3, A_4; 42 for B_3; 2316 for D_4 (F12).
4. Heaps: a play up to the order of independent moves is a stack of bricks (T8). Wildberger calls the stack a frame ([W-MS] pp. 45-46). The 16 games on A_3 make 8 different stacks [V].
5. The fired numbers. With letters in place of numbers (T7), the game from (x_1, x_2, x_3) fires x_1, $x_1 + x_2$, and so on: one linear form for each positive root (F6).
6. From ρ the fired numbers are heights. Their tally, read sideways, gives the exponents, and the exponents give the order of the group: for E_8 the value 1 is fired eight times and the value 29 once, the exponents are 1, 7, 11, 13, 17, 19, 23, 29, and $2 \cdot 8 \cdot 12 \cdot 14 \cdot 18 \cdot 20 \cdot 24 \cdot 30 = 696729600$ (F7, T5).
7. The wave: fire alternate colour classes; the game ends after exactly h rounds (F8).
8. Weyl's dimension formula as the ratio of two games (F9).

Beat	Figure
1	T8 on A_3, with T1 (A_3, numbers, legal play, a $= (1, 1, 1)$) driving it
2	T8 on A_3: hovering a vertex shows a shortest word and the separating mirrors
3	T8 counter on A_2, A_3, B_3, D_4
4	T8 heap view on A_3 with the plays 1,3,2 and 3,1,2 offered as presets
5	T7 on A_3
6	T5 tally on A_3 (hand-sized), then on E_8 with "play to the end"
7	T5 wave on A_3 (4 rounds) and E_8 (30 rounds)
8	T5 dimension calculator on A_2 with $\lambda = (1, 1)$ (answer 8)

For readers who know Lie theory: weak order, inversion sets, the Shapiro-Steinberg-Kostant-Macdonald theorem, the bipartite Coxeter element. Sources: [BB05] section 4.3, [Don08], [Ste59], [Ste96], [FZ03b] Lemma 2.1.

M6. Multiple edges: B, C, F, G and folding

Question: what changes when two cities are joined by more roads one way than the other? (LO6.)

Beats:

1. One-way counts. On the board with two roads from city 1 to city 2 and one back (this is B_2), the reader finds 8 root populations, of two kinds.
2. Orders 4 and 6: the pair of moves at a double or triple road repeats after 4 or 6 rounds ([W20] Theorem 5.1).
3. Balanced boards: cities get sizes so that the form can still be conserved; the sizes are the squared lengths (section 2.5). More roads run from the long city to the short one.

4. The two games stop looking alike: the numbers game on a board is the mutation game on the reversed board, transposed (section 2.4(b)). The fired numbers of M5 are heights of the roots of the reversed board. B_n and C_n are each other's reversals.
5. Folding (T11): play symmetrically on A_3 and you are playing on B_2; on D_4 with its threefold symmetry you are playing on G_2; E_6 folds to F_4 (section 2.13(a)).
6. The other folding: average the roots instead, and get the dual type. A pointer forward to M9.
7. A warning box on names: for $n \geq 3$, Wildberger's B_n and C_n are Bourbaki's C_n and B_n (section 2.10).

Beat	Figure
1	T1, B_2, populations, free play, roads drawn one by one (`roads` style), p $= (1, 0)$; T2 beside it (8 populations)
2	T1 on B_2 and G_2 with a scripted alternation of the two moves and a round counter
3	T3 builder on B_2: passes exactly for city weights in the ratio $2 : 1$
4	T7 on B_2 beside the list of positive roots of B_2 (the lists differ: $2x_1 + x_2$ against $(1, 2)$)
5	T11 on A_3, then D_4 (rotation), then E_6
6	T11 "average" tab on A_5: folded board B_3, averaged roots C_3
7	static table from section 2.10

Sources: [W20] sections 4, 5, 8, 9; [Ste08]; [FSS96].

M7. Geometry: mirrors, polytopes and the E8 picture

Question: what shapes do the positions make? (LO7.)

Beats:

1. Change of coordinates: from integer populations to points in ordinary Euclidean space (section 2.14). The hexagon of A_2; the 12 roots of A_3 as a cuboctahedron; the 24 roots of D_4 as a 24-cell in projection.
2. Wythoff's construction on the board: put a positive number on some nodes ("ring" them), zero on the rest, and take the orbit. From A_3: tetrahedron, octahedron, cuboctahedron, truncated tetrahedron, truncated octahedron. From B_3: cube, octahedron and five more (table in section 2.14).
3. Counting faces from the board alone, with the rule of section 2.14.
4. Two graphs on one set of points: what the game does and what is adjacent in space.
5. Uniform or not: on B_3 equal integers on nodes 2 and 3 give a truncated cube with two edge lengths; the uniform one needs the ratio $\sqrt{2}$.
6. The Coxeter plane and the picture of E_8: 240 points on 8 rings of 30, each joined to its 56 nearest neighbours. On other boards a ring holds a multiple of h points, and points can coincide.
7. E_8 as a lattice: every position is a population; shells of 240, 2160, 6720, 17520.
8. (Phase 7) Golden boards: H_3, H_4, and E_8 folding onto H_4 as two 600-cells (section 2.17).

Beat	Figure
1	T10, root polytope of A_2, then A_3, then D_4
2	T10 on A_3 with clickable rings, starting from $(1, 0, 0)$
3	T10 face table on B_3 with rings $(1, 0, 1)$: 24, 48, 26

Beat	Figure
4	T10 on A_2 with rings $(1,1)$, switch between the two graphs (four shared edges)
5	T10 on B_3 with rings $(0,1,1)$ and the "make uniform" option
6	T10 showpiece: E_8, root polytope, Coxeter plane; then D_4 in the Coxeter plane (12 dots for 24 roots)
7	static table of the four shells with T10 links
8	T11 golden tab (phase 7)

Tools: T10, T11. Sources: [Cox35], [Cox73], [CKPS95], [Cas17], [Ste59].

M8. From games to Lie algebras

Question: if a move is a whole reflection, what is half a move? (LO8.)

Beats:

1. Strings: at a position, the number on node i says how the position sits in a line of positions spaced by α_i; firing flips the line end over end (section 2.18).
2. Single steps up and down, E_i and F_i, and the counting operator H_i. The smallest case, A_1 with numbers 1 and -1.
3. The three positions of A_2 from $(1,0)$ give 3×3 matrices. The reader checks $[E_1, F_1] = H_1$ and $[E_1, F_2] = 0$ by clicking (T12).
4. The same recipe on the 8 positions of D_4, the 27 of E_6 and the 56 of E_7. All the defining relations hold [V].
5. A move as a group element: $\exp(E)\exp(-F)\exp(E)$ is the move with signs.
6. Why E_8 is different: it has no minuscule orbit; its smallest representation is the 248-dimensional one built on the 240 roots and 8 zeros.
7. (Phase 7) One zero weight: the 7 of G_2 and the vector representations of type B.
8. Sidebars: reflection functors and Gabriel's theorem; "what mutation is not" (section 2.16).

Beat	Figure
1	T12 string viewer on A_2: the string through $(1,0)$ at node 1 has two positions; the string through $(2,0)$ has three, $(2,0)$, $(0,1)$, $(-2,2)$, and its middle position lies outside the orbit of $(2,0)$
2	T12 on A_1, node 1
3	T12 on A_2, node 1, relation checker
4	T12 on D_4 node 1, then E_6 node 1, then E_7 node 7
5	T12 on A_2: the move matrix beside its signed version
6	static: the 240 roots of E_8 and 8 zeros, with a link to T10
7	T12 "one zero weight" on G_2 (phase 7)
8	static, with the table of section 2.16

Sources: [W03a], [W03c], [Gre13], [vE], [BGP73], [FZ02].

M9. Real forms and Satake diagrams

Question: the same board describes several different real Lie algebras. What tells them apart? (LO9.)

Beats:

1. Two real forms of the smallest algebra: the board A_1 with a white node ($\mathfrak{sl}(2,\mathbb{R})$) and with a black node ($\mathfrak{su}(2)$). What an involution is and why black means "compact".
2. Decorating a board: black nodes and arrows. The involution $\theta = -w_X \circ \tau$, played in three stages on the board (T13).
3. Shadows. Each root has a shadow $\frac{1}{2}(\alpha - \theta\alpha)$; black roots have none; several roots can share one. The count is the multiplicity. Worked case $\mathfrak{su}(1,2)$ on A_2: one shadow direction, multiplicities 2 and 1.
4. Which decorations are allowed: the three conditions of section 2.15 as three lights. The reader finds a forbidden decoration (A_2 with one black node) and sees which light fails. The lesson says what the failure means in the careful wording of section 2.15, and does not say that θ "is not an involution".
5. Reading the data: real rank, the type of the shadows, $\dim \mathfrak{k}$, the character.
6. The shadows have their own board and their own game; one small move is a fixed sequence of big moves. Worked case $\mathfrak{e}_{6(-26)}$: restricted board A_2, multiplicity 8.
7. The catalogue: every real form for every board up to rank 8, and the extremes (all white, all black, no black with arrows).
8. What the picture hides: τ on black nodes.
9. Sidebar: Vogan diagrams, the other standard picture.

Beat	Figure
1	T13 on A_1, node white, with a button to paint it black
2	T13 "play θ" on A_3 with black nodes 1, 3 ($\mathfrak{su}^*(4)$), root $(0,1,0)$
3	T13 shadows on A_2 with the flip and no black node ($\mathfrak{su}(1,2)$); the table of Appendix A.5
4	T13 on A_2, free decoration, lights visible; then A_3 with the examples of section 2.15 as presets
5	T13 data panel on A_3 with black node 2 and the flip ($\mathfrak{su}(1,3)$)
6	T13 restricted board on E_6 with black nodes 2, 3, 4, 5
7	T13 catalogue on D_4 (six records, two of them the same real form)
8	T13 on A_5 with $\mathfrak{su}(1,5)$ and the toggle "show τ on black nodes"
9	static

For readers who know Lie theory: maximally split Cartan subalgebras, restricted root systems, the relative Weyl group. Sources: [Sat60], [Ara62], [Hel01], [Kol14], [DK19], [Kna02].

M10. Atlas, sandbox, history and sources

The Atlas (T14) is a data sheet for every board, with a switch between conventions. The sandbox (T16) is the free editor. The history page gives the timeline with correct credit, in order of date: Proctor's use of the firing process (1984); the Olympiad problem (1986); Björner's preprint on Mozes' game (1988) and Mozes' paper (1990); Eriksson's papers (1992 to 1996); Wildberger's manuscript and first papers (2001 to 2003); Donnelly (2008); Gashi, Schedler and Speyer (2012); Wildberger's Algebra Colloquium paper (2020). Beside the timeline: Kostant's game as an oral tradition; Satake (1960) and Araki (1962) for the diagrams. The

sources page is generated from `SOURCES.md` (section 7.3). Further pages belonging to this module: glossary (from Appendix C), credits and licences, the open problem (section 2.11), and "common confusions" (decision D16).

4. Catalogue of interactive tools

Sixteen tools. Each is a self-contained island that reads and writes one state object (section 6.6), calls only the engine of section 5 for mathematics, and obeys the common rules of 4.0. "Phase" refers to section 9. Acceptance criteria are written so that they can be turned into automated tests; the numbers in them come from `golden_data.json` or Appendix A. A criterion that ranges over every board or every pair is tested at engine level (Vitest); the browser tests (Playwright) cover the named examples and one case per kind of board.

Tool	Id (section 6.6)	Phase	Used first in
T1 Game Board	`board`	1	M0
T2 Orbit Garden	`garden`	1	M1
T3 Form Finder	`form`	2	M2
T4 Classification Lab	`lab`	2	M3
T5 Numbers Game Lab	`numbers`	3	M4
T6 Kaleidoscope	`mirror`	3	M4
T7 Symbolic Game	`symbolic`	3	M5
T8 Weyl Explorer	`weyl`	3	M5
T9 Kostant's Game	`kostant`	1	M1
T10 Geometry Studio	`geometry`	4	M7
T11 Folding Bench	`fold`	4	M6
T12 Representation Builder	`rep`	5	M8
T13 Satake Studio	`satake`	6	M9
T14 Atlas	`atlas`	6	M10
T15 Affine Playground	`affine`	2	M3, M4
T16 Sandbox	`sandbox`	2	every module from M2

4.0 Rules common to every tool [D]

- **Starts meaningful.** A tool loads in a state that already shows something worth reading, and the surrounding prose states the result it demonstrates. Nothing the argument needs is available only after a click (principles P1, P2 of section 8.1).
- **Works before and without JavaScript.** The default state of every tool is rendered to HTML and SVG at build time, with its caption. Controls are present but disabled until the island has hydrated. With JavaScript off the reader sees the default figure, the caption and a one-line note; section 7.4 gives the details and the two tools that ship a picture instead of markup (T10, and T8 on its larger boards).
- **One state, derived views.** State is `{board, coords, start, moves}` plus view options (section 6.6). The current position is always recomputed by the engine from the start and the move list.
- **History.** Undo removes the last move from the list; redo puts it back; reset empties the list. A visible strip of past positions can be scrubbed; acting from an earlier position discards the moves after it. Because the position is recomputed, undo needs no legality test. A strip of more than 50 positions is shown as a slider with a counter.
- **Step and play to the end.** Wherever a rule fixes the next move (legal play and raising play, with the lowest-node rule), a tool offers "step", which makes one move, and "play to the end", which computes the whole play in one call (`play`, `raisingPlay`, or `playFate` off the finite types), appends its moves to the move list and shows the end. Afterwards the history holds every position of the play and can be

stepped through or scrubbed, and one announcement is made for the whole play (section 7.5). Where a play need not end, the control is labelled "play on" and reports the fate (T9, T15). Neither control acts by itself on page load. Motion respects the reduced-motion setting.

- **Configuration.** Every tool is configured by properties. Common to all: `id` (the instance id of section 6.6), `board`, and, where they apply, `coords`, `start`, `moves`, `linkTo` and `prediction`. T1 lists its further properties below. For the other tools, the figure tables of section 3 name the configuration of each figure by a short phrase after the tool's number ("three fates", showpiece, counter, tally, face table); each phrase is a value of the tool's `preset` property, and `default` is the state described in this section. A preset fixes only things the reader could have set by hand, so every preset is also a state object and a link. Property names beyond these are the builder's choice and are listed in the tool's test file.
- **Predict.** Tools placed in the guided part of a module accept a `prediction` configuration: a question, an answer type (a number, a position, a choice), the correct answer computed by the engine, and the text of the reveal. The board is frozen until the reader has answered or pressed "show me". Without JavaScript the question is followed by a `<details>` element holding the answer and the reveal.
- **Keyboard and screen reader**: section 7.5. Each board is one tab stop; arrow keys move between nodes; Enter or Space acts; a polite live region announces each result with the wording fixed in section 7.5.
- **Signs** are shown by the signed numeral, by fill (positive filled, negative outlined, zero neutral) and by colour, never by colour alone.
- **Share and export.** "Copy link" encodes the state in the URL and "Download data" gives JSON, from phase 1. SVG and PNG export of the figure arrive in phase 2. LaTeX export (`dynkin-diagrams` and TikZ, section 6.7) arrives in phase 6.
- **Stated limits.** When a computation is cut off, the tool says so in words ("stopped after 40 layers", "stopped after 100000 positions") and never presents a truncated result as complete. The words "not of finite type" are used only when `classify` has said so, never as an inference from a cap.
- **Growth off the finite types has two bounds.** The examples of `layer_growth_examples` show the two ways a search can run on [V]. On the triangle, on the star with four arms (both affine) and on the two-node board with three roads each way, the number of new root populations per layer stays bounded (6, then 9 or 10, then 4) while the entries grow, to 17 digits within 40 layers on the last of these; there a cap on positions is reached only after hundreds of layers (layer 333 on the triangle and layer 500 on the two-node board, at a cap of 2000 [V]), so the bound on layers is the one that acts. On the star with five arms, which is indefinite, the entries stay small and the number of positions grows geometrically, each layer about 1.8 times the one before; there the count ends the search. Every tool that grows a set of positions (T2, T4) therefore takes a `layers` property, default 40 on any board that `classify` does not call finite, passes it to the engine as `maxDepth` together with its cap on positions, and says which bound ended the search: "stopped after 40 layers", or "stopped after 2000 positions; layers 0 to 10 are complete". Before phase 2, when `classify` does not exist yet, both bounds apply to every custom board.
- **Board pickers.** A tool whose description names no particular boards offers the 32 named types, grouped by letter; T6, T8, T11 and T15 list their own boards; T4 and T16 offer presets and the editor. A picker changes `board` and resets `start` and `moves` to the tool's defaults for that board.
- **Long numbers.** A value of more than 12 digits is shown shortened ("1.2345… x 10^83", with the number of digits), with the exact digits in the table view, in the tooltip and in every export.
- **Small screens.** Every tool is usable at a width of 320 CSS pixels. Targets are 44 pixels where the layout allows and never below 24; the rule is in section 6.1, and at that width boards with more than six nodes in one row get less than 44.
- **Test hooks.** Every node button carries `data-node` (its label), `data-board` (`populations` or `numbers`, which matters in the linked view), `data-value` (the value shown, as decimal text, or as `p/q` for a fraction) and `data-can-act` (`true` or `false`). The tool's root element carries `data-tool` and `data-moves`. Every other output a criterion mentions (a verdict, a counter, a light, a message) is an element with `data-out="<name>"`, the names being listed in the tool's test file. Browser tests read these hooks and the live region, and nothing else.

T1. Game Board (phase 1; used by every module)

The core component: a board with nodes, roads and numbers, on which either game or both are played.

Views and modes. The view says which boards are drawn; the mode says which moves are allowed.

View	Draws	Modes allowed
`populations`	one board with the population	`free`; `raising` (only cities whose number a_i is negative: Kostant's game)
`numbers`	one board with the numbers	`free`; `legal` (only nodes whose number is positive; "game over" when none is)
`linked`	both, one above the other, with the line $a = Cp$ between them	`free`; `legal`; `raising`. Acting on a node of either board acts on both.

Coordinates of the start. `coords` says whether `start` is a population or a position of the numbers game. From a population the numbers are always Cp. From numbers, the population is $C^{-1}a$, which can be fractional; the populations board then shows exact fractions, and the `populations` view with `coords: numbers` is allowed only when $C^{-1}a$ is integral (otherwise the tool falls back to `linked`). On a board whose Cartan matrix is singular (affine boards) a start given as numbers has a population only when its level is zero, and then only up to multiples of the frozen population; for such a start only the `numbers` view is offered. On a named finite-type board a link whose start or play leaves the safe range of `SafeInt` is reloaded with `Big` arithmetic, with no message.

Shows: node labels; roads with multiplicity and arrowheads (section 6.2); the current value on each node; which nodes may be acted on; move counter; optional panels for the form value (p, p), the level on affine boards, the two colour classes, long and short nodes. Each panel arrives with its engine functions: form value, level, long and short nodes in phase 2; colour classes in phase 3.

Hints (optional, off in prediction steps): on focus or hover of a node, a ghost of the value it would take.

Configuration (component properties): `board` (a named board, an affine board, or a custom Cartan matrix), `view`, `mode`, `coords`, `start`, `moves`, `script` (a list of moves with a caption each, advanced by a "next" button), `goal` (one of: reach a given position; reach a position with no positive number; make no city unhappy; a named predicate from a short list in the code), `prediction`, `hints`, `panels`, `roadStyle`.

Acceptance:

- On A_2, `numbers`, `legal`, start $(1, 1)$: firing 1, 2, 1 shows $(-1, 2)$, $(1, -2)$, $(-1, -1)$, then "game over after 3 moves". On B_2 and G_2 the sequences of Appendix A appear exactly.
- On every type of `golden_data.json`, playing `longest_word` from all ones in `legal` mode is accepted move by move, fires exactly `fired_values`, and ends at all minus ones.
- In `linked` view, for 100 random populations and moves on each of A_3, B_3, G_2, E_8: the numbers shown equal Cp before and after every move.
- In `linked` view on A_2 with `coords: numbers` and start $(1, 0)$ the populations board shows $\frac{2}{3}$ and $\frac{1}{3}$.
- Undo after any sequence restores the previous position exactly, in all three modes.
- With a screen reader, firing node 2 on A_2 from $(1, 1)$ in the numbers view is announced with the template of section 7.5: "Fired node 2. Numbers now 2, minus 1. Move 1."

T2. Orbit Garden (phase 1)

Grows a set of positions as a graph: one vertex per position found, one edge per move, coloured and labelled by node.

- **What grows.** In `roots` mode the seeds are all the single Martians and the moves are mutations, so the garden is the set of root populations. In `orbit` mode the seed is one position of either game (by default the current position of a T1 on the same page) and the garden is its orbit.
- **Modes of growth**: `follow` (vertices appear as the reader visits them on the linked T1); `explore` (breadth-first, one layer per press of a button). The property `grown` gives the number of layers already grown when the page loads (default 0; M1 uses 40 on the triangle). That growth is computed at build time and is part of the default state.
- **Size.** The garden holds at most 2000 positions, so that every call it makes stays on the main thread (E-9). Up to 300 positions every vertex and edge is drawn; above that, vertices are dots without labels, and only the edges at the focused vertex and along the current play are drawn, as in T8. At the bound the tool says "stopped after 2000 positions" and names the complete layers (`completeLayers` of the engine's result).
- **Positive roots only** (in `roots` mode): the positive root populations drawn in layers by total population, with an edge from p to $S_i p$ whenever that mutation raises city i. On a simply laced board of finite type every such edge raises by 1 and the picture is the Hasse diagram of the root poset. On B, C, F, G a rise of 2 or 3 occurs and is drawn as a double or triple line, as in [W20] section 9. On a board that is not of finite type the rise can be any positive integer and is written on the edge.
- **Interactions**: click a vertex to jump there (the board shows a shortest word, from `wordTo`); toggle labels; a "found so far / total" counter whose total is hidden until the reader asks. In `roots` mode on a named finite-type board the total is the number of roots. In `orbit` mode the total is known when the enumeration completes within the garden's bound; otherwise the counter says "more than 2000", and from phase 3 on a finite-type board it gives the exact size from `orbitSize`. On other boards there is no total.
- **Boards that are not of finite type**: growth stops after `layers` layers (section 4.0) and a small chart shows the number of new positions per layer.

Acceptance: `roots` mode finds 12, 24, 72, 126, 240 root populations on A_3, D_4, E_6, E_7, E_8 and `n_roots` on every other type; the layered view of the positive roots of A_4, B_3, G_2 matches the `positive_roots` lists; on the triangle, 40 layers give 243 root populations, 6 new ones per layer, with the message "stopped after 40 layers"; on the star with five arms the garden stops at 2000 positions with layers 0 to 10 complete, which hold 1271 of them (`layer_growth_examples`). From phase 2 the verdict "affine" from `classify` is shown beside the message; that part of the criterion belongs to gate G2.

T3. Form Finder (phase 2)

- **Meter**: shows the form value (p, p) of the current population during play.
- **Builder**: the reader sets a weight w_i on each city and a weight w_{ij} on each joined pair, which defines

$$Q_w(p) = \sum_i w_i \, p_i^2 - \sum_{i<j} w_{ij} \, p_i p_j .$$

  City weights are positive; road weights are zero or positive. "Test" asks the engine (`formIsInvariant`) whether every mutation preserves Q_w and, if not, shows the population and the mutation that the engine returns as a witness. On a balanced connected board exactly one choice up to scale passes: $w_i = 2d_i$ and $w_{ij} = 2d_i A_{ji}$ (section 2.5). On a connected bidirected board that is not balanced, and on a board with a one-way road, no choice with positive city weights passes [V on two unbalanced triangles, 100 random choices each].
- **Small values**: the reader types populations, with entries of at most six digits, and the tool reports the form value and, through `isRootPopulation`, whether each is a root population ("too large to test here" when the engine answers `null`). On a finite board the least value over nonzero populations is 2, attained exactly at the short roots, which on a simply laced board are all the roots [V in the box $[-2, 2]^n$ for eight types]; on an affine board the frozen population has value 0.

Acceptance: the builder accepts weights proportional to $w_i = 2d_i$, $w_{ij} = 2d_i A_{ji}$ with d from `symmetrizer`, and rejects a perturbation of any one weight, on A_2, B_2, B_3, F_4, G_2; the meter is constant along 1000

random moves on each finite type; on each of the two unbalanced triangles of `classification_tests`, 100 random choices of positive weights all fail, each with a witness.

T4. Classification Lab (phase 2)

A graph editor with a live verdict.

- **Editor**: up to 12 nodes; between two nodes, 0 to 4 roads each way, including roads one way only; drag to arrange, or add and remove with buttons and keys (no action is drag-only). Deleting a node renumbers the labels $1, \dots, n$ in their old order and resets `start` and `moves`. Presets: every finite type, the affine boards of the golden data, the test boards of `classification_tests`.
- **Verdict** for each connected component, exactly as `classify` returns it: finite with its name and the matching of the reader's nodes to the Bourbaki labels (`bourbaki_order`); affine with its name, marks and comarks; indefinite; or "has a one-way road: the theory does not apply". B_2 and C_2 are one board, reported as "$B_2 = C_2$".
- **Also shown**: whether the board is balanced and the weights d_i if it is; on a bidirected board, the spectral radius of A to four decimals (`spectralRadius`; not shown on a board with a one-way road); and, when a bidirected board is not of finite type, a smallest set of nodes whose induced board is already affine or indefinite (`smallestNonfiniteSubboard`; compare [W20] Corollary 4.3, which allows roads as well as cities to be removed).
- **Root counter**: number of root populations by breadth-first layer. On a board that `classify` calls finite the count is complete and immediate. On any other board it runs on the main thread for `layers` layers or 2000 positions, whichever comes first (section 4.0, E-9), and a button "keep counting" continues in the worker up to 100000 positions. Each stop is reported in the words of section 4.0. On a board with a one-way road the editor draws every road as its own arrow (`roads` style) and the counter still runs.
- **Conjecture hunter**: searches the root populations found for one with both signs; reports the first, or "none among the k root populations of the first 40 layers".

Acceptance: recognises every type of `golden_data.json` and every board of `affine` and `twisted_affine` under random relabelling, with B_2 for C_2 and `C2~` for the affine board of B_2; reproduces every entry of `classification_tests` (kind, class, name, balanced or not, marks and comarks, smallest non-finite subboard, and the spectral radius to four decimals where one is recorded); on the one-way road reports the mixed population $(-1, 1)$ after two moves and gives no finite or affine verdict; never reports a mixed population on a bidirected board; on the star with five arms the root counter first stops at 2000 positions with layers 0 to 10 complete, and after "keep counting" at 100000 positions with layers 0 to 17 complete, which hold 74311 of them (`layer_growth_examples`).

T5. Numbers Game Lab (phase 3)

- **Start chooser**: all ones, a fundamental weight, or custom.
- **Tape**: the list of numbers fired so far.
- **Tally**: a bar for each value showing how often it has been fired; when the game from all ones ends, the bars are read sideways to give the exponents, and the product $\prod(m_i + 1)$ is shown beside the order of the group.
- **Wave**: fires one colour class per step and counts rounds.
- **Two games compared**: replay from the same start in a different order and overlay the two tapes sorted; they agree.
- **Dimension calculator**: for a chosen dominant λ, shows the tape from ρ, the tape from $\lambda + \rho$ along the same word, and the quotient of the products (exact big integers).

Acceptance: reproduces `exponents`, `weyl_order` and `fired_tally` for every type; the wave takes `coxeter_number` rounds on every type of rank at least 2 (on A_1 it is a single move and the tool says so); reproduces all 30 entries of `weyl_dimension`, including 3875 for E_8.

T6. Kaleidoscope (phase 3)

For the rank 2 boards A_2, B_2, C_2, G_2: the plane with the two mirrors, the simple roots drawn as the mirror normals, the chambers, and one point.

- **Coordinates**: the triangular basis of section 2.14 (`choleskyBasis`): α_1 points along the positive horizontal axis with length $\sqrt{2d_1}$, and α_2 has length $\sqrt{2d_2}$ and makes the angle 120°, 135° or 150° with it, counterclockwise.
- The point is a position a of the numbers game with integer numbers. Its **numbers** are drawn as its two perpendicular distances to the mirrors, scaled as in section 2.4(d); its **population** $p = C^{-1}a$ is drawn as the parallelogram of its coefficients along the two normals and printed as exact fractions.
- Firing node i reflects the point. Both readouts update; the new population is $p - a_i e_i$, and the line $a = Cp$ is shown with the actual numbers.
- The point is placed on the weight lattice by buttons and keys as well as by dragging (the drag snaps to the lattice).
- Optional overlays: all roots; the orbit of the point; the root lattice and the weight lattice as two dot patterns.

Acceptance: for 50 random lattice points on each board, the displayed population and numbers satisfy $a = Cp$ in exact rational arithmetic, firing node i changes the numbers as `fire` says and the population by $-a_i e_i$; the six, eight and twelve roots are drawn at the correct angles and lengths (ratio $\sqrt{2}$ for B_2 and C_2, $\sqrt{3}$ for G_2), tested through $U^{\mathsf{T}}U = B$.

T7. Symbolic Game (phase 3)

Two boards side by side. On the left the reader plays the numbers game in `legal` mode from a start with all numbers positive (all ones by default). On the right the same moves are played with letters $x_1, \ldots, x_n$ as the start (`symbolicPlay`). Each fired entry on the right is displayed as a linear form, and a side table matches it with a positive root of the reversed board. At the end every positive root has been used once. The left board decides which moves are legal; the right board only follows.

Acceptance: on every type of rank at most 4 and on E_6, with the word `longest_word`, the multiset of fired forms equals the positive roots of the transposed Cartan matrix; on B_3 the table visibly differs from the list of positive roots of B_3 itself.

T8. Weyl Explorer (phase 3)

For boards with at most 1152 group elements (A_1, A_2, A_3, A_4, A_5, B_2, B_3, B_4, C_2, C_3, C_4, D_4, F_4, G_2): the graph of all positions reachable from all ones by legal play, drawn in layers by the number of moves.

- The reader's current game is a path in this graph. Hovering or focusing a vertex shows the position, a shortest word (`wordTo`), and the positive roots whose mirrors separate it from the start (`separatingRoots`).
- **Counter**: the number of complete games (F12), from `countCompleteGames`. Every board offered here has at most 1152 group elements, so the count runs on the main thread (E-9).
- **Heap view**: the current word as a stack of bricks, one column per node, a brick resting on the earlier bricks of the same or a joined node; and the number of plays with the same stack (`commutationClassSize`).
- **Size.** Number of positions in the widest layer: A_3 6, B_3 8, A_4 22, D_4 30, B_4 46, A_5 101, F_4 94. Vertices are small dots without labels; the drawing is as wide as its widest layer needs and scrolls sideways inside its own box. Above 192 group elements, edges are drawn only along the current game and around the focused vertex.

Acceptance: layer sizes equal `length_distribution` and sum to `weyl_order`; the count of top-to-bottom paths equals `n_reduced_words_w0`; two plays that differ by exchanging adjacent moves at unjoined nodes show the same heap; on A_3 the 16 complete games show 8 different heaps, with the numbers of games per heap equal to `heaps.A3.games_per_heap`.

T9. Kostant's Game (phase 1)

T1 in the `populations` view and `raising` mode, started from one Martian, with the three moods shown (unhappy: a mutation would raise the city; happy: no change; manic: a mutation would lower it). Goal: no unhappy city. At the end the tool names the population (the highest root, or the highest short root after a start at a short city), shows the number of moves (beside $h - 2$ on a simply laced board), and marks the cities whose final number is positive with the caption of section 2.12: a new city joined to these, with the road counts given there, makes an affine board. From phase 2 the caption names that board (`kostant[T].affine_board_from_node`) and a button opens it in T4. The engine function is `raisingPlay`, which starts from any population; `kostantPlay` is the special case of one Martian.

Acceptance, gate G1: for every type T and every starting node, the play reproduces three lists of `kostant[T]`: `end_from_node`, `moves_from_node` and `final_numbers_from_node`. Gate G2 adds: the board named is the entry of `affine_board_from_node`; on an affine board the tool offers "play on" in place of "play to the end" and reports "still running after 20000 moves" with the verdict of `classify`.

T10. Geometry Studio (phase 4)

For the named finite types.

- **Input**: a board and a non-negative integer on each node (click to ring), or the shortcut "root polytope", which rings the board with `numbers_of_highest_root` and offers the short roots as a second set of points.
- **Output**: the orbit as points in Euclidean coordinates (`euclideanBasis`); edges by the rule of section 2.14 (`wythoffEdges`); the table of face counts by type (`wythoffFaceTypes`).
- **Projection**: rank 2 flat; rank 3 as a wireframe that can be turned (points and segments with a depth cue; no shaded faces in version 1); rank 4 and above by orthogonal projection, with "Coxeter plane", "three chosen coordinates" and rotation in any coordinate plane, by buttons and keys as well as by dragging.
- **Switch**: polytope graph or move graph.
- **Make uniform** (display only): the engine keeps, for every vertex, a shortest word from the start (`orbitGraph`, `wordTo`); the tool replaces the start by the real vector with $a_i = c/\sqrt{d_i}$ on the ringed nodes and replays those words in floating point (`applyWordFloat`).
- **Showpiece**: E_8, root polytope, Coxeter plane, with the 8 rings of 30. In the Coxeter plane of other types the tool states the number of distinct dots when it is smaller than the number of points.
- **Limits**: edges are enumerated and drawn up to 70000 when the picture is at rest; while it is being turned at most 8000 are drawn; beyond 70000 only vertices, with a message. Vertices are drawn up to 20000.
- **Presets**: the default and every configuration that section 3 names (the showpiece among them) are computed at build time. Their vertices and edges are shipped as data files that the tool fetches, each at most 30 kB after compression [D], so no default needs the worker. After the reader changes the rings or the board, an enumeration of more than 2000 vertices or edges runs in the worker (E-9).
- **Without JavaScript** the default projection is shown as an SVG image produced at build time.

Acceptance: face counts equal `wythoff[*].f_vector` for all 26 records; for records with `edges_enumerated` true the numbers of vertices and edges of the drawn object equal the first two entries, and for the others the tool shows vertices only with the message; for the root polytope in the Coxeter plane with both sets of points shown (all roots), the ring sizes, innermost first, equal `types[T].coxeter_plane.rings` (radii to five decimals) and the number of distinct dots equals `.distinct_points`; on A_2 from $(1, 1)$ the two graphs share four edges.

T11. Folding Bench (phase 4)

Boards offered: A_2 to A_8, D_4 to D_8 and E_6, the named types that have a symmetry other than the identity. Big board and folded board side by side. The reader picks a symmetry (`symmetries`); orbits are coloured; in "symmetric play" one click acts on a whole orbit; the folded board shows the same numbers. The folded

board is drawn with its nodes in Bourbaki order (`folded_orbits_in_bourbaki_order`). A second tab, "average", averages the roots instead and names the resulting type (`averagedRootsType`); it is available for every symmetry, including the flip of A_4, where the board does not fold. Phase 7 adds the golden foldings with values shown as $a + b\varphi$.

Acceptance: for every row of `folding` that has a folded board, 200 random symmetric plays agree between the two boards in both games and the folded board is the one named in `folds_to`; for every row the averaged type equals `averaged_roots_type`; for the flip of A_4 the first tab says that an orbit contains an edge.

T12. Representation Builder (phase 5)

For any node i of a named board; the construction succeeds exactly when the node is minuscule, and the tool shows the failure otherwise. For a node that is not minuscule the matrices are built only when the orbit of ω_i has at most 2000 positions (`orbitSize`; node 2 of D_4 has 24), and otherwise the tool shows the explanation alone. The basis is the orbit of ω_i in decreasing lexicographic order of the positions, as in the oracle.

- the orbit drawn in layers with edges coloured by node;
- the matrices E_k, F_k, H_k shown as grids (as lists of nonzero entries above dimension 27);
- a **relation checker**: the reader picks one of the Chevalley-Serre relations and one or two nodes, and the tool displays the matrix on each side of the relation and whether they are equal (`chevalleySerre`);
- the move matrix beside its signed version $\exp(E)\exp(-F)\exp(E)$ (`signedMoves`);
- a **string viewer** (`stringThrough`): for a chosen position μ and node i with $\mu_i \geq 0$, the line of positions $\mu,\ \mu - \alpha_i,\ \ldots,\ \mu - \mu_i\alpha_i$ (subtracting column i of C each time), with firing shown as the flip of the line end over end. The viewer accepts any position, so it can show a line with three or more positions, which a minuscule orbit never has.

Acceptance: dimensions equal `minuscule_representations[*].dimension`; every relation checks for every listed representation; choosing the non-minuscule node 2 of D_4 shows a failing relation and the explanation that some number in the orbit is 2.

T13. Satake Studio (phase 6)

- **Decorate**: paint nodes black or white; choose τ among the symmetries of the board of order at most 2 (the identity; the flip on A_n and E_6; the swap of the last two nodes on D_n; the three swaps on D_4).
- **Three lights**: one for each condition of section 2.15, each judged independently (`admissibility`), each with its reason in words when red. The tool writes the sentence from the engine's `Reason` (section 5.2), with node labels.
- **When not admissible**: no data are shown. The engine's `satake` throws `NotAdmissible`, and the tool shows the lights and the sentence of section 2.15 on what a failed condition means.
- **When admissible**: the name or names of the real form (`matchCatalogue`, which returns two records on D_4 for the class containing $\mathfrak{so}(2,6)$ and $\mathfrak{so}^*(8)$; both are shown, each with its own `restricted_type_display`, here B_2 and C_2, which are the same system); real rank; type of the restricted roots (shown as `restricted_type_display`); the multiplicities $(m_\lambda, m_{2\lambda})$ at each node of the restricted board; $\dim \mathfrak{k}$, $\dim \mathfrak{p}$, character.
- **Play** θ: on a chosen root, in the three stages of section 2.15.
- **Shadows**: for real rank 1 and 2 a picture, otherwise a table: each restricted root with the roots above it (`shadow2`). The picture places each restricted root by its coordinates in the simple restricted roots, drawn in the triangular basis (`choleskyBasis`) of the restricted board; doubled roots of a BC system are drawn at twice the distance.
- **Restricted board**: drawn in Bourbaki order (`restricted_orbits_in_bourbaki_order`); clicking one of its nodes plays the composite word on the big board, in steps, and the shadow reflects.
- **Catalogue**: all records of the board; selecting one loads it. Export as `\dynkin` LaTeX.
- **Toggles**: show τ on black nodes; (phase 7) generalized diagrams.

Acceptance: for every type, the set of pairs (X, τ) accepted, over all subsets X and all symmetries τ of order

at most 2, equals `admissible_pairs[T]`; every field of every record of `satake` is reproduced; each composite word acts on shadows as the reflection given by `restricted_cartan`; every entry of `satake_examples` shows the recorded verdict for each of the three lights; on D_4 with black nodes 3 and 4 both names are shown.

T14. Atlas (phase 6)

One static page per board, generated at build time by calling the engine: the Cartan matrix with a convention switch (this site, Kac and SageMath; or Bourbaki and Humphreys, transposed) and the note on Wildberger's names; roads; weights d_i; roots (up to 120 positive ones listed); highest roots and their numbers; exponents; h; $|W|$; $\det C$; minuscule nodes; $-w_0$; orbit sizes of the fundamental weights; number of complete games where the golden data has it; the ring sizes in the Coxeter plane; marks and comarks of the affine board; real forms. Every item links to the tool that demonstrates it with that board loaded. Downloads: JSON, CSV, LaTeX.

Acceptance: every displayed value that has a golden value equals it (all of the items above have one, except the lists of roots for types where `positive_roots` is absent, for which the count is compared); the transposed matrix equals the transpose.

T15. Affine Playground (phase 2)

Boards offered: the untwisted affine board of every type of rank at most 8, the ten twisted boards of `twisted_affine`, and two cycles drawn as polygons, the square and the pentagon.

- **Frozen population** (populations view; used in M3): started at the marks; every city is happy; adding one Martian and playing shows the growth.
- **Marks and comarks** side by side, on a board where they differ.
- **The pentagon** (numbers view; used in M4): the puzzle in the Olympiad's own form (a negative number is flipped), with a switch to the site's convention and a level meter. The default start is the example of section 2.12.
- **Three fates** (numbers view): the reader sets a start, sees its level, and plays or presses "play on". The report comes from `playFate`: "ended after k moves"; "returned to an earlier position after k moves"; or "still running after 20000 moves", shown with the level and the sentence of section 2.12 that applies.

Acceptance: the level is constant along 1000 random firings on every board offered; the pentagon example ends after 14 moves in 20 random orders of play, with the word, fired numbers and end of `pentagon_example` under the lowest-node rule; on every board, 10 random starts with entries between -9 and 9 and negative level report "ended", 10 such starts of positive level report "still running", and the numbers of 5 random root populations report "returned".

T16. Sandbox (phase 2)

The editor of T4 with both games in free play and integers of any size (the `Big` functions of section 5). No verdicts beyond those of T4. Import and export of boards as JSON. It is the last link of every module from M2 on; M0 and M1, which are written before the sandbox exists, receive the link when phase 2 is released.

Acceptance: values beyond 2^{53} are computed exactly: both sequences of `bigint_test` (200 moves on the board with three roads each way) end at the recorded integers; the first entry beyond 2^{53} appears after the move recorded there.

5. Engine specification

The engine is one TypeScript package with no runtime dependencies: nothing from the DOM, from a UI framework, from the network or from npm (its only development dependencies are TypeScript and the test tools). Every tool calls it; nothing else computes mathematics. It is a port of `reference/lie_games.py`, function for function, and its tests are the golden data. The column "Phase" in the tables below says in which phase of section 9 a function is written. A tool of phase k uses only functions of phases 0 to k; where

a tool gains a feature in a later phase (the Kostant tool learns the name of the affine board in phase 2, for instance), section 4 says so and assigns that part of its acceptance to the later gate.

5.1 Principles

- **E-1 Exact.** All game arithmetic is on integers (or on $\mathbb{Z}[\varphi]$ in phase 7). Floating point appears only in the functions of the table "Geometry" and in `spectralRadius`.
- **E-2 One code path, three kinds of number.** The core functions are written once against a small arithmetic interface with three instances: `SafeInt` (JavaScript numbers, every result checked with `Number.isSafeInteger`, a failure throws `EngineOverflow`), `Big` (`bigint`) and, in phase 7, `ZPhi`. `SafeInt` and `Big` are both written in phase 0. This mirrors the oracle, where Python integers and the class `Phi` pass through the same functions. A board carries its arithmetic: named finite-type boards use `SafeInt`; **every other board** (custom boards, affine boards, the sandbox) uses `Big`, because off the finite types the entries grow without bound, and on an indefinite board they can pass 2^{53} within a few dozen moves (after move 39 on the board of `bigint_test`). If generics prove awkward, writing the half-dozen core functions twice is acceptable; the tests do not care [D].
- **E-3 Rationals.** A small rational type on `bigint` (numerator, positive denominator, lowest terms) is part of the engine from phase 1. It serves populations of weights, the inverse Cartan matrix, the symmetrizer, the form, determinants and Kolb's third condition. No library is used.
- **E-4 Counts that can be large are `bigint`**: the products in `weylDimension` (for E_8 the product of the fired numbers has more than a hundred digits), `countCompleteGames`, `commutationClassSize`. Group orders and orbit sizes fit in safe integers for every board of at most 12 nodes and are numbers.
- **E-5 Pure and deterministic.** Functions do not mutate their arguments and have no hidden state, apart from the cache of E-11, which changes no result. Where the oracle makes a choice, the engine makes the same one: `play`, `raisingPlay` and `kostantPlay` take the **lowest position** among the allowed nodes unless a `choose` function is given; breadth-first searches try nodes in increasing position; `identify` returns the lexicographically smallest relabelling; the basis of `minusculeMatrices` is the orbit in decreasing lexicographic order.
- **E-6 Bounded.** Every enumeration takes a cap on the number of items and returns `{items, truncated}`; the searches `orbit`, `orbitGraph` and `roots` also take `maxDepth`, a number of layers, which is the bound to use off the finite types (section 4.0). Every play takes a cap and returns a `status`. `relabellings` and `symmetries` take a limit on the number of results (default 10000). No function loops without a bound.
- **E-7 Positions inside, labels at the boundary.** Inside the engine a node is a position $0, \dots, n-1$, exactly as in the oracle, and row and column k of every matrix belong to position k. A board also carries `labels`, the integer shown to the reader for each position:

Board	Labels, in the order of the positions
named finite-type board	$1, \dots, n$, Bourbaki order
untwisted affine board (`affineExtension`, or a name such as `B3~`)	$0, 1, \dots, n$: the added node first, then the finite board
twisted affine board by name (`D4^(3)`)	$0, 1, \dots, n$ in the order of its matrix in `twisted_affine`
presets `pentagon`, `square`; custom boards	$1, \dots, n$, in order round the cycle, or in order of creation
`subBoard(board, nodes)`	the labels of the chosen nodes, unchanged

Board	Labels, in the order of the positions
folded board, restricted board	$1, \dots, m$ in the Bourbaki order of the type that `identify` names (`folded_orbits_in_bourbaki_order`, `restricted_orbits_in_bourbaki_order`); in the order of the orbits when no type is named

Everything outside the engine uses labels: the golden file, state objects, URLs, exports, the screen, announcements. Two functions, `toLabels` and `toPositions`, convert node lists and words; nothing else converts. The oracle is less uniform than this rule: its functions `classify`, `satake`, `smallest_nonfinite_subboard` and `wythoff_face_types` return node numbers from 1, and the `reasons` of its `admissibility` are sentences with such numbers in them. The engine returns positions in all of these, and returns each reason as data (section 5.2) from which the tool writes its sentence with labels. No golden test would notice a stray `+ 1` on a board whose labels start at 0, so this is checked by a unit test on an affine board.

- **E-8 Three kinds of board.** `boardKind` sorts a matrix into `not-a-board` (rejected with `NotABoard` by every constructor), `one-way` and `gcm` (section 2.1). The moves, orbits and plays work on both kinds. Everything that belongs to the theory requires `gcm`; section 5.6 lists what each function returns on a `one-way` board.
- **E-9 No threads inside.** Engine functions are synchronous and know nothing about workers. At build time, when Astro renders the pages in Node, every call runs in-process and the rule that follows does not apply. In the browser the site layer (`site/src/lib/worker.ts`, built in phase 1) decides **before** a call where it runs. Everything runs on the main thread except the following, which run in a Web Worker behind a promise:

  1. an enumeration (`orbit`, `orbitGraph`, `roots`, `legalLayers`, `minusculeMatrices`, the vertices and edges of a polytope) that is not known in advance to yield at most 2000 items. Known in advance are: the roots of a board that is of finite type (at most 288 on twelve nodes); from phase 3, an orbit on a finite-type board whose size `orbitSize` gives; any search called with a cap of at most 2000;
  2. a play or a word of more than 2000 moves: `playFate`, a play to the end on a board that is not of finite type, a long list of moves read from a link. Before phase 2, when `classify` does not exist, every custom board counts as not of finite type;
  3. the counts of games and of heaps on a board whose group has more than 2000 elements; Satake data (`satake`, `satakeRecord`, `restrictedBoard`) for E_7 and E_8; the sweep over all pairs of a board.

  A call is never started in one place and continued in the other. **No tool may need the worker for its default state** (section 7.6): where a default would need it, the result is computed at build time and shipped as data (the presets of T10), or the tool shows the bounded main-thread result and offers the longer computation on a button (the root counter of T4). The limit of 2000 comes from measurement (section 5.5): it keeps a main-thread call near 10 ms on a desktop processor. The engine package itself runs unchanged in Node, which is where its tests run.
- **E-10 Errors.** `NotABoard` (the matrix is not a board at all); `NeedsTwoWayRoads` (a function of the theory was called on a one-way board); `NeedsConnectedBoard` (`cartanClass`, `identify` or `identifyAffine` was given a board in several pieces); `NotFiniteType` (a function of the finite theory was called on a board that is not of finite type); `NotAffine` (`marks`, `comarks` or `level` was called on a board that is not affine); `NotAdmissible` (`satake`, `satakeRecord` and `restrictedBoard` on a pair that fails a condition); `EngineOverflow`. Reaching a cap is reported in a field of the result and is never thrown.
- **E-11 The engine guards; the oracle does not.** The oracle's functions of the finite theory assume

a finite-type board and do not check (on other input some of them run until a cap and some return meaningless values). The engine adds the checks of section 5.6. The test for finite type is `classify`, whose verdict is kept in a `WeakMap` from board objects to verdicts inside the engine module; boards are immutable, so the cache cannot go stale. Because `classify` is written in phase 2, the phase 1 tools call the functions of the finite theory only on named finite-type boards, and the guards are added in phase 2.

5.2 Types

```
// three instances: SafeInt: Arith<number>, Big: Arith<bigint>, ZPhiArith: Arith<ZPhi>
interface Arith<T> {
  zero: T; one: T; fromInt(n: number): T;
  add(a: T, b: T): T; sub(a: T, b: T): T; mul(a: T, b: T): T; neg(a: T): T;
  sign(a: T): -1 | 0 | 1; eq(a: T, b: T): boolean;
  show(a: T): string;                    // decimal text; also the hash key
  toBig(a: T): bigint;                   // SafeInt and Big only; used by the rational functions
}
type Vec<T> = readonly T[];
type Matrix<T> = readonly (readonly T[])[];
interface Board<T = number> {
  n: number;
  cartan: Matrix<T>;                     // C[i][j] = <alpha_i^vee, alpha_j>, positions 0..n-1
  labels: readonly number[];             // what the reader sees for each position (E-7)
  kind: 'gcm' | 'one-way';
  arith: Arith<T>;
  name?: string;                       // "E8", "B3", "A4~", "D4^(3)", "pentagon"; undefined if custom
}
type Word = readonly number[];           // positions, in the order played
interface Rational { num: bigint; den: bigint; }
interface Enumeration<X> { items: X[]; truncated: boolean; }
interface PlayResult<T> { status: 'ended' | 'looped' | 'cap'; moves: number; word: Word;
                          fired: T[]; position: Vec<T>; firstVisit?: number; }
interface OrbitGraph<T> { items: Vec<T>[]; truncated: boolean;
                          depthReached: number;          // the largest depth among items
                        completeLayers: number;        // every position of depth <= this is in items
                          depth: number[];               // the three arrays are indexed like items
                          parent: (number | null)[]; node: (number | null)[]; }
type Symmetry = readonly number[];       // image of each position
interface SatakeDiagram { board: Board; black: readonly number[]; tau: Symmetry; }
interface Admissibility { symmetry: boolean; condition_1: boolean; condition_2: boolean;
                          condition_3: boolean; admissible: boolean;
                          reasons: Record<string, Reason>; }   // keys: symmetry, condition_1, ...
interface Reason { code: 'not-a-symmetry' | 'not-an-involution' | 'moves-black-set'
                       | 'differs-from-minus-wX' | 'not-an-integer';
                   nodes: number[]; value?: string; }      // value: the fraction, as text
interface FoldResult { folded: Board | null; orbits: number[][]; reason: string | null; }
interface InvarianceResult { invariant: boolean;
                             witness?: { node: number; population: number[] }; }
type SparseMatrix = Map<string, bigint>;   // key "row,column"
interface RelationCheck { relation: 'hh' | 'he' | 'hf' | 'ef' | 'serre-e' | 'serre-f';
                          i: number; j: number; holds: boolean;
                          left: SparseMatrix; right: SparseMatrix; }
```

Five functions return records with the golden field names: `classify` (one record per component), `smallestNonfiniteSubboard`, `admissibility` (apart from `reasons`, which the golden file holds as sentences and no test compares), `satake` and `satakeRecord`. Their results are plain objects with exactly the field names of the golden file (snake case, as in Appendix G and section 6.4), with nodes as positions; a test converts nodes to labels and then compares field by field, and the types of these records are written from Appendix G.

Every other function returns the types above, in camel case, and its golden test says how the result maps to the stored fields: for `fold`, `folded.cartan` is compared with `folded_cartan` and `identify(folded)` with `folds_to`; for `orbitGraph`, `completeLayers` with `complete_layers`; for `play`, `word`, `fired` and `position` with `word_lowest_node_rule`, `fired` and `end`.

5.3 Boards

Function	Meaning	Phase	Oracle
`cartan(name)`	`"A3"`, `"B4"`, …, `"G2"`, any rank the families allow	0	`cartan`
`boardFromName(name)`	every name of section 6.6: finite types (phase 0); `X~`, twisted names, `pentagon`, `square` (phase 2). `B2~` and `C2~` are both accepted; they are the affine extensions of the two numberings	0, 2	`board_from_name`
`fromAdjacency(A)`, `adjacency(board)`	$C = 2I - A^{\mathsf{T}}$ and back	0	`adjacency`
`boardKind(C)`	`not-a-board`, `one-way` or `gcm`	0	`board_kind`
`toLabels`, `toPositions`	E-7	0	(boundary only)
`components(board)`	connected components, roads in either direction	1	`components`
`transposeBoard(board)`, `subBoard(board, nodes)`	the reversed board; an induced board	1	`transpose`, `sub_matrix`
`determinant(M)`	exact, rational	1	`determinant`
`symmetrizer(board)`	the d_i as rationals with least value 1 on each component, or `null` when the board is not balanced	2	`symmetrizer`, guarded by `is_symmetrizable`
`gram(board)`	$B = DC$ as rationals, or `null`	2	`gram`, same guard
`cartanClass(board)`	`finite`, `affine`, `indefinite`, for a connected `gcm` board	2	`cartan_class`
`relabellings(b1, b2, limit?)`	all ways to match two boards	2	`relabellings`
`identify(board)`	name and Bourbaki order of a connected finite-type board in any numbering; `null` when there is none	2	`identify_labelled`
`affineCandidates(m)`, `identifyAffine(board)`	the named affine boards with m nodes; the name of a connected affine board, or `null`	2	`affine_candidates`, `identify_affine`
`classify(board)`	the full verdict, one record per component (section 2.12; fields in Appendix G under `classification_tests`)	2	`classify`
`smallestNonfiniteSubboard(board)`	section 4, T4; `null` on a finite board and on a one-way board	2	`smallest_nonfinite_subboard`
`affineExtension(board, short?)`	node 0 first; returns board, marks, comarks	2	`affine_extension`
`marks(board)`, `comarks(board)`	null vectors of an affine board	2	`marks`, `comarks`, `null_vector`
`spectralRadius(board)`	of the adjacency matrix; float, for display; `null` on a one-way board	2	`spectral_radius`
`bipartition(board)`	two colour classes, or `null` if the board has an odd cycle	3	`bipartition`
`symmetries(board, limit?)`	all node permutations preserving C	4	`automorphisms`

5.4 The games and everything built on them

Function	Meaning	Phase	Oracle
`mutate(board, p, i)`	section 2.2	0	`mutate`
`fire(board, a, i)`	section 2.3	0	`fire`
`mutationMatrix`, `firingMatrix`	S_i, R_i	1	same names
`toNumbers(board, p)`	$a = Cp$	1	`dynkin_labels`
`toPopulation(board, a)`	$C^{-1}a$ as rationals whenever $\det C \neq 0$, on any kind of board; `null` when C is singular	1	`weight_to_root_coords`

Function	Meaning	Phase	Oracle
`applyWord(board, v, word, game)`	play a word in free play	1	`act_on_roots`, `act_on_weights`
`legalNodes(a)`, `raisingNodes(board, p)`	nodes with a positive number; cities whose number is negative	1	inside `play`, `raising_play`
`play(board, a, choose?, cap)`	legal play; returns a `PlayResult` (`ended` or `cap`)	1	`play`
`raisingPlay(board, p, choose?, cap)`, `kostantPlay(board, i, ...)`	raising moves only, from any population; from one Martian	1	`raising_play`, `kostant_play`
`orbit(board, seeds, game, cap, maxDepth?)`	breadth-first closure under all moves	1	`orbit`, `orbit_graph`
`orbitGraph(board, seeds, game, cap, maxDepth?, legalOnly?)`	the same, remembering for each position its depth and the first move that reached it; order and truncation as in section 5.5	1	`orbit_graph`, `orbit_graph_capped`
`wordTo(graph, v)`	a shortest word to a position	1	`word_to`
`roots(board, cap, maxDepth?)`, `positiveRoots`	root populations; positive ones sorted by total, then lexicographically	1	`roots`, `positive_roots`
`highestRoot(board)`, `highestShortRoot(board)`	the positive root of largest total; on a board with two root lengths, the other positive root with no negative number (`null` on a simply laced board)	1	same names
`coxeterNumber(board)`	$2N/n$	1	`coxeter_number`
`longestWord(board, nodes?)`	word of the longest element of the sub-board	1	`longest_word`
`minusW0(board)`	the images of the nodes under $-w_0$	1	`minus_w0`
`minusculeNodes(board)`	F11	1	`minuscule_nodes`
`playFate(board, a, choose?, cap)`	legal play with loop detection (`ended`, `looped`, `cap`)	2	`play_fate`
`isRootPopulation(board, v, cap?)`	exact test on a `gcm` board: `true`, `false`, or `null` when the descent has not finished after `cap` steps (default 20000; the number of steps is at most the total of v)	2	`is_root_population`
`coroot(board, root)`	coordinates in simple coroots	2	`coroot_coords`
`formValue(board, p, q)`	(p, q) as a rational	2	`bil` with `gram`
`formIsInvariant(board, M)`	does every mutation preserve the form with matrix M? Returns an `InvarianceResult` with a witness when it does not	2	`form_is_invariant`, `form_invariance_witness`
`level(board, a)`	affine boards: $\sum c_i a_i$	2	`comarks`
`separatingRoots(board, a)`	positive roots β with $\langle a, \beta^\vee \rangle < 0$	3	`separating_roots`
`wavePlay(board, a, firstColour)`	rounds and legality of the wave	3	`wave_play`
`exponentsFromFired(fired, n)`, `weylOrder(board)`	F7	3	same names
`orbitSize(board, a)`	the size of the orbit of any position, without enumeration: play a to the end, let J be the nodes where the end is zero, and return $\lvert W\rvert/\lvert W_J\rvert$	3	`orbit_size` (which takes a position with no negative number)
`weylDimension(board, lambda)`	F9; `bigint`	3	`weyl_dimension`
`countCompleteGames(board, cap)`	F12, by counting paths layer by layer; `bigint`	3	`count_reduced_words_w0`
`legalLayers(board, start?, cap?)`	positions first reached after 0, 1, 2, … legal moves; `null` when more than `cap` positions are found (default 100000)	3	`legal_layers`
`allCompleteGames(board, start, cap)`	every complete legal play from `start`, as an `Enumeration` of words; works on any board, and stops at the cap when the plays do not end	3	`all_complete_games`
`heap(board, word)`, `heapKey`	brick levels; a key that is equal for plays with the same heap	3	`heap`, `heap_key`
`commutationClassSize(board, word)`	number of plays with the same heap; `bigint`	3	`commutation_class_size`
`symbolicPlay(board, word)`	the linear forms fired	3	`symbolic_play`
`fold(board, symmetry)`	a `FoldResult`	4	`fold`

`averagedRootsType(board, symmetry)`, `averagedSimpleCartan(board, symmetry)`	name and length classes of the averaged roots; the Cartan matrix of the averaged simple roots	4	same names, `classify_restricted`

The edges of a **move graph** are not stored by `orbitGraph`, which records only the first way each position was reached. A tool that draws all of them applies every move to every position of `items` once more.

Satake functions (phase 6):

Function	Meaning	Oracle
`theta(diagram, p)`	$-w_X \circ \tau$ on a population	`theta`
`shadow2(diagram, root)`	the integer vector $\alpha - \theta\alpha$	`shadow2`
`admissibility(diagram)`	an `Admissibility` record: each condition judged independently, with a reason for each failure	`admissibility`
`satake(diagram)`	the computed fields of section 6.4 (from `rank` to `restricted_reflection_word_lengths`); throws `NotAdmissible`	`satake`, `classify_restricted`
`restrictedBoard(diagram)`	orbits, simple restricted roots, restricted Cartan matrix, composite words	`restricted_board`
`satakeCatalogue(name)`	the entries for one board, generated from the parametric rules of section 6.4: label, real form, black nodes, τ, and the type as the tables print it	`satake_catalogue`
`satakeRecord(name, entry)`	the complete record of section 6.4: the catalogue fields, the fields of `satake`, and the C_2 adjustment described there	`satake_record`
`canonicalPair(diagram)`	a representative of the pair under the symmetries of the board	`canonical_pair`
`matchCatalogue(diagram)`	**all** catalogue entries in the same class: one, or two on D_4, or none	`match_catalogue`

Geometry (floating point):

Function	Meaning	Phase	Oracle
`choleskyBasis(board)`	upper triangular U with $U^{\mathsf{T}}U = B$	3	`cholesky_basis`
`symEig(M)`	eigenvalues and eigenvectors of a symmetric matrix, cyclic Jacobi	4	`sym_eig`
`euclideanBasis(board)`	the matrix E of section 2.14 and the eigenvalues	4	`euclidean_basis`
`coxeterPlanePoints(board)`, `coxeterPlaneRings(board)`	rows 1 and n of E applied to the roots; ring sizes and number of distinct points, with radii and coordinates rounded to 6 decimals and -0 counted as 0	4	same names
`coxeterWord(board)`	the bipartite Coxeter element as a word	4	`coxeter_word`
`applyWordFloat(board, v, word, game)`	`applyWord` on a vector of floats, for the "make uniform" display	4	none (the same formulas on floats)
`wythoffFaceTypes(board, rings)`, `wythoffFVector`	section 2.14, exact	4	same names
`wythoffEdges(board, rings, cap)`	exact	4	`wythoff_edges`

Representations:

Function	Meaning	Phase	Oracle
`minusculeMatrices(board, i, cap?)`	basis, E_k, F_k, H_k and the move matrices, as sparse integer matrices; `null` when the orbit of ω_i has more than `cap` positions (default 2000)	5	`minuscule_matrices`
`signedMoves(basis, E, F)`	the matrices $E_k - F_k + P_k^0$	5	`signed_moves`

Function	Meaning	Phase	Oracle
`chevalleySerre(board, E, F, H)`	a `RelationCheck` for every relation, with both sides	5	`chevalley_serre_ok`
`stringThrough(board, mu, i)`	the positions $\mu, \mu-\alpha_i, \ldots, \mu-\mu_i\alpha_i$	5	`string_through`
`orbitPlusZeroMatrices(board, lambda)`	section 2.18	7	`orbit_plus_zero_matrices`

Golden boards (phase 7): the type `ZPhi {a: bigint, b: bigint}` with its `Arith` instance; `coxeterH(n)`; `goldenFold(name, v)`. Oracle: class `Phi`, `coxeter_H`, `golden_fold`, `GOLDEN_FOLDS`.

5.5 Notes on algorithms

- **Orbits** are breadth-first with a hash set keyed by the entries joined as decimal text. Timings of a direct implementation of this, measured for this specification in Node 22 on one server processor with the script `reference/bench_orbit.mjs` (measurements, with no check in `verify.py`): the 240 roots of E_8 in about 2 ms; 2000 positions on a board with 12 nodes in 6 ms with `SafeInt` and 9 ms with `Big`; 5000 positions on that board in about 15 ms and 25 ms; 100000 positions in about 300 ms with `SafeInt` and 450 to 700 ms with `Big`. The figures are medians and vary by a few tens of percent from run to run. A mid-range phone is several times slower. The same script, written independently of the oracle, reproduces the complete layers recorded in `layer_growth_examples` for the star with five arms (10 at a cap of 2000, 17 at 100000). These figures are the basis of the rule of E-9 and of the caps of section 5.6. The site never asks for more than 100000 positions. The engine itself accepts any cap: the golden test of the star with five arms passes 150000 and finds 136008 positions in 18 layers.
- **Order and truncation of a search.** Positions are found in breadth-first order: the seeds in the order given, then, for each position in the order found, the nodes in increasing position. `items` is in that order, so the layers are contiguous. A position at depth `maxDepth` is not expanded; ending this way leaves `truncated` false. The cap acts when the search is about to add one position more than the cap: it stops at once with `truncated` true and exactly `cap` items. The last layer present is then incomplete, unless the cap fell exactly at the end of a layer. In both cases `completeLayers` is the depth of the position that was being expanded; when the search was not cut it is the largest depth present, which is also `depthReached`. A tool says "stopped after L layers" when `truncated` is false and `depthReached` equals `maxDepth` (the positions of that depth were not expanded, so more may exist), and "stopped after N positions; layers 0 to L are complete" when `truncated` is true. This is the oracle's `orbit_graph_capped`, and `layer_growth_examples` records its results at caps of 2000 and 100000.
- **The longest word** of any sub-board is obtained by playing the numbers game on that sub-board from all ones with the lowest-position rule. This one routine serves F4, w_X for Satake diagrams and the composite restricted moves.
- **Matching two boards** (`relabellings`): order the nodes of the second board so that each comes next to an earlier one; assign images one node at a time, keeping only candidates whose row and column of road counts match and whose entries against all nodes already placed agree; return all complete assignments in lexicographic order. This is the oracle's routine. It is fast for every board the tools offer and replaces any search over all permutations. (A board with a very large symmetry group, such as a star with eleven arms, has millions of matchings with itself; hence the limit of E-6. `identify` needs only the first few.)
- **Finite, affine or neither** (`cartanClass`): the rule of section 2.12. Compute the d_i by walking a spanning tree of the component and test $d_iC_{ij} = d_jC_{ji}$ on every pair; if that fails the board is indefinite. Otherwise compute the leading principal minors of $B = DC$ exactly with rationals. This needs n determinants; the definition by all principal minors needs 2^n and is used only by the oracle as a cross-check.
- **Naming** (`identify`, `identifyAffine`): compare with the standard matrix of each candidate of the same size. The finite candidates are tried in the order A, B, C, D, E, F, G, so the rank 2 board is named `B2`; the affine candidates are listed by `affine_candidates` of the oracle, where the affine board of B_2 appears once, as `C2~`.

- **The highest short root without the form** (`highestShortRoot`, phase 1): a finite-type board has one positive root with no negative number when it is simply laced and two otherwise; the one of smaller total is the highest short root [V on all 32 types].
- **Smallest non-finite sub-board**: subsets of the nodes by increasing size, in lexicographic order within a size; the first connected one whose class is not finite. At most 4095 subsets for 12 nodes.
- **Spectral radius**: power iteration on $M = A + I$, which converges also on bipartite boards where iteration on A oscillates. Start from the all-ones vector scaled to length 1; repeat $y = Mx$, estimate $x \cdot y$, $x = y/\|y\|$; stop when two successive estimates differ by less than 10^{-13} or after 5000 steps; subtract 1. This is the oracle's `spectral_radius` [V: $2\cos(\pi/(n+1))$ on paths, 2 on affine boards]. It is defined for `gcm` boards only.
- **Root test** (`isRootPopulation`): the greedy descent of section 2.11. Each step lowers the total population by at least one, so the test of $e_1 + k\delta$ on the triangle takes $2k$ steps; the cap of 20000 steps keeps a typed population from stalling the page, and the answer `null` is shown as "too large to test here".
- **Type of the restricted or averaged roots**: from the rank r, the number of vectors, whether the double of some vector is again in the set (then BC_r), and the numbers of vectors of each length; the decision table is the function `classify_restricted` of the oracle and must be ported as it stands. It writes the rank 2 system as `B2`.
- **Heaps**: brick k lands on level $1 +$ max of the levels of earlier bricks at the same or a joined node. **Plays with the same heap** are counted by dynamic programming over the sets of bricks already laid (a bit mask; a brick may be laid when every earlier brick at the same or a joined node has been).
- **Counting games**: dynamic programming over positions layer by layer from all ones; counts are `bigint`. Offered for $|W| \le 60000$.
- **Fates** (`playFate`): keep the set of positions seen; stop with `looped` at the first repeat, with `ended` when no number is positive, with `cap` otherwise. On a finite-type board the result is always `ended`.
- **Eigenvectors**: the cyclic Jacobi method of `sym_eig` is sufficient for $n \le 12$ and has no dependencies. Sort eigenvalues ascending. The sign and, for repeated eigenvalues, the choice of eigenvectors are not unique; tests therefore check invariants ($E^{\mathsf{T}}E = B$, ring sizes, the rotation by $2\pi/h$), never individual coordinates.
- **The class of a Satake pair** (`canonicalPair`): apply every symmetry g of the board to the pair, giving $(g(X), g\tau g^{-1})$, and keep the least result in the order "sorted black set, then the images under τ".

5.6 Caps, and what happens off the finite boards

Quantity	Cap	Behaviour at the cap
Nodes of a custom board	12	editor refuses a 13th
Roads between two nodes, each way (editor only; the engine accepts any count)	4	editor refuses more
Layers of a search on a board that `classify` does not call finite	40 (the `layers` property)	stop, report "stopped after 40 layers"
Items of an enumeration on the main thread (E-9)	2000	larger enumerations run in the worker
Positions held by the Orbit Garden (T2)	2000; every vertex and edge drawn up to 300	stop, report "stopped after 2000 positions" with the number of complete layers
Positions in an enumeration in the worker	100000	stop, `truncated: true`
Moves in "play to the end" and "play on" (in the worker when the board is not of finite type)	20000	stop, `status: 'cap'`, reported as "still running after 20000 moves"

Quantity	Cap	Behaviour at the cap
Moves in a link	20000	section 6.6
Vertices drawn (T10)	20000	tool refuses, with the orbit size
Edges drawn (T10)	70000 at rest; 8000 while turning	vertices only above 70000
Group order for the Weyl Explorer	1152	tool not offered
Group order for counting games	60000	count not offered
Steps of the root test (`isRootPopulation`)	20000	answer `null`, shown as "too large to test here"
Digits of an entry typed into T3	6	the field refuses more
Orbit for which T12 builds matrices	2000 positions	explanation only
Dimension for matrix grids	27	list view

Function group	On a `one-way` board	On a `gcm` board that is not of finite type
`mutate`, `fire`, `applyWord`, `orbit`, `orbitGraph`, `roots`, `play`, `playFate`, `raisingPlay`, `allCompleteGames`, matrices, `toNumbers`, `symbolicPlay`, `heap` and the other functions of words	work as defined	work; searches are bounded by `maxDepth` and by their caps
`components`, `bipartition`, `relabellings`, `symmetries`, `transposeBoard`, `subBoard`, `determinant`	work (roads in either direction join)	work
`toPopulation`	rationals when $\det C \neq 0$, else `null`	the same
`symmetrizer`, `gram`, `formValue`, `formIsInvariant`	`null` for the first three; `formIsInvariant` works	values, or `null` when not balanced
`classify`	record with `kind: 'one-way'`, `class: null`, `balanced: false`, `name: null`	`affine` or `indefinite`, with marks and comarks for affine
`identify`, `identifyAffine`, `smallestNonfiniteSubboard`, `spectralRadius`	`null`	`identify` gives `null`; the others work
`cartanClass`, `isRootPopulation`	throw `NeedsTwoWayRoads`	work (`cartanClass`, `identify` and `identifyAffine` require a connected board and throw `NeedsConnectedBoard` otherwise; `classify` is the function for boards in several pieces)
`affineExtension`	throws `NeedsTwoWayRoads`	throws `NotFiniteType`
`marks`, `comarks`, `level`	throw `NeedsTwoWayRoads`	work on affine boards; throw `NotAffine` (with the class in the message) on every other board, finite ones included
`weylOrder`, `weylDimension`, `orbitSize`, `longestWord`, `minusW0`, `minusculeNodes`, `highestRoot`, `highestShortRoot`, `coxeterNumber`, `wavePlay`, `countCompleteGames`, `legalLayers`, `coroot`, `separatingRoots`, `fold`, `averagedRootsType`, the Wythoff, Satake, representation and geometry functions	throw `NeedsTwoWayRoads`	throw `NotFiniteType`

A tool decides whether a board is of finite type by calling `classify` once when the board is loaded, never by running an enumeration into its cap.

5.7 Tests

Golden tests (Vitest). For every key of `golden_data.json` there is a test that recomputes the value with the engine and compares it under the rules below. The table says which functions each key exercises and in which phase its test must pass; gate Gk of section 9 requires the tests of phases 0 to k.

Golden key	Engine functions exercised	Phase
`types[*]`: `cartan`, `adjacency`	`cartan`, `adjacency`, `boardKind`	0
`types[*]`: `n_roots`, `n_positive_roots`, `positive_roots`, `highest_root`, `highest_short_root`, `numbers_of_highest_root`, `numbers_of_highest_short_root`, `highest_root_of_reversed_board`, `coxeter_number`, `longest_word`, `fired_values`, `fired_tally`, `minus_w0`, `minuscule_nodes`	`roots`, `positiveRoots`, `highestRoot`, `highestShortRoot`, `toNumbers`, `transposeBoard`, `coxeterNumber`, `play`, `minusW0`, `minusculeNodes`	1
`kostant`: `end_from_node`, `moves_from_node`, `final_numbers_from_node`	`kostantPlay`, `toNumbers`	1
`one_way_counterexample`, `layer_growth_examples`, `bigint_test`	`mutate`, `fire` with `Big`; `orbitGraph` with `maxDepth` and with a cap (`truncated`, `completeLayers`)	1
`types[*]`: `symmetrizer`, `simply_laced`, `n_short_positive`, `n_long_positive`, `long_to_short_ratio`, `det_cartan`	`symmetrizer`, `gram`, `formValue`, `determinant`	2
`affine`, `twisted_affine`, `affine_names`, `classification_tests`, `pentagon_example` (without `complete_games`), `kostant[*].affine_board_from_node`	`boardFromName`, `affineExtension`, `marks`, `comarks`, `classify`, `affineCandidates`, `identifyAffine`, `smallestNonfiniteSubboard`, `spectralRadius`, `level`, `playFate`, `isRootPopulation`	2
`types[*]`: `exponents`, `weyl_order`, `bipartite_colouring`, `fundamental_orbit_sizes`, `n_reduced_words_w0`, `length_distribution`	`exponentsFromFired`, `weylOrder`, `wavePlay`, `bipartition`, `orbitSize`, `orbit`, `countCompleteGames`, `legalLayers`	3
`weyl_dimension`, `heaps`, `pentagon_example.complete_games`, `e8` (without `h4_projection`)	`weylDimension`, `allCompleteGames`, `heap`, `commutationClassSize`, `toPopulation`, `formValue`	3
`folding` (without `quasi_split_real_form`), `wythoff`, `types[*].coxeter_plane`, `coxeter_plane_types_with_coincident_points`, `e8.h4_projection`	`fold`, `averagedRootsType`, `averagedSimpleCartan`, `identify`, `wythoffFVector`, `wythoffEdges`, `euclideanBasis`, `coxeterPlaneRings`	4
`minuscule_representations`	`minusculeMatrices`, `signedMoves`, `chevalleySerre`	5
`satake`, `admissible_pairs`, `admissible_counts`, `satake_examples`, `folding[*].quasi_split_real_form`	`admissibility`, `satake`, `restrictedBoard`, `satakeCatalogue`, `satakeRecord`, `matchCatalogue`	6
`golden_boards`, `golden_folds`, `orbit_plus_zero_representations`, `generalized_satake_example`	`ZPhi`, `coxeterH`, `goldenFold`, `orbitPlusZeroMatrices`, `shadow2`	7
`wildberger_lists`, `wythoff_hull_checks`, `wythoff_note`, `about`, `conventions`	none: documentation of what the oracle checked	none

Details of some rows. `determinant` is written in phase 1, because `toPopulation` uses it; its golden test, `det_cartan`, is listed with phase 2 and may be switched on earlier. `coxeter_number` is compared with $2N/n$ in phase 1 and, for rank at least 2, with the number of rounds of `wavePlay` in phase 3. `highest_short_root` is computed in phase 1 by the rule of section 5.5, with no form. `fundamental_orbit_sizes` is compared with `orbitSize`, and with the size of the enumerated orbit only where it is at most 20000. `classification_tests[*].spectral_radius_4_decimals` compares with the float formatted to four decimals. `affine_names` is compared with the names from `affineCandidates(m)` for $m = 2, \dots, 9$. `e8.h4_projection` is tested as in the oracle: the roots projected to the rows of E belonging to the exponents 1, 11, 19, 29 lie on two shells of 120 points whose radii are in the ratio φ to five decimals.

Comparison rules.

1. Integers compare exactly. Every JSON number in the golden file is a safe integer, except the ring radii of rule 3. Integers beyond 2^{53} are stored as decimal strings (`bigint_test`) and compare with `bigint` values by their decimal text.
2. Nodes in the golden file are labels (E-7); tests convert with `toLabels`. Appendix G lists the fields that hold nodes.

3. The only floating-point numbers in the file are the ring radii under `types[T].coxeter_plane.rings`. A ring list compares as: the same number of rings, the same sizes in the same order (innermost first), radii equal to within 10^{-5}. (`spectral_radius_4_decimals` is text, compared as text.)
4. Rationals are stored as text, `"p/q"` in lowest terms or `"p"` when the denominator is 1 (`form_value`, `edge_length_squared`), and compare as text.
5. Lists whose order has a meaning compare in order: words, fired values, `positive_roots` (sorted by total, then lexicographically), `length_distribution`, `new_per_layer`, orbits in the order given. `admissible_pairs[T]` is sorted and compares as a set. A list of records (`satake`, `folding`, `wythoff`, `weyl_dimension` and the like) is matched record by record through its identifying fields, never by index.
6. Identifiers compare exactly: board and type names (`B2`, `A4~`, `D4^(3)`, `BC2`), `class`, `kind`, `real_form`, `cartan_label`, `restricted_type`, `restricted_type_display`, `restricted_board`. Other text fields (`name` of a polytope, `label`, `note`, `rule`, `reason`, `reasons`) are documentation and are not compared.
7. A field that Appendix G describes as present only in some records (the short-root fields on simply laced types, `positive_roots` above rank 4 except on E_6, E_7, E_8, `n_reduced_words_w0` and `length_distribution` above $|W| = 60000$, `coxeter_plane` on A_1, `edge_length_squared`, `reverse_of`, `weights_dynkin_labels` above dimension 27) is not tested for a record in which it is absent or `null`. A `null` that carries meaning (`restricted_type` of a compact form, `folds_to` of a board that does not fold, `quasi_split_real_form` of a symmetry of order 3) compares as `null`.

Property tests (fast-check). The board is drawn at random from the type list, and vectors have random integer entries in $[-9, 9]$:

1. `mutate(B, mutate(B, p, i), i) = p` and `fire(B, fire(B, a, i), i) = a`.
2. `toNumbers(B, mutate(B, p, i)) = fire(B, toNumbers(B, p), i)` (section 2.4(a)).
3. `firingMatrix(B, i)` equals the transpose of `mutationMatrix(transposeBoard(B), i)` (section 2.4(b)).
4. Moves at unjoined nodes commute; $(S_i S_j)^m = 1$ with $m = 3, 4, 6$ for $A_{ij}A_{ji} = 1, 2, 3$.
5. `formValue` is unchanged by `mutate`.
6. Strong convergence: two random legal plays from one start give the same end, the same length and the same sorted list of fired numbers.
7. Every element of `roots(B)` has entries of one sign, and `isRootPopulation` accepts it and rejects its double.
8. `play` from a start with all numbers positive has length N and its word, applied to each simple root population, gives the negative of a simple root population.
9. `theta(theta(p)) = p` for every catalogue record of B, and `theta` permutes the roots.
10. On the affine board of B, `level` is unchanged by `fire`, and `mutate` fixes the marks.
11. For every row of `folding` with a folded board: symmetric play commutes with folding, in both games.
12. `heap` is unchanged by exchanging adjacent moves at unjoined nodes, and `commutationClassSize` is the same for both words.
13. `classify` of a randomly renumbered finite or affine board returns the same name, and `bourbaki_order` restores the standard matrix.

Cross-checks outside the repository [D]: SageMath reproduces the Cartan matrices, roots and Weyl group data without translation (`SOURCES.md` 11.5); CoReLG reproduces the Satake diagrams with the cautions listed in `SOURCES.md` 11.4. Such scripts are not part of this package. If the owner writes them they live under `reference/crosscheck/`, are run by hand, and never run in the site's build.

Regenerating the golden data: `python3 reference/verify.py <output file>`. The script needs only the standard library, exits with status 1 if any check fails, and writes every key in every run, so a regenerated file never silently loses a field. The command `npm run verify` writes to a temporary file and fails unless the result is byte-for-byte the committed `reference/golden_data.json`. The script is deterministic: its random choices use a fixed seed, and Python 3.11, 3.12 and 3.13 were each run for this specification and wrote the same bytes. Should another Python version ever write a different file, compare the two as parsed JSON under the rules above and report the difference to the owner; do not commit the new file. The engine never reads the Python; the JSON is the contract.

6. Data and drawing conventions

6.1 Node positions

Standard layouts, in grid units, used by every tool so that a board looks the same everywhere [D]:

Board	Positions
A_n, B_n, C_n, F_4, G_2	node i at $(i, 0)$
D_n	node i at $(i, 0)$ for $i \le n-2$; node $n-1$ at $(n-1,\ 0.6)$; node n at $(n-1,\ -0.6)$
E_6, E_7, E_8	nodes $1, 3, 4, 5, 6, 7, 8$ at $x = 1, \dots, 7$ on the line $y = 0$; node 2 at $(3, 1)$, above node 4
$\tilde{A}_n$, $n \ge 2$	a regular polygon with $n+1$ corners, node 0 at the top, nodes $1, \dots, n$ clockwise; $\tilde{A}_1$: nodes 0 and 1 side by side
$\tilde{B}_n$	as B_n, with node 0 at $(2, 1)$, above node 2
$\tilde{C}_n$, $\tilde{F}_4$, $\tilde{E}_7$	as the finite board, with node 0 at $(0, 0)$, to the left of node 1
$\tilde{D}_n$	as D_n, with node 1 moved to $(1, -0.6)$ and node 0 at $(1, 0.6)$, so that both ends show a fork at their neighbour
$\tilde{E}_6$	as E_6, with node 0 at $(3, 2)$, above node 2
$\tilde{E}_8$	as E_8, with node 0 at $(8, 0)$, to the right of node 8
$\tilde{G}_2$	as G_2, with node 0 at $(3, 0)$, to the right of node 2
Twisted boards	$A^{(2)}_{2n}$: node i at $(i, 0)$ for $i = 0, \dots, n$; the others: the layout of the untwisted board they reverse
The pentagon and the square (presets)	regular polygons, labels $1, \dots, n$ clockwise from the top
H_n (phase 7)	node i at $(i, 0)$; the last edge carries the label 5
Custom boards	where the reader put them: whole grid points, all different, between 0 and 11 in each direction; a custom board with no coordinates, or with coordinates that break this rule, is laid out on a circle in label order

Node 0 of an untwisted affine board is joined to the nodes on which the highest root has a positive number (section 2.12), which is what the table encodes. Node labels are always visible beside the nodes; they can be switched off only in exported figures.

Size on screen [D]. One grid unit is $u = \min(64, \lfloor (W - 16)/k \rfloor)$ CSS pixels, where W is the width of the board's box and k the number of grid columns the board occupies (for a row of nodes, the number of nodes). On a viewport narrower than 400 pixels the box runs from edge to edge of the viewport, so W is the viewport width. The hit area of a node is a square of side $\min(44, u)$ centred on it; with the layouts above no two hit areas overlap. A polygon layout is inscribed in a circle of radius $(\min(W, 360) - 60)/2$, which keeps the corners of a polygon with up to twelve nodes far enough apart for hit areas of 44. Section 7.5 (A-7) lists what this gives at a width of 320.

6.2 Roads

Two drawing styles [D]:

- `dynkin` (default for bidirected boards): one, two or three parallel lines between joined nodes; when the two counts differ, a chevron on the lines pointing in the direction of the extra roads, that is, from the long node to the short node. Pairs with $A_{ij}A_{ji} \geq 4$ (only on custom and affine boards) are drawn as a thick line with the two counts written at its ends.
- `roads`: every road is drawn as its own curved arrow, as in the Mars and Venus pictures. Used in M1, M4 and M6 where the counts in each direction are the subject, and **always** on a board with a one-way road, where the `dynkin` style would be ambiguous.

6.3 Values on nodes

- The value is the signed integer, set in tabular figures, with a true minus sign. Fractions (T1 in linked view from a start given as numbers, T6) are set as stacked fractions.
- Sign is shown three ways at once: by the numeral, by fill, and by colour.

	Light theme (page `#FFFFFF`, text `#1A1A1A`)	Dark theme (page `#121212`, text `#E8E8E8`)
Positive	filled `#0072B2`, numeral white (contrast 5.2 to 1)	filled `#56B4E9`, numeral `#121212` (8.1 to 1)
Negative	outlined `#D55E00` (3.9 to 1 against the page), numeral in the text colour	outlined `#E69F00` (8.3 to 1), numeral in the text colour
Zero	outlined `#767676` (4.5 to 1), numeral in the text colour	outlined `#8A8A8A` (5.4 to 1), numeral in the text colour

The hues are from the Okabe-Ito palette for colour-blind safety. The orange and blue are used only for fills and outlines, where WCAG asks for 3 to 1 (criterion 1.4.11). A numeral on a filled node has the colour given in the table (white in the light theme, `#121212` in the dark theme); every other numeral is in the page's text colour. All of these meet 4.5 to 1 (criterion 1.4.3). The ratios above were computed with the WCAG formula.

- "May act here" (legal firing, or raising move) is shown by a second ring and a pointer cursor, never by colour alone.
- The Populations view and the Numbers view have distinct, fixed accent colours and icons used consistently in prose, figures and formulas (principle P8) [D].
- Edges of orbit graphs are coloured by node from a palette of eight **and** carry the node label as text, so that the colour is never the only cue. Label k takes colour k for $k = 1, \ldots, 8$; label 0 takes colour 8, and labels 9 to 12 take colours 1 to 4, where the label tells them apart. Every colour gives at least 3 to 1 against its page [V]:

Colour	1	2	3	4	5	6	7	8
Light theme	`#0072B2`	`#D55E00`	`#009E73`	`#CC79A7`	`#B26F00`	`#3A8FCB`	`#807500`	`#1A1A1A`
Dark theme	`#56B4E9`	`#FF8A4C`	`#2DBE95`	`#CC79A7`	`#E69F00`	`#6FA8FF`	`#F0E442`	`#E8E8E8`

The light row is the Okabe-Ito palette with its orange, sky blue and yellow darkened, because those three fall below 3 to 1 on white.

- Typefaces: system font stacks for prose and for numerals (the monospaced system stack with tabular figures), so that no font files are needed beyond KaTeX's own [D].

- On the Satake Studio the decoration (black and white nodes, arrows) is drawn on its own copy of the diagram, placed directly above the game board with the nodes aligned, so that black and white never compete with the sign encoding [D].

6.4 The Satake catalogue

The catalogue has one record per real form of every board of rank at most 8 (`golden_data.json`, key `satake`, 167 records, compact forms included). It is produced by the oracle function `satake_catalogue` from the parametric rules below, and the engine function `satakeCatalogue` must produce the same records. Its content in closed form [V: every record is admissible and has the stated restricted roots, multiplicities and $\dim \mathfrak{k}$; the set of records equals the set of all admissible pairs up to symmetry. Every line was also compared with the Satake codes in the source of the LaTeX package `dynkin-diagrams`, whose table follows Helgason pp. 532-534, and all 28 patterns of black nodes and folds agree [S]]:

Label	Real form	Board	Black nodes X	τ	Restricted roots	Multiplicities $(m_\lambda, m_{2\lambda})$ at the simple restricted roots	$\dim \mathfrak{k}$
A I	$\mathfrak{sl}(n+1,\mathbb{R})$	A_n	none	identity	A_n	all $(1,0)$	$\frac{1}{2}n(n+1)$
A II	$\mathfrak{su}^*(2m)$, $n=2m-1\geq 3$	A_n	odd nodes	identity	A_{m-1}	even nodes $(4,0)$	$m(2m+1)$
A III ($p\geq 2$), A IV ($p=1$)	$\mathfrak{su}(p,q)$, $p<q$, $p+q=n+1$	A_n	$p+1,\dots,n-p$	flip $i\leftrightarrow n+1-i$	BC_p	orbits $\{i,n+1-i\}$, $i<p$: $(2,0)$; orbit $\{p,n+1-p\}$: $(2(q-p),1)$	p^2+q^2-1
A III	$\mathfrak{su}(p,p)$, $p\geq 2$, $n=2p-1$	A_n	none	flip	C_p	orbits $\{i,2p-i\}$, $i<p$: $(2,0)$; node p: $(1,0)$	$2p^2-1$
B I ($p\geq 2$), B II ($p=1$)	$\mathfrak{so}(p,q)$, $p\leq n$, $p+q=2n+1$	B_n	$p+1,\dots,n$	identity	B_p (A_1 if $p=1$)	nodes $i<p$: $(1,0)$; node p: $(q-p,0)$	$\binom{p}{2}+\binom{q}{2}$
C I	$\mathfrak{sp}(2n,\mathbb{R})$	C_n	none	identity	C_n	all $(1,0)$	n^2
C II	$\mathfrak{sp}(p,q)$, $p<q$, $p+q=n$	C_n	$1,3,\dots,2p-1$ and $2p+1,\dots,n$	identity	BC_p	nodes $2i$, $i<p$: $(4,0)$; node $2p$: $(4(q-p),3)$	$p(2p+1)+q(2q+1)$
C II	$\mathfrak{sp}(p,p)$, $n=2p$	C_n	$1,3,\dots,n-1$	identity	C_p (A_1 if $p=1$)	nodes $2i$, $i<p$: $(4,0)$; node n: $(3,0)$	$2p(2p+1)$
D I	$\mathfrak{so}(n,n)$	D_n	none	identity	D_n	all $(1,0)$	$n(n-1)$
D I	$\mathfrak{so}(n-1,n+1)$	D_n	none	swap $n-1\leftrightarrow n$	B_{n-1}	nodes $i\leq n-2$: $(1,0)$; orbit $\{n-1,n\}$: $(2,0)$	$\binom{n-1}{2}+\binom{n+1}{2}$
D I ($p\geq 2$), D II ($p=1$)	$\mathfrak{so}(p,q)$, $p\leq n-2$, $p+q=2n$	D_n	$p+1,\dots,n$	identity if $n-p$ is even, swap if odd (no arrow is drawn)	B_p (A_1 if $p=1$)	nodes $i<p$: $(1,0)$; node p: $(2(n-p),0)$	$\binom{p}{2}+\binom{q}{2}$
D III	$\mathfrak{so}^*(2n)$, n even	D_n	$1,3,\dots,n-1$	identity	$C_{n/2}$	nodes $2i<n$: $(4,0)$; node n: $(1,0)$	n^2
D III	$\mathfrak{so}^*(2n)$, n odd	D_n	$1,3,\dots,n-2$	swap	$BC_{(n-1)/2}$	nodes $2i<n-1$: $(4,0)$; orbit $\{n-1,n\}$: $(4,1)$	n^2
compact	$\mathfrak{su}(n+1)$, $\mathfrak{so}(2n+1)$, $\mathfrak{sp}(n)$, $\mathfrak{so}(2n)$		all	$-w_0$: the flip on A_n ($n\geq 2$), the swap on D_n for odd n, otherwise identity	none		$\dim \mathfrak{g}$

The exceptional forms, and the classical forms of low rank written out as test vectors, are in Appendix B.

Names in rank 2. Where the table says C_p or $C_{n/2}$ and the rank is 2, the system is the same as B_2. Each record therefore carries two fields: `restricted_type`, computed, which always writes `B2`;

and `restricted_type_display`, from the table above, which writes `C2` for $\mathfrak{su}(2,2)$, $\mathfrak{sp}(4,\mathbb{R})$, $\mathfrak{sp}(2,2)$ and $\mathfrak{so}^*(8)$ and `B2` for $\mathfrak{so}(2,q)$. The site prints the display field. The restricted **board** is named in `restricted_board`: B_r for a BC_r system (A_1 for $r=1$), and for the four records just named `C2`, with `restricted_orbits_in_bourbaki_order` listing the short node first, so that the board is drawn like its relatives C_p with larger p. This adjustment is made by `satake_record` (engine: `satakeRecord`); the plain function `satake` would say `B2`.

Coincidences. The catalogue is organised by board and does not merge coincidences between boards: B_2 and C_2 each list the same three algebras under their two families of names. Inside one board there is exactly one coincidence: on D_4 the records $\mathfrak{so}(2,6)$ and $\mathfrak{so}^*(8)$ are the same real form, so the 167 records make 166 classes [V].

What has and has not been confirmed for the multiplicities is stated in `SOURCES.md` 11.4 and Appendix E: the types of the restricted root systems were confirmed in two sources for every line; the multiplicities were confirmed in a source for the Grassmannian families with $p<q$, for the rank-one forms, for E III, E IV and F II, and by computation for the rest.

Fields of a catalogue record (JSON; produced by `satake_record`. Every record has every field; for a compact form the lists are empty and `restricted_type`, `restricted_type_display`, `restricted_board` and `restricted_orbits_in_bourbaki_order` are `null`): `dynkin_type`, `cartan_label`, `real_form`, `black_nodes`, `tau` (images of all nodes), `arrows` (pairs of white nodes), `tau_moves_black_nodes`, `rank`, `real_rank`, `restricted_type`, `restricted_type_display`, `multiplicities` (one entry per length class of restricted roots, with the fields `class`, `roots`, the number of restricted roots in the class, and `mult`, their multiplicity), `n_restricted_roots`, `dim_g`, `dim_k`, `dim_p`, `character`, `roots_compact_imaginary`, `roots_real`, `roots_complex`, `w_X_word`, `restricted_simple_orbits`, `restricted_simple_multiplicities` (one pair $(m_\lambda, m_{2\lambda})$ per orbit), `restricted_cartan` (rows and columns in the order of `restricted_simple_orbits`), `restricted_board`, `restricted_orbits_in_bourbaki_order`, `restricted_reflection_word_lengths`.

6.5 Names of real forms

Display names: $\mathfrak{sl}(n,\mathbb{R})$; $\mathfrak{su}^*(2m)$, also shown as $\mathfrak{sl}(m,\mathbb{H})$; $\mathfrak{su}(p,q)$; $\mathfrak{so}(p,q)$; $\mathfrak{sp}(2n,\mathbb{R})$ for the split form of C_n; $\mathfrak{sp}(p,q)$ with $p+q=n$ for the other forms of C_n; $\mathfrak{so}^*(2n)$; exceptional forms as $\mathfrak{e}_{6(6)}$, $\mathfrak{e}_{6(2)}$, $\mathfrak{e}_{6(-14)}$, $\mathfrak{e}_{6(-26)}$, $\mathfrak{e}_{7(7)}$, $\mathfrak{e}_{7(-5)}$, $\mathfrak{e}_{7(-25)}$, $\mathfrak{e}_{8(8)}$, $\mathfrak{e}_{8(-24)}$, $\mathfrak{f}_{4(4)}$, $\mathfrak{f}_{4(-20)}$, $\mathfrak{g}_{2(2)}$, where the number in brackets is the character. In data the names are plain text: `sl(4,R)`, `su*(4)`, `su(1,3)`, `so(3,5)`, `sp(6,R)`, `sp(1,2)`, `so*(8)`, `e6(-14)`, and `su(4)`, `so(7)`, `sp(3)`, `e6(-78)` and so on for the compact forms. Cartan's labels (A I, …, G) are shown beside them. Some references write $\mathfrak{sp}(2p,2q)$ and $\mathfrak{sp}(8)$ for what is here $\mathfrak{sp}(p,q)$ and $\mathfrak{sp}(4)$; the Atlas notes this.

6.6 State, links and file formats

State is a plain object. Its fields for the game tools:

```
{ "schema": 1, "tool": "board", "board": "E8", "view": "linked", "mode": "legal",
  "coords": "numbers", "start": [1,1,1,1,1,1,1,1], "moves": [1,2,3,1] }
```

Field	Values	Meaning
`schema`	1	version of this format
`tool`	the ids in the table at the head of section 4	which tool the state belongs to
`board`	a type name (`"E8"`), an affine name (`"B3~"`, `"D4^(3)"`), a preset (`"pentagon"`, `"square"`), or `{"cartan": [[2,-1],[-3,2]], "xy": [[0,0],[1,0]]}` for a custom board (`xy` optional)	the board

Field	Values	Meaning
`view`	`populations`, `numbers`, `linked`	section 4, T1
`mode`	`free`, `legal`, `raising`	section 4, T1
`coords`	`populations`, `numbers`	**which game `start` is a position of**. Required whenever `start` is present.
`start`	integers, one per node, in the order of the board's labels	the starting position; default: all ones for numbers, one Martian in the first city for populations
`moves`	node labels	the moves made so far

Tool-specific fields: `black` and `tau` (T13: labels of black nodes; the images of all nodes); `rings` and `projection` (T10: one non-negative integer per node; `coxeter` or `xyz:1,2,3`, either with three turning angles); `graph` (T10: `polytope` or `moves`); `symmetry` (T11: the images of all nodes); `node` and `relation` (T12); `lambda` (T5 dimension calculator). Options that are not listed here (the growth mode of T2, the weights typed into T3, the tab of T5, T11 or T15, the root chosen in T13, a rotation of T10 in a coordinate plane beyond the three turning angles) are not part of a link in schema 1; a link opens the tool with those at their defaults.

Instances. Every tool placed on a page has an instance id, set by the page author (`id="b2"`) and unique on the page. A page holds one state object per instance.

URL form. Readable query parameters, one letter each: `z` schema, `g` board, `v` view (`p`, `n`, `l`), `m` mode (`f`, `l`, `r`), `c` coords (`p`, `n`), `s` start, `w` moves, `x` black nodes, `t` tau, `r` rings, `j` projection, `e` graph (`p`, `m`), `y` symmetry, `k` node, `q` relation, `d` lambda. Example: `?z=1&g=E8&v=l&m=l&c=n&s=1,1,1,1,1,1,1,1&w=1.2.3.1`. Custom boards: `g=c:2,-1;-3,2` (rows separated by semicolons). Satake: `x=2.3.4.5&t=1.2.3.4.5.6`. The values:

Key	Value
`g`	a board name (`E8`, `B3~`, `D4^(3)`, `pentagon`), or `c:` and the rows of the Cartan matrix, entries separated by commas and rows by semicolons, optionally followed by `@` and the grid coordinates of the nodes written the same way (`c:2,-1;-3,2@0,0;1,0`)
`s`, `d`	one integer per node in label order, separated by commas
`w`	node labels in the order played, separated by dots; empty for no move
`x`	the labels of the black nodes, separated by dots; empty for none
`t`, `y`	the image of every node, in label order, separated by dots
`r`	one non-negative integer per node in label order, separated by dots
`j`	`coxeter`, or `xyz:` and three coordinate numbers counted from 1 (`xyz:1,2,3`); either may be followed by `@` and three whole numbers of degrees, the turn of the picture about its horizontal, vertical and viewing axes (`xyz:1,2,3@30,20,0`)
`k`	one node label

Key	Value
`q`	a relation code of section 5.2 with its node labels: `ef:1,2`, `serre-e:2,3`

A key that is absent keeps the value the page author configured for that instance (on a tool's own route: the tool's default), so a link changes only what it names. `s` must come with `c`. On a tool's own route (`/tools/...`, `/embed/...`) the keys are bare. On a lesson page each key is prefixed by the instance id and a dot (`b2.g=A3&b2.w=1.2`), and only instances the reader has changed are written. The query string is assembled by hand, so that commas, semicolons, dots, `~` and `^` stay readable, and is parsed with `URLSearchParams`. A link carries no site version; the "Cite" button gives a citation text that includes the version.

Validation. The codec validates everything it reads. An unknown key is ignored. Any of the following makes the instance load its default state and show a notice that the link could not be read: a known key with a value that does not parse; a board that is `not-a-board` or has more than 12 nodes; a `g` parameter longer than 512 characters; a start with an entry longer than 40 digits; more than 20000 moves; a move that the mode in force forbids at that point of the play; a `t` or `y` that is not a symmetry of the board, or a `t` whose square is not the identity; a board that the tool does not offer (section 4); a list (`s`, `d`, `r`, `t`, `y`) whose length is not the number of nodes; a label in `w`, `x`, `k` or `q` that is not a label of the board; a view and a mode that the table of T1 does not allow together; `s` without `c`; an unknown schema number. Coordinates after `@` that break the rule of section 6.1 spoil only the layout, which falls back to the circle. A missing `z` is read as schema 1. Old schema versions stay readable for as long as the site exists.

Data export from any tool: JSON with the state, the engine version, the site's git commit, and the derived tables shown on screen (positions, roots, edges, matrices), all integers as JSON numbers when safe and as decimal strings otherwise.

6.7 LaTeX export

Diagrams are exported for the `dynkin-diagrams` package with explicit marks in Bourbaki order, one character per node (`o` white, `*` black), and arrows as an option:

```
\dynkin[involutions={16}] E{oo***o}   % E III, e6(-14)
\dynkin E{o*oo*o*}                    % E VI, e7(-5)
\dynkin[involutions={15;24}] A{oo*oo} % su(2,4)
```

This syntax was compiled and inspected by hand during this research, with the version of the package dated 2 June 2023 [S; this is not a check of `verify.py`]; the current release on CTAN is dated 4 December 2024 and its manual shows the same forms [S]. The package's own named diagrams (`\dynkin E{VI}` and so on) are not used for export because the mapping of its sub-labels to particular ranks was not checked. Orbit graphs and projections are exported as plain TikZ coordinates. Compiling one exported file of each kind is in the owner's part of gate G6, since the build pipeline has no TeX installation.

7. Architecture

7.1 Stack [D]

The site is static: HTML, CSS and JavaScript files served from a CDN, with no server-side code and no runtime dependency on any service.

Layer	Choice	Version to pin (registry state on 9 October 2026 [S])	Why
Runtime for building	Node	22.12 or later (LTS line 22)	required by Astro 7 and Vitest 5
Package manager	npm workspaces	the npm that ships with Node 22	no extra tool; one `package-lock.json` at the root
Site generator	Astro, static output	`astro` 7.3.8	pages are HTML first; only the tools hydrate ("islands"), so a page reads without JavaScript and a broken tool cannot break a page
Prose	MDX	`@astrojs/mdx` 8.0.3	lessons are text with components placed in it
Tools	Svelte 5 components	`svelte` 5.57.2, `@astrojs/svelte` 9.0.1	SVG written as markup; small output; compile-time accessibility warnings
Language	TypeScript, strict	`typescript` 6.0.3 (not 7; see 7.2)	
Engine	plain TypeScript package with no dependencies, including its own small rational type		section 5
Mathematics in prose	KaTeX rendered at build time	see 7.2	no typesetting script is sent to the reader; KaTeX output carries MathML for assistive technology
Drawing	SVG for boards, diagrams, posets and small graphs; Canvas 2D for projections with many edges	no library	the projector for higher-dimensional point sets is a few hundred lines of our own code (rotations in coordinate planes, orthogonal projection, depth cue)
Shared state between islands	Nano Stores	`nanostores` 1.5.5	islands hydrate separately and cannot share a framework context
Unit and property tests	Vitest, fast-check	`vitest` 5.0.3, `fast-check` 4.10.2	
Browser tests	Playwright with axe	`@playwright/test` 1.64.0, `@axe-core/playwright` 4.13.0	

Layer	Choice	Version to pin (registry state on 9 October 2026 [S])	Why
Performance checks	a size script of our own (Playwright and Node's `zlib`); Lighthouse CI for lab timings	Lighthouse CI: the version current when phase 3 starts [U: not read from the registry]; section 7.6 gives a fallback that needs only Playwright	the budget of section 7.6
Hosting	GitHub Pages through the official Astro action		free for public repositories; Cloudflare Pages is the fallback

Not used in version 1, with reasons: three.js (0.186.1) and Threlte, because every picture needed is a projection of points and segments, which a Canvas 2D projector draws with full control and no 300 kB dependency (revisit if shaded solids are wanted); Rust and WebAssembly, because the data are small and a TypeScript engine is sufficient; Pyodide, because shipping Python to the browser would cost megabytes; a fraction library, because the engine's own rational type is thirty lines; a layout library for graphs, because every graph the site draws is ranked (by number of moves or by height) and a purpose-built layered layout is simpler and deterministic. If a general layout is ever needed, use `d3-dag` 1.2.2 or `@dagrejs/dagre` 3.1.1.

Alternative stack, if the owner prefers one continuous application to a set of articles: SvelteKit 3 with `adapter-static` and mdsvex, same engine and tests. Its cost is that SvelteKit 3.0 was released on 1 October 2026 and every page hydrates in full. A React variant (Astro with `@astrojs/react`) is acceptable only if the maintainers strongly prefer React. Both are open decisions in section 10.

7.2 Recent changes the builder must know about

These postdate most training data. Each was read from the package registry or the project's own published files on 9 October 2026 unless marked.

1. **Astro 7** (7.0.0 on 22 June 2026) builds with Vite 8 and requires Node 22.12 or later. Its **default Markdown processor is new** (Sätteri) and does not run remark or rehype plugins. To use them, install `@astrojs/markdown-remark` (7.3.2), `import { unified } from '@astrojs/markdown-remark'`, and set `markdown: { processor: unified({ remarkPlugins, rehypePlugins }) }` in `astro.config.mjs`; the `processor` option is present in the published type definitions of `astro` 7.3.8 [S]. The same package exposes a renderer hook for MDX, which indicates that `.mdx` files go through the same processor [S2: read in the package by a reviewer of this specification; not run]. Astro 7's compiler rejects invalid HTML that earlier versions tolerated [S2].
2. **KaTeX** had three breaking minor releases in 2026: 0.17 (May), 0.18 (July; CSS class names were prefixed, so custom selectors written for 0.16 fail), 0.19 (1 October) [S2 for the content of the changes; the versions are [S]]. `rehype-katex` 7.0.1 still depends on `katex ^0.16`, and `remark-math` is 6.0.0. **The stylesheet and fonts shipped must come from the same KaTeX version that produced the HTML.** Self-host them.
3. **How mathematics is rendered is decided by a spike in phase 0**, between (a) `remark-math` and `rehype-katex` under the `unified()` processor, so that authors write `$...$` in MDX, and (b) a build-time component `<Tex src="..."/>` that calls `katex.renderToString` in Node. With route (a) the KaTeX version is the 0.16 release that `rehype-katex` resolves, and the stylesheet is taken from that same installed package. With route (b) pin the current KaTeX (0.19.x) directly. Route (b) depends on nothing but KaTeX and always works; route (a) is more pleasant to write. The spike renders one `.mdx` page with inline and display mathematics by each route and records the result in `DECISIONS.md`. A third candidate, `satteri-katex` 0.1.1, is too young to rely on. If both routes work, lessons use

(a). The component `<Tex>` is built in either case, because a formula that contains a golden value goes through it (section 7.3). Under route (a), `<Tex>`, `scripts/names.mjs` and `sources.astro` import the same installed KaTeX that `rehype-katex` resolves, so that one stylesheet fits all the mathematics of a page; a second, newer KaTeX beside it would put two kinds of markup under one stylesheet.

4. **TypeScript 7.0** (July 2026) is a native binary and lies outside the peer ranges of `@astrojs/svelte`, `svelte-check` and `@astrojs/check`, which stop at 6. Pin 6.0.x.
5. **Svelte 5** uses runes (`$state`, `$derived`, `$props`, `$effect`) for component state. Do not write Svelte 4 patterns (`export let`, `$:` blocks, `writable` stores for local state). One store idiom remains in use: a Nano Store read in a component as `$name`, which Svelte 5 still supports for objects with a `subscribe` method.
6. **Vitest 5** (September 2026) and **Vite 8** (March 2026) are current; older configuration examples may not apply.
7. **SvelteKit 3.0** (1 October 2026) has many breaking changes. It is not part of the default stack.
8. The unscoped npm package `dagre` is frozen; the maintained one is `@dagrejs/dagre`.
9. The Astro upgrade guide is reported to say that `astro dev` detects coding agents and runs as a background server [S2]. For automated checks prefer `astro build` followed by `astro preview`.
10. **GitHub Pages serves a project site under `/<repository>/`.** `astro.config.mjs` takes `site` and `base` from the environment, so that nothing has to be decided before the repository exists: the variables `SITE_URL` and `BASE_PATH` when they are set; otherwise, in CI, the values that follow from `GITHUB_REPOSITORY` (`owner/repo` gives `https://owner.github.io` and `/repo/`); otherwise `http://localhost` and `/`. Every internal link and asset path is built from `import.meta.env.BASE_URL`. One browser test runs the site built with `BASE_PATH=/sub/path/`, to catch a link that was typed with a leading slash. GitHub Pages has no preview deployment per pull request; pull requests are checked by the CI build, which uploads `dist/` as an artifact.

Rule: commit the lockfile, pin exact versions in `package.json`, and record the versions in the footer of the built site.

7.3 Repository layout and pages

```
/
  SPEC.md  SOURCES.md  CLAUDE.md  DECISIONS.md  LICENSE  LICENSE-content
  package.json  package-lock.json            (npm workspaces: packages/*, site)
  reference/            lie_games.py  verify.py  golden_data.json  bench_orbit.mjs
  scripts/              size.mjs  verify.mjs  numbers-in-prose.mjs  names.mjs
  packages/
    engine/             src/*.ts   test/golden/*.test.ts   test/properties/*.test.ts
  site/
    astro.config.mjs
    src/
      content/modules/  m0-start.mdx ... m10-atlas.mdx
      content/pages/    glossary.mdx  history.mdx  open-problem.mdx  confusions.mdx
                        not-quiver-mutation.mdx  credits.mdx  about.mdx
      content/unverified.json                (the open items of Appendix E)
     generated/      names.json          (names typeset at build time by scripts/names.mjs)
      components/
      board/        Board.svelte  Node.svelte  Roads.svelte  History.svelte  Predict.svelte
        tools/          T01GameBoard.svelte ... T16Sandbox.svelte
        math/           Tex.astro  MatrixView.svelte  Frac.svelte  LinearForm.svelte
        content/        G.astro  Unverified.astro  Cite.astro
        draw/           projector.ts  layeredLayout.ts
      lib/              state.ts  urlCodec.ts  worker.ts  export.ts  a11y.ts  golden.ts
      pages/            index.astro  modules/[slug].astro  tools/[tool].astro
                        atlas/[board].astro  embed/[tool].astro  sources.astro
                        open-items.astro  [page].astro
```

```
    styles/
  tests/unit/          *.test.ts (Vitest: urlCodec, golden paths, layout)
  tests/e2e/           *.spec.ts (Playwright)
.github/workflows/     ci.yml  deploy.yml  links.yml
```

The engine is a workspace package with its own tests and no import from `site/`. Components contain no mathematics beyond calling the engine. The four scripts: `size.mjs` measures the budget of section 7.6; `verify.mjs` is the command `npm run verify` of section 5.7; `numbers-in-prose.mjs` is the report of section 7.7; `names.mjs` writes the typeset names described below and runs at the start of every build.

Pages other than lessons and tools, with the phase in which each appears: `about` and `credits` (phase 0, extended at each release); `glossary`, seeded from Appendix C (phase 1, growing); `open-problem`, the Mutation Conjecture page (phase 2); `sources`, generated from `SOURCES.md`, and `open-items`, generated from `content/unverified.json` and present in preview builds only (both phase 1, so that citations and marks have somewhere to point from the first lesson on); `not-quiver-mutation`, the page of section 2.16 (phase 5); `history`, `confusions`, the Atlas pages (phase 6). A link to a page that has not been released yet is left out by the build, driven by a `released` flag in each page's front matter.

The sources page is built from `SOURCES.md` by `sources.astro`, which renders the file as Markdown (not MDX, which rejects its angle-bracket links) and publishes sections 11.1 to 11.6 with the status labels spelled out in words. Sections 11.0 and 11.7 to 11.9 are the research record and are not published. The formulas of the file (`$...$`) are typeset at build time with the same KaTeX call that `<Tex>` uses, whichever route is chosen for lessons, because the file does not pass through the MDX pipeline.

Four content components make the content rules checkable by the build:

- `<G k="types.E8.weyl_order"/>` prints a value of `golden_data.json` at build time and fails the build unless the path leads to exactly one value. Every number in a lesson that has a golden value is written this way, so prose cannot drift from the data.
  - *Path.* Steps are separated by dots. A step is a key of an object (`types.E8`), a key followed by a zero-based index into a list (`positive_roots[0]`), or a key followed by a selection from a list of records, written `[field=value;field=value]`, where each value is taken literally up to the next semicolon or closing bracket and compared with the field as text (a field that holds a list of integers is compared as its entries joined by commas): `satake[dynkin_type=E6;real_form=e6(-14)].dim_k`, `folding[type=D4;order=3].folds_to`, `wythoff[type=F4;rings=1,0,0,0].f_vector`. A selection must match exactly one record. The resolver is one function in `lib/golden.ts`, shared with `<Tex>`.
  - *Output.* An integer is printed in decimal digits with a true minus sign, and with its digits grouped in threes by narrow no-break spaces when it has five digits or more. A list of integers is printed as its entries separated by commas, in round brackets. A string is printed as it stands. The attribute `as` changes this: `as="word"` prints a list of node labels separated by thin spaces, `as="count"` prints the length of a list, `as="plain"` prints an integer without grouping or a list without brackets.
- `<Tex src="|W(E_8)| = {{types.E8.weyl_order}}"/>` typesets a formula at build time with KaTeX, after replacing each `{{path}}` by the golden value (same paths, same failure when one does not resolve; digits grouped with `\,`). It is the only way a golden value enters a formula. The attribute `display` asks for display style. The phase 0 spike confirms that backslashes in the attribute reach KaTeX unchanged from an `.mdx` file.
- `<Unverified item="E4" fallback="..."> ... </Unverified>` wraps any sentence that rests on an open item of Appendix E, named by its number there. `content/unverified.json` lists every item with a status, `open` or `closed`, and for a closed item the date and the source that settled it. Only the owner sets an item to `closed`. A preview build (the default) renders the wrapped sentence with a visible mark that links to the item on the page `open-items`. A release build (`RELEASE=1 npm run build`) renders a closed item's sentence unmarked, and for an open item renders the `fallback` text in its place, or nothing when no fallback is given. A fallback is a weaker sentence that stays true however the item turns out, for example "computed for every board of rank at most 8" in place of a statement about what a book says. A release build never fails because of an open item; it prints a report of the

open items and the pages that use them, which the owner reads at each release (section 9.1).
- `<Cite k="Don08"/>` renders the key in square brackets as a link to that entry on the sources page, with the entry's text as the link's title, and fails the build if the key is not defined in `SOURCES.md`.

Mathematics produced while a tool runs. KaTeX is never sent to the reader (section 7.6), so whatever a tool typesets after the page has loaded is built from plain markup by three small components: `MatrixView.svelte` for matrices (an HTML table with a caption and row and column headers); `Frac.svelte` for the stacked fractions of T1 and T6; `LinearForm.svelte` for the linear forms of T7 and the relations of T12, written as MathML (`<math>` with `<mi>`, `<mn>`, `<mo>`, `<msub>`), which the three browser engines render natively [U: MathML Core in Chromium since version 109, from memory; check in phase 3, and if one engine fails, use HTML spans with an `aria-label`]. Names with a fixed spelling (boards, real forms, Cartan's labels) are typeset once at build time: `scripts/names.mjs` writes the KaTeX markup of every name of the catalogue to `site/src/generated/names.json`, and the tools that show names (T13, T14, the board pickers) look them up there. The file is produced by the build and is not committed.

7.4 State, links, embedding, and life without JavaScript

- **Single source of truth.** A tool's state is the object of section 6.6. Positions are never part of the state; they are recomputed from the start and the moves. A link can therefore never show a position the rules do not allow. While a tool runs it keeps the list of positions of the current play in memory and extends it by one position per move, so a move costs one engine call however long the play is.
- **URL form and validation**: section 6.6.
- **History API.** After a move the address bar is updated with `replaceState`, half a second after the last move of a burst and inside a `try` block, so that the bar holds a shareable link, the back button is not flooded, and fast play cannot run into a browser's limit on such calls [U: Safari is remembered to refuse more than 100 in 30 seconds]. `pushState` is used only when the reader loads an example or a catalogue entry. "Copy link" always encodes the current state, whatever the address bar holds at that moment.
- **Tools on one page.** Each tool keeps its own state. Two tools share state only when the page says so with a `linkTo` property (the Orbit Garden following a Game Board, the Weyl Explorer driven by one). Independently of that, one small store per page holds the **highlight** (a node, a root, a position), so that hovering a root in the garden highlights it on every board and projection of the page that shows it.
- **Citing.** "Copy link" gives the link of section 6.6, which carries no site version. "Cite" gives a citation text with the page title, the site version (git commit), the date and the link. Old schema versions stay readable.
- **Embedding.** Every tool has a route `/embed/<tool>` that renders the tool alone for use in an `<iframe>`, accepting the same query parameters. Web components are a phase 7 option.
- **Without JavaScript.** Astro renders every Svelte island to HTML at build time. Each tool therefore ships its default state as real markup: the SVG board with its values, the caption, the data table of A-10. Controls are rendered with the `disabled` attribute, which the island removes when it hydrates, so a control never looks usable before it is. A `<noscript>` line under each tool says that the figure is interactive when scripts are allowed. Tools that draw on a canvas (T10, and T8 above 192 elements) ship an SVG snapshot of their default view, produced at build time by the same drawing code running in Node (the projector for T10, the layered layout for T8). A Playwright run with JavaScript disabled checks, for every lesson, that the text, every default figure and every table are present.

7.5 Accessibility requirements

Target: WCAG 2.2 level AA [S for the criteria]. Two requirements below go beyond AA by choice and are marked. Requirements that automated scans do not catch:

Id	Requirement	Criterion
A-1	Every tool is fully operable from the keyboard. A board is a group with an accessible name; its nodes are real `<button>` elements, one of which is in the tab order (roving `tabindex`). Arrow keys move the focus as follows. On the standard layouts of section 6.1 that are rows with branches, an arrow moves to the nearest node whose centre lies within 45 degrees of the arrow's direction, the limit included, ties going to the smaller label; when no node lies there, Right and Down move to the next label and Left and Up to the previous one, stopping at the ends. On polygon layouts and on custom boards the arrows follow the labels and wrap round: Right and Down to the next label (clockwise on a polygon), Left and Up to the previous one. Every node is reachable on every layout. Home and End go to the first and last label; Enter or Space acts. Undo, redo and reset are visible buttons after the board in the tab order; the shortcuts `u` (undo) and `r` (redo) work only while focus is inside the board. Because the nodes are ordinary buttons, a screen reader in browse mode reaches them without the arrow-key layer. Focus is never trapped in a canvas.	2.1.1, 2.1.2, 2.1.4, 2.4.3
A-2	A visible focus ring on the focused node, never hidden by sticky elements.	2.4.7, 2.4.11
A-3	Each node button has a name and state built from a fixed template: "Node 3, value 2, can be fired"; "Node 5, value 0, cannot be fired" (in legal play; in free play every node can be fired); "City 2, population 1, mutation would raise it".	4.1.2

Id	Requirement	Criterion
A-4	Each move is announced through one polite live region per tool that exists, empty, before the first move. Templates, with negative numbers spoken as “minus”: “Fired node 3. Numbers now minus 1, 2, minus 3. Move 4.”; “Mutated city 2. Populations now 1, 1, 0. Move 1.”; in the linked view one sentence for both boards, “Mutated city 1 and fired node 1. Populations now 0, 1; numbers now minus 1, 2. Move 2.”; “Node 3 cannot be fired: its number is minus 2.”; “Game over after 6 moves.”; “Played 14 moves to the end. Numbers now 0, 0, 0, 0, minus 1.”; “Undid move 4. Numbers now …”. On boards with more than four nodes only the changed nodes are read: “Fired node 4. Node 3 now 3, node 4 now minus 2, node 5 now 1. Move 7.” When moves follow each other within half a second (play to the end, scrubbing) only the last state is announced, with the number of moves made.	4.1.3
A-5	Sign and state are conveyed by numeral, fill and shape as well as colour; coloured edges also carry a text label.	1.4.1
A-6	Text and numerals at least 4.5 to 1; outlines, roads, arrows and focus rings at least 3 to 1, in light and dark themes. Section 6.3 fixes colours that meet this and gives the computed ratios.	1.4.3, 1.4.11

Id	Requirement	Criterion
A-7	Targets are 44 by 44 CSS pixels where the layout allows (beyond AA, by choice) and never less than 24 by 24 with no overlap (the AA requirement), by a transparent hit area when the drawn node is smaller. The rule is in section 6.1: the hit area of a node is a square of side $\min(44, u)$ pixels, u being the grid unit. In a box 320 pixels wide this gives 44 for boards with at most six nodes in a row, 43 for seven (E_8), 38 for eight, 33 for nine (the affine board of C_8) and 25 for twelve (the largest custom board). Polygon layouts always reach 44.	2.5.8 (AA); 2.5.5 (AAA)
A-8	Nothing requires dragging or a multi-finger gesture: rotation, zoom, node placement and scrubbing all have buttons and keys.	2.5.7, 2.5.1
A-9	No motion starts by itself. Transitions are short and are replaced by instant updates under `prefers-reduced-motion` (beyond AA, by choice). Any continuous rotation has a pause control.	2.2.2 (A); 2.3.3 (AAA)
A-10	Every figure has a text alternative, and every tool has a "show as table" view that presents the same state as an HTML table with a button per node.	1.1.1
A-11	Prose and controls reflow at 320 CSS pixels. A diagram that cannot (a wide orbit graph, the Weyl Explorer) scrolls inside its own box, which is the exception the criterion allows for two-dimensional content.	1.4.10
A-12	Mathematics is rendered with MathML available to assistive technology (KaTeX's default output).	1.3.1

Id	Requirement	Criterion
A-13	Each release is checked by hand with one screen reader on desktop and one on a phone, following a written script that grows with the site. For release 0.5: play a game on A_2, grow the root garden of A_3, classify a board in T4. For release 1.0, in addition: read the Atlas page of B_3; decorate a Satake diagram.	

7.6 Performance budget [D]

A lesson page is readable with JavaScript disabled. With it enabled, the first tool of a page hydrates when the browser is idle (`client:idle`) and the others when scrolled into view (`client:visible`). Islands share code: the Svelte runtime, the core of the engine and the board components are fetched once per page. The budget is therefore stated per route, as the gzip size of **all JavaScript the route fetches before the reader's first interaction**:

Route	Limit	What it consists of
a page with no tool (`about`)	15 kB	the shell: navigation, theme switch, the highlight store
a page with one Game Board (the phase 0 page)	60 kB	the shell and the shared code (Svelte runtime, engine core, board components: 45 kB)
each tool alone, on `/tools/<id>`	90 kB	shell, shared code, and the tool's own code with the engine functions only it uses (30 kB)
any lesson page	120 kB	shell, shared code, the first tool of the page, and any other tool in view when the page opens at 360 by 800 pixels

- **Measured by `npm run size`**: a Playwright script opens each route of the built site at 360 by 800 pixels, waits until the network is idle without touching the page, collects the URL of every script the browser fetched, compresses those files from `site/dist` with Node's `zlib` (gzip, level 9), and fails when a sum exceeds its limit. It prints the table of sums. It needs no service and no dependency beyond Playwright.
- No typesetting library, no layout engine and no polyfill bundle is sent to the client. The worker's script (E-9) is fetched when a tool first needs it. No tool may need it for its default state, so it never counts toward these limits.
- **Lab timings**, on the built site with a mobile profile (360 by 800 pixels, processor slowed four times): Largest Contentful Paint at most 2.5 s, Total Blocking Time at most 200 ms, Cumulative Layout Shift at most 0.1. Tools reserve their box size in the HTML so that hydration does not shift the page. The intended tool is Lighthouse CI, whose current version and behaviour under Node 22 were not checked for this specification [U]. If it cannot be made to run within an hour, take the same three numbers in Playwright on Chromium (a `PerformanceObserver` for `largest-contentful-paint`, `layout-shift` and `longtask` entries, the processor slowed through a DevTools Protocol session) and record the choice in `DECISIONS.md`.

- Interaction to Next Paint, which only field data can measure, is checked by hand at each release on one mid-range phone, with a target of 200 ms (owner's part of the gates, section 9.1).
- **Main-thread work.** No engine call on the main thread may become a long task (50 ms) on that phone. The rule of E-9 is sized for this: its worst case, 2000 positions on a board with 12 nodes in `Big` arithmetic, took 9 ms in the measurement of section 5.5. A Vitest benchmark repeats that case (the 12-cycle with one chord, joining nodes 1 and 7, as in `reference/bench_orbit.mjs`) and fails above 25 ms (median of seven runs). The E_8 root set, its Coxeter projection and 6720 edges draw in under 100 ms on the phone; a browser test records the drawing time on the CI runner and fails above 100 ms there.

7.7 Testing

Level	What	Gate
Engine, golden	every key of `golden_data.json` assigned to the phases built so far (section 5.7)	100 percent pass; required for every merge
Engine, properties	the thirteen properties of section 5.7, as far as their functions exist	required for every merge
Components	each tool's acceptance criteria of section 4. A criterion that ranges over every board, every record or many random cases is a Vitest test of the engine functions the tool calls; the named examples, and one case for each kind of board the tool offers, are Playwright tests against the built site, reading the hooks of section 4.0	required from the phase in which the tool ships
Without JavaScript	the run described in section 7.4	required for every merge that touches a lesson or a tool
Accessibility	axe scan of every page and tool route; keyboard-only scripts for T1, T4, T13	no serious or critical violations
Content	the build fails on an unknown `<Cite>` key or an unresolved `<G>` or `<Tex>` path; a release build prints the report of open `<Unverified>` items; `scripts/numbers-in-prose.mjs` lists every number in lesson prose that is not written with `<G>`, for a person to review	the build failures: every merge; the two reports: read by the owner before a release
Performance	`npm run size` against the table of section 7.6; the lab timings of section 7.6	sizes: every merge; lab timings: gates G3, G4 and G6
Links	internal links checked by the build; external links checked weekly by `links.yml`, where a failure opens an issue and does not fail a build	

Level	What	Gate
Oracle	`npm run verify` (section 5.7): the oracle reports zero failures and reproduces the committed golden file byte for byte	required whenever `reference/` changes

Commands, from the repository root: `npm ci`; `npm test` (Vitest: the engine, and the unit tests of `site/tests/unit`); `npm run build` (Astro, into `site/dist`; a preview build); `RELEASE=1 npm run build` (a release build); `npm run e2e` (Playwright against `astro preview`, including the run without JavaScript and the axe scans); `npm run size` and `npm run lab` (the sizes and the lab timings of section 7.6); `npm run report` (the two content reports of the table above, written to `reports/`); `npm run verify` (the Python oracle; needs `python3`, which the CI image has).

7.8 Deployment and durability

- `ci.yml` runs on every pull request and on every push to the main branch: the tests of section 7.7, then a preview build, whose `site/dist` it uploads as an artifact.
- `deploy.yml` runs when a version tag (`v*`) is pushed and on manual dispatch. It makes a release build and publishes it to GitHub Pages. The job runs only when the repository variable `DEPLOY_PAGES` is `true`, which the owner sets after enabling Pages; until then it is skipped and nothing fails. The public site therefore changes only at a release (section 9.1); between releases the state of the main branch is seen through the CI artifact (decision D18).
- `links.yml` runs weekly (section 7.7).
- Each release attaches two archives of the built site to the GitHub release: one built for the Pages address, and one built with `BASE_PATH=/`, which can be served as plain files from the root of any domain with no build step. Serving it from a subdirectory needs one rebuild with another `BASE_PATH`. This is the protection against the fate of the Java applets in Appendix F.
- No third-party script at runtime. Fonts and KaTeX assets are self-hosted. If visit counts are wanted, use a cookie-free counter and say so on the site.

7.9 Licensing and credit [D; open decision in section 10]

- Proposed: text and figures under CC BY-SA 4.0 (or CC BY 4.0 for the widest reuse); code under MIT (or Apache-2.0). State both in the footer and in the repository, with SPDX identifiers, and say that code samples shown in the text fall under the code licence.
- Ideas, rules, theorems and data from the literature are restated in the site's own words with citations. No sentence, figure or table layout is copied from a paper, preprint or lecture, beyond short attributed quotations. Videos are linked or embedded with the platform's own player and never re-hosted.
- Reference software under GPL or LGPL (SageMath, CoReLG, Atlas, Group Explorer) is used only as a black-box oracle for test data; its source is not translated. The MIT-licensed `mutation-game` package may be read.
- A credits page lists every third-party asset with title, author, source and licence, and the licences of bundled libraries and fonts.
- The site should acknowledge N. J. Wildberger's story and terminology prominently. A courtesy message to him before launch is recommended (section 10).

8. Design direction

8.1 Twelve principles

These come from the literature on explorable explanations and from measured reader behaviour (`SOURCES.md` 11.6). They are requirements: the definition of done in section 9.2 names them for every tool and every lesson,

and the owner's part of each gate includes reading the new pages against them.

	Principle	Consequence here
P1	The page works with no interaction at all.	Every lesson reads as an illustrated article. Each tool has a meaningful starting state and a caption that states its point.
P2	Nothing the argument needs sits behind a click, a hover or a tab.	Results are stated in the prose. Tooltips repeat information; they never hold it alone.
P3	Open with a question and a concrete case the reader can touch, then generalise.	Each module starts on A_2 or A_3 and only later says "for any board".
P4	One new mechanic at a time.	M0 shows both views once, as a preview. M1 to M3 then use one game and one view; the second view returns in M4; multiple roads arrive in M6.
P5	Ask for a prediction before a reveal.	The `prediction` configuration of section 4.0. At least two predictions in each of M0 to M9 (section 9.2).
P6	Guided goals with immediate feedback first; the open sandbox last.	Challenges with a stated goal ("reach the highest root", "end the game in the fewest moves"). The sandbox link closes each module.
P7	Show a worked play-through, then fade the support.	Each new game is first shown as a stepped example with commentary, then as a half-finished game to complete, then free. Readers may skip.
P8	Several linked views of one object, with identical colours and names everywhere.	Populations and Numbers have fixed colours and icons in prose, figures and formulas. Node i is always called "node i" or "city i", never both in one sentence.
P9	The reader controls time.	Steppers and scrubbers. No autoplay. The whole history stays visible.
P10	Direct manipulation with obvious affordances and forgiving targets.	Click the node to act on it. Hit areas larger than the drawn node. Every drag has a button equivalent.
P11	Phones are a main audience.	Layouts are designed at 360 pixels first and must work at 320. At 360 pixels E_8 fits seven nodes across with 44-pixel targets; section 6.1 gives the rule for wider boards and narrower screens.

	Principle	Consequence here
P12	Playtest, and plan for a long life.	The script of section 9.4 before each release; pinned versions; archived builds.

Pitfalls to avoid, each observed in the prior art of Appendix F: a sandbox with no guidance; a form-and-table research interface with no narrative; controls for everything at once; reliance on a plug-in or a server; pictures that cannot be exported.

8.2 Anatomy of a lesson page

1. Title and the module's question.
2. A first figure, already showing something, with a one-sentence caption.
3. Beats: a short paragraph, then a small tool or figure beside or below it. One idea per beat.
4. "What we have seen": the general statement in a framed box, with its provenance (a citation key, or "verified by computation").
5. "For readers who know Lie theory": the standard names, in one paragraph.
6. Challenges: three to five goals, each with a check that reports success.
7. Links: the sandbox with this module's board loaded; the Atlas page; the next module. (A link whose target has not been released yet is left out by the build; section 7.3.)
8. Sources for the module.

A lesson is between 800 and 1800 words. Tools take the full width of the screen on phones (section 6.1) and sit in a wide column on desktop; text lines stay near 66 characters.

8.3 Visual language [D]

- Calm, bright, low-chrome: the boards are the content. One typeface for prose, one monospaced face with tabular figures for numbers on nodes and in tables, and KaTeX's fonts for mathematics.
- Two accent colours for the two views, taken from the same colour-blind-safe palette as the sign colours of section 6.3 and different from them: bluish green `#009E73` for Populations and reddish purple `#CC79A7` for Numbers (3.4 and 3.1 to 1 on white, 5.5 and 6.1 to 1 on `#121212`, computed with the WCAG formula). They mark the borders and icons of panels and the underlines of the two words, never text and never the fill of a node, so that an accent cannot be read as a sign. Coloured edges in orbit graphs take one colour per node label from the palette of eight.
- Light and dark themes from the start, driven by CSS custom properties.
- Icons: a small city for Populations, a small numbered disc for Numbers, used wherever the view is named.
- Figures exported from the site carry the board name, the state and the site address in a footer line that can be switched off.

8.4 Writing rules

- Plain declarative sentences. Define every term on first use and link later uses to a glossary entry (Appendix C is its seed).
- Use the site's words consistently: board, node (city in the Mars story), road, population, numbers, mutate, fire, legal, free play. Give the standard term once, in the Lie theory paragraph.
- State the convention whenever a result is quoted from a source that uses another one (section 2.9).
- Every theorem-like statement carries its provenance. "Verified by computation for all boards of rank at most 8" is an acceptable provenance and must not be dressed up as a citation. A statement that rests on an open item of Appendix E is wrapped in `<Unverified>` (section 7.3).
- Numbers that have a golden value are written with `<G>`, never typed.

- Credit accurately: the mutation game is Wildberger's; the numbers game is Mozes', with Proctor's earlier use of the same process and Eriksson's development acknowledged; Kostant's game is "attributed to Kostant".
- House style, taken from the project owner's stated preferences for prose and to be confirmed for the site (section 10): no em-dashes; no stock rhetorical contrasts or fashionable filler words; the specific in place of the grand.
- No sentence of the lessons is copied from a source.

8.5 Patterns for predictions and challenges

Pattern	Example
Predict a number	"City 2 holds 1 and its neighbours hold 1 and 0. What will it hold after it mutates?"
Predict a yes or no	"Will a different order of firing change how long the game lasts?"
Reach a target	"Reach the population $(1, 2, 1, 1)$ on D_4."
Reach by raising	"Reach the highest root of E_6 using only moves that raise a city." (Every such play takes exactly 10 moves, which the reader discovers.)
Find a counterexample	"Find a board and a play that puts Martians and anti-Martians on the board together." (Impossible with two-way roads.)
Classify	"Paint a decoration of A_3 that all three lights accept."
Explain	A free-text box that is not graded, followed by the site's own explanation.

9. Build plan

9.1 Phases and gates

Each phase ends with a gate in two parts. The **automatic part** is a set of tests, run by the commands of section 7.7: in continuous integration once the repository is on GitHub, and on the builder's machine before that. Claude Code may start the next phase as soon as it passes. The **owner's part** is a short list of things only a person can do. It never blocks the next phase. It is due before the next public release: the one named in the last column of its row, or, where the row says "none", the first release after it. Its outcome is recorded in `DECISIONS.md`. Sizes are relative (S, M, L); no dates are implied. The engine functions of each phase are those marked with its number in section 5, and the golden tests of each phase are those of the table in section 5.7.

Phase	Builds	Gate: automatic part	Gate: owner's part	Release
0 Foundations (S)	Repository as in section 7.3; the three workflows of section 7.8; engine skeleton (phase 0 functions) with the `Arith` interface and its `SafeInt` and `Big` instances; golden-test harness; the mathematics-rendering spike of section 7.2 item 3; one page with one board on A_2, rendered at build time and hydrated; the four content components; the commands `npm run size` and `npm run verify`.	G0: phase 0 golden tests pass for all 32 types; the page shows mathematics and the board with JavaScript disabled; the keyboard script passes on the board; the first two limits of section 7.6 hold; a preview build and a release build both succeed; the rendering route is recorded in `DECISIONS.md`.	Put the repository on GitHub and enable Pages, then run the deploy workflow once by hand (until then the build uses the defaults of section 7.2 item 10 and the deploy job is skipped); answers, or acceptance of the defaults, for D1 (name), D3 (framework), D4 (licences) and D17 (who writes lessons).	none
1 The two moves (M)	Engine phase 1. Tools T1, T2, T9. Lessons M0, M1. URL state and history. The worker of E-9. "Copy link" and JSON export. The glossary, sources and open-items pages.	G1: phase 0 and 1 golden tests; acceptance of T1, T2, T9; the run without JavaScript; axe scan; the size limits of section 7.6.	A first reading of M0 and M1 by two readers of the primary audience.	none
2 Form and classification (M)	Engine phase 2, with `classify` and the guards of E-11. Tools T3, T4, T15, T16. Lessons M2, M3. SVG and PNG export. The open-problem page.	G2: golden tests through phase 2; acceptance of T3, T4, T15, T16, and the parts of the criteria of T2 and T9 that section 4 assigns to this gate.	Appendix E item 6 (wording of the fate table).	none
3 Duality and the Weyl group (L)	Engine phase 3. Tools T5, T6, T7, T8. Lessons M4, M5.	G3: golden tests through phase 3; tool acceptance; sizes and lab timings of section 7.6 on the core path M0 to M5; a release build succeeds and prints its two reports (section 7.7).	The playtest of section 9.4 with five readers; accessibility script A-13; Appendix E items 2 and 3; reading of M0 to M5 against principles P1 to P12, and approval of their text (D17); the two content reports read; Interaction to Next Paint on one phone; decision D6 carried out.	**0.5, the core path**
4 Multiple edges and geometry (L)	Engine phase 4; the projector. Tools T10, T11. Lessons M6, M7.	G4: golden tests through phase 4; tool acceptance; the drawing-time test of section 7.6 for the E_8 picture; lab timings on M6 and M7.	The E_8 picture tried on one mid-range phone; Appendix E items 7 and 9.	none
5 Representations (M)	Engine phase 5. Tool T12. Lesson M8, with the page on what mutation is not.	G5: golden tests through phase 5; every relation checks in the tool.	none	none

Phase	Builds	Gate: automatic part	Gate: owner's part	Release
6 Real forms, atlas, release (L)	Engine phase 6. Tools T13, T14. Lessons M9, M10. LaTeX export, embed routes, history, confusions, credits, licences.	G6: all golden tests of phases 0 to 6; full accessibility scan; sizes and lab timings of section 7.6 on every page; a release build succeeds, and its report of open items is attached to the pull request (or left in `reports/` where there is none).	**Every item of Appendix E not settled at an earlier gate, items 1, 4, 5 and 8 first**: each is either closed in `content/unverified.json` or left open with its fallback text read and accepted; accessibility script A-13 on the whole site; Interaction to Next Paint on one phone; one exported LaTeX file of each kind compiled (section 6.7); approval of the text of M6 to M10.	**1.0**, with the built site archived
7 Extensions (each S or M, any order)	Golden boards and golden folding; representations with one zero weight; generalized Satake diagrams; Vogan diagrams side by side; web components; further languages.	Per item: its golden tests (section 5.7, phase 7) and tool acceptance.	Appendix E item 9 for the golden folding.	1.x

9.2 Definition of done

For a tool: its acceptance criteria are automated and pass; A-1 to A-12 hold; it obeys every rule of section 4.0 (default state with a caption, rendering without JavaScript, link, undo, reset, export as far as the current phase provides it, test hooks, stated limits); it stays within its size budget; it has a "show as table" view. Checked by the owner at the gate, and so no condition for merging: principles P1, P2, P9 and P10 of section 8.1 can be pointed to in it.

For a lesson: it follows the anatomy of section 8.2; it reads completely without JavaScript; every statement has a provenance, with `<Unverified>` around anything that rests on an open item; every number that has a golden value is written with `<G>`; for M0 to M9, it has at least two predictions and three challenges, which the drafter adds from the patterns of section 8.5 wherever section 3 names fewer (M10 is reference pages and has none); it follows the writing rules of section 8.4. Checked by the owner at the gate: principles P3 to P8 can be pointed to in it; it has been read aloud once for flow. Until then the page carries the draft mark of decision D17.

9.3 Working agreements for Claude Code

- Port the engine function by function from `reference/lie_games.py`, writing the golden test first. Keep the oracle's choices (E-5): lowest position first.
- Never change a convention of section 2, a label, or the sign of the legal move. If something seems wrong, add a failing check to `reference/verify.py` that demonstrates it and stop.
- Do not edit `golden_data.json` by hand. Regenerate it.
- When an open decision of section 10 blocks work, take the default, and add a line to `DECISIONS.md` with the date and the reason.
- Keep changes small: one tool or one lesson per pull request, with its tests. Claude Code merges its own pull request when every required check of section 7.7 is green; nothing waits for a review by the owner except a release.
- The engine has no dependencies. For the site, any added dependency, whether shipped to the reader or used only by the build, is recorded in `DECISIONS.md` with its version and licence, and with its size

if it is shipped.
- Do not present anything labelled [U] as fact in a lesson; wrap it in `<Unverified>` with a fallback. Never set an item of `content/unverified.json` to `closed`; that is the owner's act.
- For automated checks use `astro build` and `astro preview`, not `astro dev` (section 7.2 item 9).
- Lessons are drafted from sections 2 and 3 in the site's own words (decision D17). A draft is marked as a draft on the page until the owner has read it.

9.4 Playtest script (owner's part of gate G3, and each later release)

Five readers who know linear algebra and no Lie theory, observed one at a time, thinking aloud, for about 40 minutes on M0 to M5. No help unless they are stuck for two minutes. Record where they stop reading, which tools they skip, and every wrong prediction. Afterwards, without the site:

1. "Here is a board with three cities in a row holding 2, 1 and 0. City 2 mutates. What does it hold?" (Expected: 1.)
2. "The same three numbers are now on the numbers game. Fire node 1. What are the three numbers?" (Expected: -2, 3, 0.)
3. "On the path with three cities the population is $(1, 1, 0)$. What numbers does the other view show?" (Expected: $(1, 1, -1)$.)
4. "Why does a complete game from $(1, 1, 1)$ on that board always take six moves?" (Looking for: each move uses up one of the six positive roots, or an equivalent idea in their own words.)
5. "What is the Cartan matrix of this board, and what is it for?" (Looking for: twos on the diagonal and minus the road counts off it; it converts one view into the other.)

Pass: four of five readers answer 1 to 3 correctly and give an acceptable answer to 4 or 5. Any tool that three readers skip is redesigned or moved.

9.5 Risks

Risk	Mitigation
A fact from an unread source is wrong (Mozes' text, Helgason's multiplicities, Kac's numbering)	Appendix E; such items are worded as "computed" or wrapped in `<Unverified>` until checked
Toolchain churn (several major releases among the chosen tools in 2026)	pinned versions, lockfile, the phase 0 spike, thin framework layer over a plain TypeScript engine
Scope: sixteen tools and eleven lessons	the core path (phases 0 to 3) is a complete, publishable site by itself; later phases are independent
E_8 on phones	caps, worker, canvas drawing, fewer edges while turning, vertices-only fallback
Screen-reader behaviour of SVG	real buttons over the drawing, table view, manual testing at each release
Convention drift during the build	golden tests on every merge; the conventions block is repeated in `CLAUDE.md`
Confusion between B and C when readers compare with Wildberger	section 2.10 surfaced in M6 and in the Atlas
A claim of novelty that turns out to be wrong	the site makes no such claim; see Appendix E item 8
The owner's checks lag behind the build	they are due at releases and never hold up a phase; an unchecked statement stays wrapped, marked in previews and replaced by its fallback wording in a release

10. Open decisions for the project owner

Each row gives the default that Claude Code will use if no answer has been recorded.

#	Decision	Options	Default
D1	Name and address of the site	any	working title “Two Games on a Dynkin Diagram”; GitHub Pages address
D2	Level of the primary reader	first-year linear algebra; or readers who already know Lie theory	linear algebra, with the “for readers who know Lie theory” paragraphs
D3	Framework	Astro with Svelte islands; SvelteKit; Astro with React	Astro with Svelte (section 7.1)
D4	Licences	CC BY-SA 4.0 or CC BY 4.0 for content; MIT or Apache-2.0 for code	CC BY-SA 4.0 and MIT
D5	Use of the Mars and Venus story	retell with credit; or neutral wording only	retell with credit
D6	Contact with N. J. Wildberger before launch	a courtesy message describing the site and asking about the lecture links and about his preferred citation of the manuscript; or none	send the message before the release of phase 3; the build does not wait for an answer
D7	Sign of the linked numbers	$a = Cp$ (Dynkin labels; “the drop”); or Wildberger’s $q = -Cp$ (“the gain”)	$a = Cp$
D8	A reader-facing switch to the fire-negative convention of Mozes and Eriksson	switch in T1 and T5; or explanation only	explanation only, except in T15 where the pentagon appears in its original form
D9	Order of the phase 7 extensions	golden boards; one zero weight; generalized Satake; Vogan; web components	golden boards first
D10	Relation to the project’s research-software specification “MutationGraph”	share `golden_data.json` as fixtures and state the common mutation rule; or keep the projects separate	keep separate; note in both that on simple graphs the rule and $C = 2I - A$ agree
D11	Who carries out the human checks of Appendix E, and with which library access	the owner; a collaborator	the owner
D12	Languages	English; others later	English
D13	Visit statistics	none; a cookie-free counter	none

#	Decision	Options	Default
D14	House writing style for lessons (section 8.4)	as proposed; amended	as proposed
D15	An "open problems" page inviting readers' attempts at the Mutation Conjecture, with a contact address	yes; no	yes, without a contact form (a link to the repository's issue tracker)
D16	Whether to state the three corrections of Appendix D publicly (for example in a short "common confusions" page)	yes; no	yes, as general confusions with no reference to the note
D17	Who writes the lessons	the owner; Claude Code from sections 2 and 3; jointly	Claude Code drafts each lesson from sections 2 and 3 and the figure tables, marked as a draft on the page; the owner edits and approves before the release that contains it
D18	Preview deployments for pull requests	none beyond the CI artifact; Cloudflare Pages previews	none beyond the CI artifact
D19	The $B_2 = C_2$ naming on the site (section 2.8)	show "$B_2 = C_2$"; show the name of the board the reader chose	the name of the board the reader chose in pickers, and "$B_2 = C_2$" wherever a board is recognised from its shape

Nothing here blocks the start. Phase 0 can be built and its automatic gate passed in a local repository with every default above. Until the repository is on GitHub and named, `site` and `base` come from the environment (section 7.2 item 10) and the deploy job is skipped. What the owner supplies before the first release: the repository and with it the Pages address; answers to D1, D3, D4 and D17, or a word that the defaults stand.

One thing this specification deliberately does not decide: the visual identity beyond section 8.3.

Appendix A. Worked examples

Everything in this appendix was generated by the oracle and can be used directly as test vectors.

A.1 The three rank 2 boards

A_2. Cartan matrix $C = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$, roads: 1 from 1 to 2, 1 from 2 to 1.

Numbers game from (1, 1), always firing a positive node:

step	position before	node fired	number fired	position after
1	(1, 1)	1	1	(-1, 2)
2	(-1, 2)	2	2	(1, -2)
3	(1, -2)	1	1	(-1, -1)

Mutation game: the positive root populations, grouped by height: (0, 1), (1, 0); (1, 1).

Translation check: the highest root population $p = (1, 1)$ has numbers $a = Cp = (1, 1)$. Mutating city 1 gives $(0, 1)$, whose numbers are $(-1, 2)$, the same as firing node 1 on $(1, 1)$. Mutating city 2 gives $(1, 0)$, whose numbers are $(2, -1)$, the same as firing node 2 on $(1, 1)$.

B_2. Cartan matrix $C = \begin{pmatrix} 2 & -1 \\ -2 & 2 \end{pmatrix}$, roads: 2 from 1 to 2, 1 from 2 to 1.

Numbers game from (1, 1), always firing a positive node:

step	position before	node fired	number fired	position after
1	(1, 1)	1	1	(-1, 3)
2	(-1, 3)	2	3	(2, -3)
3	(2, -3)	1	2	(-2, 1)
4	(-2, 1)	2	1	(-1, -1)

Mutation game: the positive root populations, grouped by height: (0, 1), (1, 0); (1, 1); (1, 2).

Translation check: the highest root population $p = (1, 2)$ has numbers $a = Cp = (0, 2)$. Mutating city 2 gives $(1, 0)$, whose numbers are $(2, -2)$, the same as firing node 2 on $(0, 2)$.

G_2. Cartan matrix $C = \begin{pmatrix} 2 & -3 \\ -1 & 2 \end{pmatrix}$, roads: 1 from 1 to 2, 3 from 2 to 1.

Numbers game from (1, 1), always firing a positive node:

step	position before	node fired	number fired	position after
1	(1, 1)	1	1	(-1, 2)
2	(-1, 2)	2	2	(5, -2)
3	(5, -2)	1	5	(-5, 3)
4	(-5, 3)	2	3	(4, -3)
5	(4, -3)	1	4	(-4, 1)
6	(-4, 1)	2	1	(-1, -1)

Mutation game: the positive root populations, grouped by height: (0, 1), (1, 0); (1, 1); (2, 1); (3, 1); (3, 2).

Translation check: the highest root population $p = (3, 2)$ has numbers $a = Cp = (0, 1)$. Mutating city 2 gives $(3, 1)$, whose numbers are $(3, -1)$, the same as firing node 2 on $(0, 1)$.

The numbers fired on B_2 (1, 3, 2, 1) and on G_2 (1, 2, 5, 3, 4, 1) are the heights of the positive coroots. Sorted, they are the same lists as the heights of the positive roots, because a board and its reverse have the same exponents. The difference between roots and coroots shows in the symbolic game: on B_2 from (x_1, x_2) the same four moves fire x_1, $2x_1 + x_2$, $x_1 + x_2$, x_2, which are the positive root populations $(1, 0)$, $(2, 1)$, $(1, 1)$, $(0, 1)$ of the reversed board C_2; the board B_2 itself has $(1, 2)$ where this list has $(2, 1)$ (F6).

A.2 The generic game on A_3

step	before	fire	number fired	after
1	$(1,1,1)$	1	1	$(-1,2,1)$
2	$(-1,2,1)$	2	2	$(1,-2,3)$
3	$(1,-2,3)$	1	1	$(-1,-1,3)$
4	$(-1,-1,3)$	3	3	$(-1,2,-3)$
5	$(-1,2,-3)$	2	2	$(1,-2,-1)$
6	$(1,-2,-1)$	1	1	$(-1,-1,-1)$

Tally: the value 1 is fired 3 times, the value 2 is fired 2 times, the value 3 is fired 1 times. Read sideways, the tally $(3,2,1)$ has conjugate partition $(3,2,1)$, read as exponents $1,2,3$, and $(1+1)(2+1)(3+1)=24$ is the order of the group. The wave: fire nodes 1, 3: $(-1,3,-1)$; fire nodes 2: $(2,-3,2)$; fire nodes 1, 3: $(-2,1,-2)$; fire nodes 2: $(-1,-1,-1)$. 4 rounds. The Coxeter number of A_3 is 4.

A.3 Weyl's dimension formula as two games

- A_2, $\lambda=(1,1)$: word 1, 2, 1; from ρ the numbers fired are 1, 2, 1 (product 2); from $\lambda+\rho=(2,2)$ along the same word they are 2, 4, 2 (product 16); quotient 8. This is the 8-dimensional adjoint representation.
- G_2, $\lambda=(1,0)$: word 1, 2, 1, 2, 1, 2; from ρ the numbers fired are 1, 2, 5, 3, 4, 1 (product 120); from $\lambda+\rho=(2,1)$ along the same word they are 2, 3, 7, 4, 5, 1 (product 840); quotient 7.
- B_2, $\lambda=(0,1)$: word 1, 2, 1, 2; from ρ the numbers fired are 1, 3, 2, 1 (product 6); from $\lambda+\rho=(1,2)$ along the same word they are 1, 4, 3, 2 (product 24); quotient 4. This is the spin representation.

A.4 Kostant's game

- A_3 from city 1: $(1,0,0)\to$ (city 2) $(1,1,0)\to$ (city 3) $(1,1,1)$; 2 moves; final numbers $(1,0,1)$.
- D_4 from city 1: $(1,0,0,0)\to$ (city 2) $(1,1,0,0)\to$ (city 3) $(1,1,1,0)\to$ (city 4) $(1,1,1,1)\to$ (city 2) $(1,2,1,1)$; 4 moves; final numbers $(0,1,0,0)$. The only positive final number is at city 2, which is where the extra city of the affine board $\tilde{D}_4$ attaches.
- B_3 from the long city 1: $(1,0,0)\to$ (city 2) $(1,1,0)\to$ (city 3) $(1,1,2)\to$ (city 2) $(1,2,2)$; 3 moves; final numbers $(0,1,0)$ (the highest root).
- B_3 from the short city 3: $(0,0,1)\to$ (city 2) $(0,1,1)\to$ (city 1) $(1,1,1)$; 2 moves; final numbers $(1,0,0)$ (the highest short root).

Where the game ends on the boards with two root lengths, and which affine board the recipe of section 2.12 then builds:

Board	start at a long city: end, new board	start at a short city: end, new board
B_2	(1, 2), $\tilde{C}_2$	(1, 1), $D_3^{(2)}$
B_3	(1, 2, 2), $\tilde{B}_3$	(1, 1, 1), $D_4^{(2)}$
C_3	(2, 2, 1), $\tilde{C}_3$	(1, 2, 1), $A_5^{(2)}$
F_4	(2, 3, 4, 2), $\tilde{F}_4$	(1, 2, 3, 2), $E_6^{(2)}$
G_2	(3, 2), $\tilde{G}_2$	(2, 1), $D_4^{(3)}$

A.5 Three small Satake diagrams

$\mathfrak{su}(1,2)$: board A_2, no black nodes, τ exchanges the two nodes, $\theta(p_1,p_2)=-(p_2,p_1)$.

shadow $\bar{\alpha}$	number of positive roots above it	positive roots α with $\theta\alpha$
$(1/2, 1/2)$	2	$(0,1) \mapsto (-1,0)$; $(1,0) \mapsto (0,-1)$
$(1,1)$	1	$(1,1) \mapsto (-1,-1)$

One restricted root λ of multiplicity 2 and its double of multiplicity 1: type BC_1, real rank 1, $\dim \mathfrak{k} = 4$.

$\mathfrak{su}^*(4)$: board A_3, black nodes 1 and 3, τ the identity; the word of w_X is 1, 3.

shadow $\bar{\alpha}$	number of positive roots above it	positive roots α with $\theta\alpha$
0	2	$(0,0,1) \mapsto (0,0,1)$; $(1,0,0) \mapsto (1,0,0)$
$(1/2, 1, 1/2)$	4	$(0,1,0) \mapsto (-1,-1,-1)$; $(0,1,1) \mapsto (-1,-1,0)$; $(1,1,0) \mapsto (0,-1,-1)$; $(1,1,1) \mapsto (0,-1,0)$

One restricted root of multiplicity 4: type A_1, real rank 1, $\dim \mathfrak{k} = 10$.

$\mathfrak{su}(1,3)$: board A_3, black node 2, τ the flip; the word of w_X is 2.

shadow $\bar{\alpha}$	number of positive roots above it	positive roots α with $\theta\alpha$
0	1	$(0,1,0) \mapsto (0,1,0)$
$(1/2, 1/2, 1/2)$	4	$(0,0,1) \mapsto (-1,-1,0)$; $(0,1,1) \mapsto (-1,0,0)$; $(1,0,0) \mapsto (0,-1,-1)$; $(1,1,0) \mapsto (0,0,-1)$
$(1,1,1)$	1	$(1,1,1) \mapsto (-1,-1,-1)$

Type BC_1 with multiplicities 4 and 1, real rank 1, $\dim \mathfrak{k} = 9$.

A.6 The pentagon

Board: the 5-cycle, nodes 1 to 5 in order round the cycle. The Olympiad problem prescribes no numbers; the site's example start is $(-1, 2, -3, 4, -1)$ in the Olympiad's convention, which is $(1, -2, 3, -4, 1)$ in the convention of this specification, of level -1. With the lowest-node rule the nodes fired are 1, 3, 2, 1, 5, 1, 2, 4, 3, 2, 5, 1, 4, 5, the numbers fired are 1, 3, 2, 1, 3, 2, 1, 2, 2, 1, 1, 1, 1, 1, and the end is $(0, 0, 0, 0, -1)$ after 14 moves. Every other order of play also takes 14 moves, fires the same numbers in another order, and reaches the same end [V].

Appendix B. Satake tables for comparison with Helgason

These tables are laid out to be compared, line by line, with the table on pp. 532-534 of [Hel01] (Appendix E, item 1). They are printed from the golden data. Multiplicities are given at each simple restricted root as $(m_\lambda, m_{2\lambda})$, which is how that table is reported to present them. "τ" is the full symmetry, including its action on black nodes; "arrows" are what a printed diagram shows. The column "Restricted roots" is the field `restricted_type_display` (section 6.4), so it writes C_2 in the families whose general member is C_p.

B.1 Exceptional boards [V]

Board	Label	Real form	Black nodes	τ	Arrows	Real rank	Restricted roots	Simple restricted roots: orbit $(m_\lambda, m_{2\lambda})$	$\dim \mathfrak{k}$	Character
E_6	E I	`e6(6)`	none	identity	none	6	E6	every node (1, 0)	36	6
E_6	E II	`e6(2)`	none	$1 \leftrightarrow 6$, $3 \leftrightarrow 5$	$1 \leftrightarrow 6$, $3 \leftrightarrow 5$	4	F4	{1,6} (2, 0); {2} (1, 0); {3,5} (2, 0); {4} (1, 0)	38	2
E_6	E III	`e6(-14)`	3, 4, 5	$1 \leftrightarrow 6$, $3 \leftrightarrow 5$	$1 \leftrightarrow 6$	2	BC2	{1,6} (8, 1); {2} (6, 0)	46	-14
E_6	E IV	`e6(-26)`	2, 3, 4, 5	identity	none	2	A2	{1} (8, 0); {6} (8, 0)	52	-26
E_6	compact	`e6(-78)`	all	$1 \leftrightarrow 6$, $3 \leftrightarrow 5$	none	0	none	none	78	-78
E_7	E V	`e7(7)`	none	identity	none	7	E7	every node (1, 0)	63	7
E_7	E VI	`e7(-5)`	2, 5, 7	identity	none	4	F4	{1} (1, 0); {3} (1, 0); {4} (4, 0); {6} (4, 0)	69	-5
E_7	E VII	`e7(-25)`	2, 3, 4, 5	identity	none	3	C3	{1} (8, 0); {6} (8, 0); {7} (1, 0)	79	-25
E_7	compact	`e7(-133)`	all	identity	none	0	none	none	133	-133
E_8	E VIII	`e8(8)`	none	identity	none	8	E8	every node (1, 0)	120	8
E_8	E IX	`e8(-24)`	2, 3, 4, 5	identity	none	4	F4	{1} (8, 0); {6} (8, 0); {7} (1, 0); {8} (1, 0)	136	-24
E_8	compact	`e8(-248)`	all	identity	none	0	none	none	248	-248
F_4	F I	`f4(4)`	none	identity	none	4	F4	every node (1, 0)	24	4
F_4	F II	`f4(-20)`	1, 2, 3	identity	none	1	BC1	{4} (8, 7)	36	-20
F_4	compact	`f4(-52)`	all	identity	none	0	none	none	52	-52
G_2	G	`g2(2)`	none	identity	none	2	G2	every node (1, 0)	6	2
G_2	compact	`g2(-14)`	all	identity	none	0	none	none	14	-14

B.2 Classical boards of low rank, as test vectors [V]

Board	Label	Real form	Black nodes	τ	Arrows	Real rank	Restricted roots	Simple restricted roots: orbit $(m_\lambda, m_{2\lambda})$	$\dim \mathfrak{k}$	Character
A_1	A I	`sl(2,R)`	none	identity	none	1	A1	every node (1, 0)	1	1
A_1	compact	`su(2)`	all	identity	none	0	none	none	3	-3
A_2	A I	`sl(3,R)`	none	identity	none	2	A2	every node (1, 0)	3	2
A_2	A IV	`su(1,2)`	none	$1 \leftrightarrow 2$	$1 \leftrightarrow 2$	1	BC1	{1,2} (2, 1)	4	0
A_2	compact	`su(3)`	all	$1 \leftrightarrow 2$	none	0	none	none	8	-8
A_3	A I	`sl(4,R)`	none	identity	none	3	A3	every node (1, 0)	6	3
A_3	A II	`su*(4)`	1, 3	identity	none	1	A1	{2} (4, 0)	10	-5
A_3	A IV	`su(1,3)`	2	$1 \leftrightarrow 3$	$1 \leftrightarrow 3$	1	BC1	{1,3} (4, 1)	9	-3
A_3	A III	`su(2,2)`	none	$1 \leftrightarrow 3$	$1 \leftrightarrow 3$	2	C2	{1,3} (2, 0); {2} (1, 0)	7	1
A_3	compact	`su(4)`	all	$1 \leftrightarrow 3$	none	0	none	none	15	-15
A_4	A I	`sl(5,R)`	none	identity	none	4	A4	every node (1, 0)	10	4
A_4	A IV	`su(1,4)`	2, 3	$1 \leftrightarrow 4$, $2 \leftrightarrow 3$	$1 \leftrightarrow 4$	1	BC1	{1,4} (6, 1)	16	-8
A_4	A III	`su(2,3)`	none	$1 \leftrightarrow 4$, $2 \leftrightarrow 3$	$1 \leftrightarrow 4$, $2 \leftrightarrow 3$	2	BC2	{1,4} (2, 0); {2,3} (2, 1)	12	0
A_4	compact	`su(5)`	all	$1 \leftrightarrow 4$, $2 \leftrightarrow 3$	none	0	none	none	24	-24
A_5	A I	`sl(6,R)`	none	identity	none	5	A5	every node (1, 0)	15	5
A_5	A II	`su*(6)`	1, 3, 5	identity	none	2	A2	{2} (4, 0); {4} (4, 0)	21	-7
A_5	A IV	`su(1,5)`	2, 3, 4	$1 \leftrightarrow 5$, $2 \leftrightarrow 4$	$1 \leftrightarrow 5$	1	BC1	{1,5} (8, 1)	25	-15
A_5	A III	`su(2,4)`	3	$1 \leftrightarrow 5$, $2 \leftrightarrow 4$	$1 \leftrightarrow 5$, $2 \leftrightarrow 4$	2	BC2	{1,5} (2, 0); {2,4} (4, 1)	19	-3
A_5	A III	`su(3,3)`	none	$1 \leftrightarrow 5$, $2 \leftrightarrow 4$	$1 \leftrightarrow 5$, $2 \leftrightarrow 4$	3	C3	{1,5} (2, 0); {2,4} (2, 0); {3} (1, 0)	17	1
A_5	compact	`su(6)`	all	$1 \leftrightarrow 5$, $2 \leftrightarrow 4$	none	0	none	none	35	-35

Board	Label	Real form	Black nodes	τ	Arrows	Real rank	Restricted roots	Simple restricted roots: orbit $(m_\lambda, m_{2\lambda})$	$\dim \mathfrak{k}$	Character
B_2	B II	so(1,4)	2	identity	none	1	A1	{1} (3, 0)	6	-2
B_2	B I	so(2,3)	none	identity	none	2	B2	every node (1, 0)	4	2
B_2	compact	so(5)	all	identity	none	0	none	none	10	-10
B_3	B II	so(1,6)	2, 3	identity	none	1	A1	{1} (5, 0)	15	-9
B_3	B I	so(2,5)	3	identity	none	2	B2	{1} (1, 0); {2} (3, 0)	11	-1
B_3	B I	so(3,4)	none	identity	none	3	B3	every node (1, 0)	9	3
B_3	compact	so(7)	all	identity	none	0	none	none	21	-21
B_4	B II	so(1,8)	2, 3, 4	identity	none	1	A1	{1} (7, 0)	28	-20
B_4	B I	so(2,7)	3, 4	identity	none	2	B2	{1} (1, 0); {2} (5, 0)	22	-8
B_4	B I	so(3,6)	4	identity	none	3	B3	{1} (1, 0); {2} (1, 0); {3} (3, 0)	18	0
B_4	B I	so(4,5)	none	identity	none	4	B4	every node (1, 0)	16	4
B_4	compact	so(9)	all	identity	none	0	none	none	36	-36
C_2	C I	sp(4,R)	none	identity	none	2	C2	every node (1, 0)	4	2
C_2	C II	sp(1,1)	1	identity	none	1	A1	{2} (3, 0)	6	-2
C_2	compact	sp(2)	all	identity	none	0	none	none	10	-10
C_3	C I	sp(6,R)	none	identity	none	3	C3	every node (1, 0)	9	3
C_3	C II	sp(1,2)	1, 3	identity	none	1	BC1	{2} (4, 3)	13	-5
C_3	compact	sp(3)	all	identity	none	0	none	none	21	-21
C_4	C I	sp(8,R)	none	identity	none	4	C4	every node (1, 0)	16	4
C_4	C II	sp(1,3)	1, 3, 4	identity	none	1	BC1	{2} (8, 3)	24	-12
C_4	C II	sp(2,2)	1, 3	identity	none	2	C2	{2} (4, 0); {4} (3, 0)	20	-4
C_4	compact	sp(4)	all	identity	none	0	none	none	36	-36
D_4	D II	so(1,7)	2, 3, 4	$3 \leftrightarrow 4$	none	1	A1	{1} (6, 0)	21	-14
D_4	D I	so(2,6)	3, 4	identity	none	2	B2	{1} (1, 0); {2} (4, 0)	16	-4
D_4	D I	so(3,5)	none	$3 \leftrightarrow 4$	$3 \leftrightarrow 4$	3	B3	{1} (1, 0); {2} (1, 0); {3,4} (2, 0)	13	2
D_4	D I	so(4,4)	none	identity	none	4	D4	every node (1, 0)	12	4
D_4	D III	so*(8)	1, 3	identity	none	2	C2	{2} (4, 0); {4} (1, 0)	16	-4
D_4	compact	so(8)	all	identity	none	0	none	none	28	-28
D_5	D II	so(1,9)	2, 3, 4, 5	identity	none	1	A1	{1} (8, 0)	36	-27
D_5	D I	so(2,8)	3, 4, 5	$4 \leftrightarrow 5$	none	2	B2	{1} (1, 0); {2} (6, 0)	29	-13
D_5	D I	so(3,7)	4, 5	identity	none	3	B3	{1} (1, 0); {2} (1, 0); {3} (4, 0)	24	-3
D_5	D I	so(4,6)	none	$4 \leftrightarrow 5$	$4 \leftrightarrow 5$	4	B4	{1} (1, 0); {2} (1, 0); {3} (1, 0); {4,5} (2, 0)	21	3
D_5	D I	so(5,5)	none	identity	none	5	D5	every node (1, 0)	20	5
D_5	D III	so*(10)	1, 3	$4 \leftrightarrow 5$	$4 \leftrightarrow 5$	2	BC2	{2} (4, 0); {4,5} (4, 1)	25	-5
D_5	compact	so(10)	all	$4 \leftrightarrow 5$	none	0	none	none	45	-45
D_6	D II	so(1,11)	2, 3, 4, 5, 6	$5 \leftrightarrow 6$	none	1	A1	{1} (10, 0)	55	-44
D_6	D I	so(2,10)	3, 4, 5, 6	identity	none	2	B2	{1} (1, 0); {2} (8, 0)	46	-26
D_6	D I	so(3,9)	4, 5, 6	$5 \leftrightarrow 6$	none	3	B3	{1} (1, 0); {2} (1, 0); {3} (6, 0)	39	-12
D_6	D I	so(4,8)	5, 6	identity	none	4	B4	{1} (1, 0); {2} (1, 0); {3} (1, 0); {4} (4, 0)	34	-2
D_6	D I	so(5,7)	none	$5 \leftrightarrow 6$	$5 \leftrightarrow 6$	5	B5	{1} (1, 0); {2} (1, 0); {3} (1, 0); {4} (1, 0); {5,6} (2, 0)	31	4
D_6	D I	so(6,6)	none	identity	none	6	D6	every node (1, 0)	30	6
D_6	D III	so*(12)	1, 3, 5	identity	none	3	C3	{2} (4, 0); {4} (4, 0); {6} (1, 0)	36	-6
D_6	compact	so(12)	all	identity	none	0	none	none	66	-66

Appendix C. Dictionary

This site	Wildberger ([W20], [W-MS])	Numbers-game literature	Lie theory
board	directed multigraph X	weighted graph, Coxeter graph, "GCM graph"	Dynkin diagram; generalized Cartan matrix
bidirected board; one-way board	bidirected multigraph; a multigraph with an edge of type $(m, 0)$		generalized Cartan matrix; (no counterpart)
node, city	vertex, city	node	simple root α_i
roads, counts (m, n)	edge of type (m, n)	amplitudes, weights k_{ij}	Cartan integers $-C_{ji}$, $-C_{ij}$
population	population	(not used)	element of the root lattice, simple-root coordinates
numbers, position	dual population; weight	position, configuration, labelling	weight, Dynkin labels
mutate	mutation, reflection s_x		simple reflection on roots
fire	dual mutation r_x	firing, a move	simple reflection on weights
legal play	positive dual mutations; "increasing" mutations seen from the other side	legal play, positive (or negative) play	descent in the weak order; reduced words
the translator $a = Cp$	the map τ (with the opposite sign)		$\lambda \mapsto (\langle \lambda, \alpha_i^\vee \rangle)_i$
root population	root population, root		real root
simple root population	simple root δ_x		simple root
fundamental position e_i	fundamental weight ϵ_x	fundamental position	fundamental weight ω_i
balanced board, weights d_i	balanced multigraph, balanced weights α_i		symmetrizable matrix; $(\alpha_i, \alpha_i)/2$
form value (p, p)	$Q(p)$, the Tits form		invariant bilinear form
frozen population	Perron-Frobenius population α_0		null root δ; marks
level		$\delta \cdot v$ (with their matrix)	level $\langle \lambda, K \rangle$; comarks
finite, affine, neither	List B, List A, spectral radius above 2	admissible, looping, divergent	finite, affine, indefinite type
raising move; Kostant's game	increasing mutation; incremental when the rise is 1		climbing the root poset
heap	frame; X-string for the class of plays		heap of a word; commutation class
minuscule node	incremental frames, minuscule heaps	λ-minuscule elements	minuscule weight
ring			nonzero Dynkin label of a dominant weight

This site	Wildberger ([W20], [W-MS])	Numbers-game literature	Lie theory
black, white, arrows			compact simple roots; non-compact; the involution τ
shadow			restricted root
restricted board			Dynkin diagram of the restricted root system; relative Weyl group
folded board		folding	folding by orbit sums; orbit Lie algebra

Appendix D. Corrections to the project note "algebraic data"

The note gets the rule of the numbers game right: the move, the matrix form $x \mapsto x - x_i C e_i$ (which is R_i of section 2.3), the condition $x_i > 0$, termination on finite type, and, in its section 2, the reading of the numbers as coordinates of weights. It is also right that transposing C exchanges a root system with its dual ($B_n \leftrightarrow C_n$). Its description of the mutation game needs three main corrections. Three smaller points follow, two of which (5 and 6) concern what the note says the positions of the numbers game are.

Main corrections.

1. **The graph does not change.** The note says the mutation game "mutates the graph itself", with arrows reversed and edges between neighbours toggled. In Wildberger's game a mutation changes only the population of the mutated city, and the multigraph is fixed throughout ([W20] sections 2 and 5: "a mutation at A changes only the population of A"). The operation the note describes is modelled on Fomin-Zelevinsky quiver mutation, a different construction with the same name (section 2.16), and the mutation game is not its "combinatorial shadow".
2. **The label rule in the note is a numbers-game move.** The note's update $x \mapsto x - x_i C^{\mathsf{T}} e_i$ negates x_i and changes the neighbours. That is the firing move of the numbers game on the reversed board. Wildberger's mutation is
$$x \mapsto x - (e_i^{\mathsf{T}} C x)\, e_i, \qquad \text{that is} \qquad x_i \mapsto x_i - \sum_j C_{ij} x_j,$$
which changes one entry and leaves every neighbour alone (S_i of section 2.2).
3. **The games do not coincide on simply laced diagrams.** With C symmetric, $R_i = S_i^{\mathsf{T}}$: the two moves are transposes, and they are different matrices. The correct relations are $CS_i = R_i C$ (translate a population by $a = Cp$ and the moves correspond) and $R_i(C) = S_i(C^{\mathsf{T}})^{\mathsf{T}}$ (section 2.4). "The mutation game on C is the numbers game on C^{T}" is true as a statement about adjoint actions on dual spaces and false as a statement that the two games have the same positions and moves.

Smaller points.

4. There is no sign condition on a mutation. Any city may mutate. Conditions define variants: only raising moves (Kostant's game), or, on the numbers side, only positive firings.
5. The positions reached by the numbers game from a generic start correspond to the elements of the **Weyl group**; the root system is the set of positions of the **mutation** game (and appears in the numbers game as the fired values, F6).
6. The lattices are: populations in the **root lattice**, numbers in the **weight lattice**. The note places the numbers game on the root lattice and the mutation game on the coroot lattice.

Wildberger's own statement of the duality is Proposition 0.3 of the manuscript, $p\, s_x \tau = p\, \tau r_x$ with $\tau = A - 2I$ (section 2.4(e)).

Appendix E. What a person should check before publishing

In order of importance. Each item says what to open and what to compare. Item 4, for example, is written `E4` in `content/unverified.json` and in the `item` attribute of `<Unverified>` (section 7.3). The gates of section 9.1 name the items due at each release; an item that no gate names is due at G6. An item that stays open does not stop a release: the sentences that rest on it are published in their fallback wording.

1. **Satake multiplicities against Helgason.** Open [Hel01] pp. 532-534 (or Araki's table, [Ara62] pp. 32-33) and compare with section 6.4 and Appendix B. The black nodes and arrows were confirmed in two independent published encodings. The multiplicities for A II, A III and C II with $p = q$, D III (except $n = 5$), E II, E VI, E VII and E IX rest on computation and on dimension counts, with no published number read. A reviewer of this specification recalls Helgason's table as agreeing with every line of section 6.4, these lines included; that is memory, and it does not close the item.
2. **Mozes' paper.** Open [Moz90] (free to read, 15 pages). Record its sign convention, its matrix convention, the numbers of the theorems on order independence and termination, whether its introduction mentions the Olympiad problem, and whether it uses the words "numbers game". Update the history page and section 2.9.
3. **Björner and Brenti, section 4.3.** Confirm the number of the main theorem (believed to be 4.3.1) and its wording.
4. **Kac's book.** Look up and record: the definition of symmetrizable and Exercise 2.1; the classification by principal minors (believed to be Proposition 4.7) and the fact that matrices of finite and affine type are symmetrizable, both used by `cartan_class`; the statement that real roots are positive or negative, and the lemma that a positive real root other than a simple one is lowered by some simple reflection, used by `is_root_population`; marks, comarks and the word "level"; the names of the twisted affine diagrams and which is the transpose of which; Kac's numbering of E_6, E_7, E_8, G_2 for the Atlas. Places suggested from a reviewer's memory, to be confirmed: Theorem 4.3, Proposition 4.7 and Theorem 4.8 for the classification; Proposition 3.12 for the Tits cone, which gives the lowering lemma; Proposition 5.8 for the Tits cone of an affine matrix; Proposition 6.3 for the real roots of affine algebras, which gives the table of least k in section 2.12.
5. **Wildberger's lectures.** Open the two channels and list the episodes of "Dynamics on Graphs" with titles, dates and whether each is public. Check every video link in `SOURCES.md` 11.1. Decide which to link.
6. **The affine fate table for non-simply-laced and twisted boards** (section 2.12). The section now carries an argument supplied by a reviewer. Confirm its second reference (Kac, the Tits cone of an affine matrix), or find the table stated in Eriksson's survey [Eri94a]; until then the site keeps the wording "computed, with this argument".
7. **The face-counting rule** of section 2.14: it now agrees with exact convex hulls in 48 cases, so what remains is its attribution. Compare with [CKPS95] or Coxeter before naming anyone. The published face counts of 1_{22}, 2_{31}, 2_{41} and 1_{42}, with which the rule was also compared, were typed from memory; look them up before the site calls them published values (a reviewer recalls the four lists as the published ones).
8. **Is the link between Satake diagrams and node-firing games new?** No source was found that plays θ, or the restricted reflections, as moves of these games; but the literature search for this question was not run, because the search allowance was exhausted. The site should make no claim of novelty until a search has been done.
9. **Attribution of the golden folding** $E_8 \to H_4$ (Lusztig; Shcherbak; Moody and Patera) and of the name "rectified 4_{21}" for the orbit of ω_7.
10. **Section and theorem numbers quoted from memory**: everything marked [U] in `SOURCES.md` 11.5 (Humphreys, Bourbaki, Fulton and Harris), Smith's theorem on graphs of spectral radius 2, the item of [Ste59] behind F14 (a reviewer suggests, from memory, Bourbaki, Chapter VI, section 1, no. 11, Proposition 31), and the journal data marked [U] in 11.1, 11.2 and 11.4.
11. **The proposer of the 1986 Olympiad problem**, if the history page names one.
12. **Whether route (a) for mathematics works with MDX under Astro 7** (section 7.2 item 3). This is the phase 0 spike.

13. **Kostant's game**: A. Knutson's MathOverflow answer (thread 171999), for the wording of the attribution.
14. **Wildberger's List A against Kac's affine tables**, edge by edge, if the site says they are the same list. A reviewer's reading of the printed labels, to be confirmed: (a) $\tilde{A}_n$, (b) $\tilde{D}_4$, (c) $\tilde{D}_n$, (d) to (f) $\tilde{E}_6$, $\tilde{E}_7$, $\tilde{E}_8$, (g) $D^{(2)}_{n+1}$, (h) $\tilde{B}_n$, (i) $\tilde{C}_n$, (j) $A^{(2)}_{2n-1}$, (k) $A^{(2)}_{2n}$, (l) $E^{(2)}_6$, (m) $\tilde{F}_4$, (n) $\tilde{G}_2$, (o) $D^{(3)}_4$, (p) $A^{(2)}_2$, (q) $\tilde{A}_1$.
15. **The last sentence on a failed condition 3** in section 2.15 (that the same permutation of roots arises from $\mathfrak{sl}(3,\mathbb{R})$ on a Cartan subalgebra that is not maximally split, as Vogan diagrams record). A reader who knows [Kna02] chapter VI can confirm or reword it. A reviewer derived the substance: on the fundamental Cartan subalgebra of $\mathfrak{sl}(3,\mathbb{R})$ the Cartan involution fixes one pair of roots, which must be non-compact because $\dim \mathfrak{k} = 3$, and exchanges the other two pairs, and this is $-s_1$. What remains open is the wording about Vogan diagrams.
16. **Attributions of the tableau counts** in F12 (Stanley for the staircase; the square shape for types B and C), and a reference for the remark in section 2.13 that the fixed subalgebra for A_{2n} is of type B_n.
17. **Everything else labelled [U].** Known cases that no item above covers: the reading of Proctor's restricted game in this document's sign convention (section 2.9); in Appendix F, the state of the Java applets, the licence of CoReLG, the remark on the orthogonal group, and the two statements that nothing was found; in `SOURCES.md`, the labels [U] of sections 11.0, 11.3 and 11.4 that items 1 and 10 do not name. A lesson that needs any other statement labelled [U] adds an item with the next free number (`E18`, `E19`, …) to `content/unverified.json`, quoting the statement and saying where it can be checked. The [U] labels of section 7 concern tools, and the builder settles each one when the tool is first used.

Appendix F. Prior art

Interactive or computational work found on the two games and their surroundings. "Borrow" and "avoid" are design notes. [S] for every row unless marked; rows read through an automated summary carry some risk in their details.

Work	What it does	Technology and state	Borrow	Avoid
"ADE Concurrent Game", on Wildberger's UNSW pages under the name of T. S. Hoang (the documentation, which names no author, is dated 25 November 2004, the same day as the metadata of the manuscript PDF; both dates were read from the files)	integers on an A, D or E graph, numbered as in section 2.10; one move fires all positive nodes at once; edit and play modes, undo, a trace of states	Java applet; pages load; applets do not run in current browsers [U]	edit and play modes, undo, visible history, "play to the end"	plug-in technology; simply laced only; no roots or weights shown
GEOMETOR "Mutation Explorer" (2021)	random or round-robin mutations on a simple graph from one Martian, with a plot of the total population	HTML, JavaScript, D3; MIT; 23 commits in June 2021	a first proof that the game works in a browser	no root sets, no directed roads, no explanation
`mutation-game`, A. Cortis (April 2026)	Python library: board from a Dynkin name or adjacency matrix, mutations, all roots by breadth-first search, shortest mutation paths, plots	Python 3.11; MIT; version 0.1.5	the shape of the API; the pairing of code with background chapters	static figures; floating-point matrices; the documentation errors listed below

Work	What it does	Technology and state	Borrow	Avoid
cut-the-knot page for the pentagon problem	the Olympiad game	Java-era page	the puzzle as an entry point	
`adn1995/numbers-game`	Sage code for the numbers game on Coxeter graphs	GPL-3.0; described by its author as incomplete		
B. Keller, quiver mutation applet	click a vertex to mutate a quiver; history with back and forward; random sequences	JavaScript on canvas, and a Java program	the interaction model: click a node, see history	a different mathematics (section 2.16); a tool with no explanation
Langlands Programs preview, Coxeter apps	Coxeter group explorer, Bruhat order viewer, Kazhdan-Lusztig polynomials, in the browser	a TypeScript port of PyCox	proof that a TypeScript engine carries serious Coxeter computations	forms and tables with no narrative
Group Explorer 3	Cayley diagrams, multiplication tables, cycle graphs for small groups	plain JavaScript, three.js; LGPL	several linked views of one object	abstract groups only; do not copy LGPL code
SimpLie (T. Nutma)	draw a Dynkin diagram, get type, roots, representations; LaTeX export	Java desktop; GPL; last commit 2021	"diagram in, structure out"; LaTeX export	desktop Java
Stembridge's Coxeter plane pictures; Vogan's explanation of the E_8 picture	static projections; the rule "join each root to its 56 nearest neighbours"	images, text	the canonical picture and the plain-language edge rule	copying the images
Stella4D; Jenn3d; Polyhedra Viewer; vZome	polytope viewers up to dimension 4; Cayley graphs as polytopes; operations linking polyhedra; a history inspector	desktop or web, various licences	projection with section views; objects linked by operations; replayable construction history	stopping at dimension 4; closed or GPL code
CoReLG (GAP)	real forms, Satake and Vogan diagrams as text output	command line; GPL [U for the licence]	an external oracle for section 6.4	
`dynkin-diagrams` (LaTeX)	typesets Dynkin, Coxeter and Satake diagrams	LPPL	the export target of section 6.7	
SageMath root systems and crystals	roots, weights, Weyl groups, plots, crystals	Python; needs a server or local install	an external oracle that needs no translation of conventions	

No interactive tool for drawing, checking or classifying Satake or Vogan diagrams was found (one search only [U]). **No tool was found that shows both games on one board**, that grows roots and weights from the reader's moves on non-simply-laced boards, or that carries the games through to polytopes, representations and real forms.

Three of the interactive items in this niche are Java applets that no longer run. A static site with no runtime dependency is the durable form.

Statements in the documentation of `mutation-game` 0.1.5 that the site must not repeat [S, read in the package; the corrections are [V] or standard]:

- that the reflection formula it gives acts "on the weight lattice": the formula is the action on root coordinates, and the action on weights is the numbers game;
- that its naming of B_n and C_n follows Wildberger and is swapped relative to Bourbaki: its B_3 is Bourbaki's B_3, and it is Wildberger's lists that are swapped;
- that the game with automatic detection of finite type works for any directed multigraph: on a one-way road its test reports finite type although the root set is infinite;
- that every root is positive or negative on any graph the package accepts: false on the one-way road;
- that positive definiteness occurs exactly for the *A*-*D*-*E* diagrams: the forms of B, C, F, G are positive definite too;
- that the Weyl group of E_8 is isomorphic to an orthogonal group over the field of two elements: the orders differ by a factor of 2 [U as worded here; the order of the Weyl group is 696729600 [V]];

- that the manuscript dates from 2003.

Appendix G. Schema of `golden_data.json`

All nodes are labels (section 5.1, E-7): 1-based Bourbaki labels on finite boards, 0 for the added node of an affine board, which is listed first. Matrices are lists of rows; words are lists of node labels; every number is an integer unless stated. The only floating-point values are the ring radii under `coxeter_plane`. Section 5.7 says how each key is compared and in which phase.

Fields whose values are node labels (rule 2 of section 5.7): `longest_word`, `minus_w0`, `minuscule_nodes` under `types`; `black_nodes`, `tau`, `arrows`, `w_X_word`, `restricted_simple_orbits`, `restricted_orbits_in_bourbaki_order` in `satake`; `black_nodes` and `tau` in `satake_examples`; `black_nodes` in `generalized_satake_example`, where τ is the identity; both lists of each pair in `admissible_pairs`; `symmetry`, `orbits`, `folded_orbits_in_bourbaki_order` in `folding`; `nodes` and `bourbaki_order` in `classification_tests`, including `nodes` of `smallest_nonfinite_subboard`; `word_lowest_node_rule` in `pentagon_example`; `moves` in `one_way_counterexample`; `mutation_nodes` and `firing_nodes` in `bigint_test`; `node` in `minuscule_representations`; `longest_word` in `golden_boards`; `pairs_plain_golden` in `golden_folds`. A list with one entry per node (a population, a position, `marks`, `comarks`, `symmetrizer`, `bipartite_colouring`, `fundamental_orbit_sizes`, the four lists under `kostant`, `least_k_with_e_x_plus_k_delta_a_root`, the rows of `e8.fundamental_weights_in_root_coordinates`) is in the order of the board's labels.

Lists of records are matched by their identifying fields, never by index (rule 5 of section 5.7): `satake` by `dynkin_type` and `real_form`; `folding` by `type` and `symmetry`; `weyl_dimension` and `orbit_plus_zero_representations` by `type` and `highest_weight`; `wythoff` by `type` and `rings`; `minuscule_representations` by `type` and `node`; `satake_examples` by `type`, `black_nodes` and `tau`; `generalized_satake_example` by `type` and `black_nodes`; `twisted_affine` by `name`; `classification_tests` and `layer_growth_examples` by `label`; `golden_folds` by `from` and `to`.

Key	Content
`about`, `conventions`	text; the conventions of section 2 in one line each (`node_labels`, `cartan`, `adjacency`, `mutation`, `firing`, `legal_firing_rule`, `longest_word_rule`, `rank_two_naming`, `floats`)
`types`	one record for each of A_1 to A_8, B_2 to B_8, C_2 to C_8, D_4 to D_8, E_6, E_7, E_8, F_4, G_2
`types[T].cartan`, `.adjacency`, `.symmetrizer`	C, A, the d_i
`.n_roots`, `.n_positive_roots`, `.positive_roots`	counts; the list (sorted by total, then lexicographically) is present for rank at most 4 and for E_6, E_7, E_8
`.highest_root`, `.numbers_of_highest_root`	θ as a population, and $C\theta$
`.highest_short_root`, `.numbers_of_highest_short_root`, `.n_short_positive`, `.n_long_positive`, `.long_to_short_ratio`	only on B, C, F, G
`.highest_root_of_reversed_board`	the highest root of the board C^{T} (F11), which differs from $\theta^\vee$ in general
`.coxeter_number`, `.det_cartan`, `.simply_laced`, `.exponents`, `.weyl_order`	
`.longest_word`, `.fired_values`, `.fired_tally`	the game from all ones with the lowest-node rule; the tally maps each value (as text) to the number of times it is fired
`.minus_w0`	images of the nodes under $-w_0$
`.bipartite_colouring`	0 or 1 for each node
`.fundamental_orbit_sizes`, `.minuscule_nodes`	
`.n_reduced_words_w0`, `.length_distribution`	present when $\lvert W\rvert \le 60000$: the number of complete games, and the numbers of positions first reached after 0, 1, 2, … legal moves from all ones
`.coxeter_plane`	present for rank at least 2: `rings`, a list of pairs [radius rounded to 6 decimals, number of roots], innermost first; `distinct_points`; `h_points_on_every_ring`

Key	Content
`coxeter_plane_types_with_coincident_points`	the types where `distinct_points` is less than the number of roots
`satake`	167 records with the fields of section 6.4
`admissible_pairs[T]`	every admissible pair on board T: black nodes, then the images of all nodes under τ; sorted
`admissible_counts[T]`	`real_forms_including_compact` (classes of admissible pairs), `catalogue_records` (they differ only on D_4), `admissible_pairs_total`
`satake_examples`	twelve pairs: `type`, `black_nodes`, `tau`, the verdicts `condition_1`, `condition_2`, `condition_3`, `admissible`, the names in `real_forms` when admissible, and `reasons` in words, keyed by the failed condition
`generalized_satake_example`	G_2 with one black node, two records: `twice_the_shortest_shadow` (of the two shortest shadows, the one that is larger in lexicographic order) and `multiplicities_of_1_2_3_times_it`
`folding`	nine rows. Fields: `type`; `symmetry` (images of all nodes); `order`; `orbits`; `folds_to` and `folded_cartan` (both `null` when the board does not fold, with the cause in `reason`); `folded_orbits_in_bourbaki_order`; `averaged_roots_type`; `averaged_simple_cartan`. The rows and columns of the two matrices follow the order of `orbits`, which need not be the Bourbaki order of the type named (for the rotation of D_4 the stored matrix is that of G_2 with its nodes exchanged); `folded_orbits_in_bourbaki_order` gives the orbits in the order in which the folded board is drawn. Further fields: `averaged_root_classes` (per class: `class`, `roots`, `form_value` as text); `quasi_split_real_form` (`null` for the symmetry of order 3)
`weyl_dimension`	30 records: `type`, `highest_weight`, `dimension`
`affine[T]`	for each of the 32 types: `name`, the name by which the board is recognised (`A4~`; `C2~` for B_2); `constructor_name`, the name that builds exactly this matrix with `boardFromName` (`B2~` for B_2); `cartan_node0_first`; `marks`; `comarks`
`twisted_affine`	ten records: `name`, `cartan`, `marks`, `comarks`, `symmetrizer`, `least_k_with_e_x_plus_k_delta_a_root` by node, and `reverse_of` on the seven boards that are a reversed untwisted board
`affine_names`	the names of all affine boards with 2 to 9 nodes, keyed by the number of nodes as text
`pentagon_example`	`cartan`; `start_olympiad_convention` and `start`; `level`; `word_lowest_node_rule`, `fired`, `end`, `moves`; `complete_games`, the number of complete games from that start, all of the same length; `note`
`classification_tests`	thirteen boards: `label`, `cartan`, `components` (the records returned by `classify`: `nodes`, `kind`, `balanced`, `class`, `name`, and `bourbaki_order` or `marks` and `comarks`), `smallest_nonfinite_subboard` (`nodes` and `class`, or `null`), `spectral_radius_4_decimals` (text, or `null` on the one-way board)
`layer_growth_examples`	four boards, three with bounded layers and one with geometric growth: `label`, `cartan`, `layers`, `root_populations_found`, `new_per_layer` (starting with the seeds), `largest_entry_digits`, and `stop_at_2000` and `stop_at_100000`, the result of the same search with that cap: `positions`, `truncated`, `complete_layers`, `positions_in_complete_layers`
`kostant[T]`	four lists indexed by starting node: `end_from_node`, `moves_from_node`, `final_numbers_from_node`, `affine_board_from_node` (the name of the affine board that the recipe of section 2.12 builds)
`one_way_counterexample`	`cartan`, `start`, `moves` and the mixed `result` of section 2.11

Key	Content
`wythoff`	26 polytopes: `type`, `rings`, `name`, `f_vector`, `edges_enumerated` (false above 70000 edges), and `edge_length_squared` (text) on the thirteen records where it was computed
`wythoff_hull_checks`, `wythoff_note`	the number of polytopes compared with exact convex hulls; the edge-length rule in one line
`heaps`	for six boards: `complete_games`, `distinct_heaps`, `games_per_heap` (sorted); for three minuscule plays (`A4 omega_2`, `D5 omega_5`, `E6 omega_1`): `moves`, `orbit_size`, `complete_games`
`minuscule_representations`	18 records: `type`, `node`, `dimension`, and `weights_dynkin_labels` (the positions; `null` above dimension 27)
`orbit_plus_zero_representations`	5 records: `type`, `highest_weight`, `dimension`, `name`, `weights_dynkin_labels`
`e8`	`fundamental_weights_in_root_coordinates`; `neighbours_at_inner_product_1` (56); `h4_projection` (`exponents_kept`, `shell_sizes`, `radius_ratio`)
`golden_boards`	H_2, H_3, H_4: `matrix_as_pairs_a_plus_b_phi` and `fired_values_as_pairs` (a pair $[a, b]$ means $a + b\varphi$), `n_roots`, `group_order`, `longest_word`
`golden_folds`	three foldings: `from`, `to`, `pairs_plain_golden`, `rule`
`bigint_test`	the board with three roads each way (`cartan`): `population_start`, `mutation_nodes` (200 moves), `population_end` (decimal strings); `numbers_start`, `firing_nodes` (200 moves), `firing_mode`, `numbers_end`; `first_move_after_which_an_entry_exceeds_2_to_53`
`wildberger_lists`	`names` (what each of Wildberger's names means in Bourbaki's terms); `misprints_in_paper_and_manuscript` and `misprints_in_manuscript_only`, the misprints of section 2.10 by list, each as a pair of lists: printed, intended

Appendix H. Starter `CLAUDE.md`

```
# Two Games on a Dynkin Diagram: rules for this repository

Read SPEC.md sections 0, 1, 2 before changing code. SOURCES.md resolves citation keys.

## Conventions (do not change; SPEC.md section 2)
- Cartan matrix: C[i][j] = <alpha_i^vee, alpha_j> (Kac, SageMath). Bourbaki prints the transpose.
- Board: A[i][j] = roads from i to j = -C[j][i]. More roads run from long to short.
- Mutation (populations, roots): only entry i changes: p[i] <- p[i] - sum_j C[i][j] p[j].
- Firing (numbers, weights): a[j] <- a[j] - C[j][i] a[i] for all j, so a[i] <- -a[i].
- Legal firing: the number is positive. The game ends when none is positive.
- Translator: a = C p. The number on a node is the drop a mutation would cause there.
- The board never changes (quiver mutation is a different operation; SPEC 2.16).
- Nodes: positions 0..n-1 inside the engine (as in the oracle); labels everywhere else
  (Bourbaki 1..n; 0 first on affine boards). Only toLabels / toPositions convert. (SPEC 5.1, E-7)
- Where a choice is free, take the lowest position first (play, kostantPlay, searches). (E-5)
- B2 and C2 are one board: computed names say B2, its affine board is C2~.

## Ground truth
- reference/golden_data.json is the contract for packages/engine. Port from reference/lie_games.py.
- Regenerate with: python3 reference/verify.py reference/golden_data.json (stdlib only; must report
  ↪ 0 failed).
  npm run verify reruns it and compares with the committed file.
- Never edit golden_data.json by hand. If the math looks wrong, add a failing check to verify.py and
  ↪ stop.
- Comparison rules and the phase of each golden key: SPEC.md 5.7.
```

```
## Build
- Node >= 22.12, npm workspaces. Astro 7 static, MDX, Svelte 5 (runes), TypeScript 6.0.x (not 7).
- Astro 7's default Markdown processor does not run remark or rehype plugins; see SPEC.md 7.2.
- KaTeX CSS and fonts must match the KaTeX version that rendered the HTML. Self-host them.
- GitHub Pages serves under /<repo>/: site and base come from the environment (SPEC 7.2 item 10);
  build every link from import.meta.env.BASE_URL.
- Exact pinned versions; commit the lockfile.
- Commands: npm ci; npm test; npm run build; RELEASE=1 npm run build; npm run e2e; npm run size;
  npm run verify.
- For automated checks use astro build + astro preview, not astro dev.

## Rules
- Mathematics lives only in packages/engine, which has NO dependencies (no DOM, no npm packages).
  Components call it. Every dependency added to the site is recorded in DECISIONS.md (version,
  licence, and size if it is shipped to the reader).
- Numbers: SafeInt (guarded number) only on named finite-type boards; bigint (Big) on every other
  board. Both exist from phase 0 behind the Arith interface. Counts that can be large (dimensions,
  numbers of games) are bigint.
- Every enumeration has a cap and returns {items, truncated}; every play returns a status.
  Off the finite types a search is bounded by layers (maxDepth, default 40), never by guessing.
- Main thread: at most 2000 items or 2000 moves per call. Anything larger runs in the worker (E-9).
- Satake records for display come from satakeRecord, which applies the C2 naming of SPEC 6.4.
- A board with a one-way road is allowed in the editor; theory functions refuse it (SPEC 5.6).
- Every tool: default state rendered at build time (works without JavaScript), keyboard operable,
  announced moves (templates in SPEC 7.5), sign shown by numeral and fill as well as colour,
  undo, reset, link, export, table view, test hooks data-node / data-board / data-value /
  data-can-act / data-out (SPEC 4.0).
- Lessons read fully without JavaScript. Nothing essential behind a click.
- Numbers in prose come from <G k="..."/>, numbers in formulas from <Tex src="...{{path}}..."/>.
  Anything resting on an open item of SPEC Appendix E is wrapped in
  <Unverified item="E..." fallback="...">. Never set an item to closed; the owner does that.
  Citations use <Cite k="..."/>. Do not copy text or figures.
- Prose style: plain sentences, no em-dashes, terms from SPEC.md Appendix C.
- One tool or one lesson per pull request, with tests. Merge it yourself when every required check
  is green. Open decisions: use the default in SPEC.md section 10 and log it in DECISIONS.md.
- The public site changes only when a version tag is pushed or the owner runs deploy.yml by hand;
  the owner tags releases.
- No tool's default state may need the worker: precompute at build time, or offer a button.

## Phases (SPEC.md section 9)
0 foundations; 1 the two moves (T1, T2, T9; M0, M1); 2 form and classification (T3, T4, T15, T16;
 ↪  M2, M3);
3 duality and Weyl group (T5-T8; M4, M5; release 0.5); 4 multiple edges and geometry (T10, T11; M6,
 ↪  M7);
5 representations (T12; M8); 6 real forms, atlas (T13, T14; M9, M10; release 1.0); 7 extensions.
Start a phase when the automatic part of the previous gate passes. The owner's part is due at
 ↪  releases and never holds up a phase.
```

11. Sources

11.0 How to read this section

Each entry gives the citation, where a copy can be read, a status, and its role on the site. Entries whose keys are not cited in `SPEC.md` are background for the history and sources pages; a few items (seminar abstracts, expository posts, software) have no key because nothing cites them by key. Status uses the labels of `SPEC.md` section 0.3: [S] read in the source itself during this research; [S2] reported by a later source that was read; [V] checked by `reference/verify.py`; [U] unverified. "Read" means that the text was obtained and read by the research tools; some texts were read through a tool that returns an extraction of a web page or PDF, so wording, formula numbers and page numbers taken from them should be compared with the original before they are quoted.

Limits of the research. Three kinds of limit shaped this record, and each entry says when one applies.

1. *Hosts that refuse automated reading.* ScienceDirect (Mozes; Eriksson's journal papers; Proctor 1984 and 1999; the published versions of two of Wildberger's 2003 papers), Project Euclid (Araki), JSTOR (Satake; Kostant), Springer and Cambridge book pages, MathOverflow, and YouTube watch pages. Papers on those hosts were confirmed as citations and described from later papers that cite them.
2. *Books.* Bourbaki, Humphreys' Lie algebra text, Kac's book, Helgason, Fulton and Harris, Carter, Coxeter's *Regular Polytopes* and Conway and Sloane were not opened. Section and theorem numbers attributed to them are from memory and marked [U].
3. *Search allowance.* The web-search allowance for this research ran out before three questions were settled: the list and dates of Wildberger's video lectures; whether anyone has connected Satake diagrams with node-firing games; and further interactive prior art. A follow-up request can continue these searches.

What was read in full: both of Wildberger's texts on the mutation game (the 2020 paper and the manuscript, from the project files), his G_2 paper and the preprints of the two other 2003 papers; Donnelly's and Donnelly and Eriksson's papers; Eriksson's 1995 paper in the Journal of Algebraic Combinatorics; Gashi, Schedler and Speyer; Kolb, Dobson and Kolb, and Regelskis and Vlaar on Satake diagrams (arXiv versions); Steinberg 1959; Fomin and Zelevinsky; Stembridge; Bernstein, Gelfand and Ponomarev; and the software sources named below.

11.1 The mutation game (Wildberger) and work derived from it

[W20] N. J. Wildberger, "The Mutation Game, Coxeter-Dynkin Graphs, and Generalized Root Systems", *Algebra Colloquium* 27 (2020), no. 1, 55-78. DOI 10.1142/S1005386720000061. MSC 05C20, 20F55, 17B22.

- Access: closed (World Scientific); no open copy was found. The PDF is in the project as `S1005386720000061.pdf` and was **read in full** for this specification [S].
- Contents by section: 1 Introduction: ADE graphs. 2 The Mutation Game on a Directed Graph (the Mars story; a three-city example with populations 5, 4, 7; the Mutation Conjecture, unnumbered, p. 57). 3 Generalized Root Systems and Dynkin Diagrams (root systems over a general bilinear form). 4 Directed Multigraphs (edge "type (m, n)"; adjacency matrix; spectral radius; **Theorem 4.1**: for a connected bidirected multigraph without loops, spectral radius 2 exactly for List A and below 2 exactly for List B; Corollary 4.3). 5 Mutation-Reflections on Directed Multigraphs (the formula for s_x; **Theorem 5.1**: $s_x^2 = 1$, non-neighbours commute, and $s_x s_y$ has finite order above 2 exactly for edge types $(1,1)$, $(1,2)$ or $(2,1)$, $(1,3)$ or $(3,1)$, with orders 3, 4, 6; definitions of simple root, root population, $R(X)$, $R^{\pm}(X)$). 6 Multigraphs with Finite Root Populations (Proposition 6.1 and List A2: explicit mutation words that add the Perron-Frobenius population α_0 to a simple root on every List A graph). 7 Simple Graphs and the Canonical Bilinear Form (the form with values 2, -1, 0; Proposition 7.1 invariance; Corollary 7.2; the remark that rational trigonometry lets one work with the Tits form $Q(x, y) = 2(x^2 - xy + y^2)$ of A_2 without diagonalising). 8 Quadratic Forms on General Multigraphs (Lemmas 8.1 to 8.3; balanced weights and balanced multigraphs; **Theorem 8.4** and **Theorem 8.7**: an invariant form exists exactly for balanced graphs and is unique up to a constant; **Theorem 8.8**: for balanced graphs, W finite iff $R(X)$ finite iff Q positive definite iff spectral radius below 2 iff List B).

9 Explicit Posets of Positive Roots (A_4, B_2 to B_4, C_2 to C_4, D_4, D_5, E_6, E_7, F_4, G_2; the E_8 diagram is left "for another paper").

- What it does not contain: the numbers game beyond one sentence in the abstract and introduction (which call the mutation game dual to it and cite Mozes, Björner, Eriksson and Donnelly); any representation theory; anything on real forms. Its reference list has 31 items. One of them is Araki's 1962 paper on the root systems of symmetric spaces [Ara62], cited once in the introduction in a sentence about Dynkin's study of root systems; nothing else in the paper touches real forms.
- Naming and misprints [V: every printed list of section 9 was typed into `verify.py` and compared with the computed roots; section 2.10]: its B_n for $n \geq 3$ is Bourbaki's C_n with the node order reversed, its C_n is Bourbaki's B_n reversed, its F_4 is Bourbaki's F_4 reversed, its G_2 agrees with Bourbaki, and its D_n and E_n use an extra vertex 0. The lists for A_4, B_2 to B_4, C_2, C_3, D_4, D_5, E_6, E_7 and F_4 are correct as printed. In the C_4 list "1211" should read "2211"; in the G_2 list "12" should read "21". In Example 8.6 (the same example is on p. 20 of the manuscript, where the figure was seen) the fourth edge joins x_4 to x_2, so "$\beta_{41} = 60$" and the term $-60p_4p_1$ should read β_{42} and $-60p_4p_2$; the corrected form is invariant under all four mutations and the printed one is not.
- List A has seventeen entries. By their printed Perron-Frobenius labels they are Kac's sixteen affine diagrams with the four-armed star entered separately from the family $\tilde{D}_n$; the comparison was not made edge by edge [U]. The paper's definition of a directed multigraph (section 4) allows one-way edges, and the statement of the Mutation Conjecture on p. 57 says "any multigraph"; sections 5, 6 and 9 and the theorems of section 8 assume connected bidirected graphs, while section 8 opens with a general directed multigraph (`SPEC.md` section 2.11).
- Role on the site: the citable published source for the mutation game on directed multigraphs, the classification by spectral radius, the invariant form, and the root posets.

[W-MS] N. J. Wildberger, "The Mutation Game, Coxeter Graphs, and Partially Ordered Multisets", unpublished manuscript, 79 pp., undated. Project file `MutationGameCoxeterGraphs.pdf`; also at https://web.maths.unsw.edu.au/~norman/papers/MutationGameCoxeterGraphs.pdf (link not re-checked).

- Status: [S] read (text and page images). A working draft: results are all numbered 0.x, there are unresolved cross-references, no abstract and no bibliography. The PDF metadata date is 25 November 2004 (read from the project file with `pdfinfo`). Wildberger cited it as "preprint (2001)" in two preprints and as "in preparation" in the published G_2 paper; Donnelly cites it as "preprint" with no year. Cite it as "unpublished manuscript, undated (circulated c. 2001 to 2004)". The date "2003" that appears in the documentation of the `mutation-game` Python package has no other support.
- Contents shared with [W20], under different numbers: the spectral classification (Theorem 0.5), the orders 3, 4, 6 (Theorem 0.6), the invariant form (Theorems 0.7 to 0.9), root posets (pp. 23-29). On p. 2 it poses "The Mutation Problem" for simple graphs, the earlier form of the Mutation Conjecture.
- Misprints in its root lists beyond the two it shares with [W20] [V; section 2.10]: the A_4 list prints 0010 twice and omits 0011; the D_5 list (p. 27) prints two entries twice in place of two others; the E_6 list (p. 28) prints three entries twice in place of three others; and the E_6 diagram on p. 28 is drawn with vertex 0 above vertex 2, where the lists need it above vertex 3 [S, page image]. The published paper corrects all of these.
- Contents beyond [W20]: the **dual mutation game** (p. 30: Venus, "fundamental weights", the move r_x, Theorem 0.10); the map τ from populations to dual populations, described as measuring the change a mutation would make at each vertex, with matrix $A - 2I$, and the adjointness $p\, s_x \tau = p\, \tau r_x$ (Proposition 0.3, pp. 31-32); increasing and incremental mutations, read off from the sign of the dual population (p. 32); the "canonical increasing sequence" on a bipartite graph (p. 33); the complementation map ψ on sequences of reflections (pp. 38-43); "frames" of mutation sequences, which are heaps in Viennot's sense (pp. 45-46); change sequences and the local sum rule (Proposition 0.8, Theorem 0.13); canonical mutation frames $M(X, x)$ listed for A_3, D_4, D_5, E_6, E_7, E_8 (pp. 49-59); "root spindles" and two statements labelled Observation, given without proof (pp. 60-64); "Perron-Frobenius extensions" (pp. 72-76).
- Role on the site: the fullest account of both games together. Because it is unpublished and unfinished, statements should be cited to [W20] or [W03c] wherever those cover them.

[W03a] N. J. Wildberger, "A combinatorial construction for simply-laced Lie algebras", *Adv. in Appl. Math.* 30 (2003), no. 1-2, 385-396. DOI 10.1016/S0196-8858(02)00541-9. Preprint: https://web.maths.unsw.edu.au/~norman/papers/CombinatorialLieAlgebras.pdf.

- Status: [S] preprint read (the published version is on a host that refuses automated fetching). Section 3 states the mutation game for simple graphs, with the Mars story; this and [W03c] are the first *published* statements of the game. Section 3 also records that every root is non-negative or non-positive and that the author knows no direct combinatorial proof. Section 4 builds a representation on the ideals of a "minuscule heap", with raising and lowering operators X_α, Y_α; Theorem 4.1 asserts that these span the simple Lie algebra with a Chevalley basis. **The theorem is stated without proof**, as R. M. Green also notes in [Gre07]. E_8 is excluded because it has no minuscule representation.
- Role on the site: source and credit for module M8. The site must present the Chevalley-Serre relations for the game-built operators as something the reader can check on the page (and that the reference script checks), with the proof attributed to the later literature (Green's book [Gre13]), since [W03a] gives none.

[W03b] N. J. Wildberger, "Minuscule posets from neighbourly graph sequences", *European J. Combin.* 24 (2003), no. 6, 741-757. DOI 10.1016/S0195-6698(03)00056-8. Preprint: https://web.maths.unsw.edu.au/~norman/papers/MinusculePosetsGraphs.pdf. [S] preprint read. Purely graph-theoretic: classifies the graphs carrying a maximal two-neighbourly heap (A_n, D_n, E_6, E_7) and recovers Proctor's minuscule posets. Neither game appears in it.

[W03c] N. J. Wildberger, "A combinatorial construction of G_2", *J. Lie Theory* 13 (2003), no. 1, 155-165. Free published version: https://emis.univie.ac.at/journals/JLT/vol.13_no.1/wildla.pdf.

- Status: [S] read. Section 2, "The mutation and numbers games on a graph", is the first published definition of **both** games on directed graphs: a mutation at z negates the value at z and adds each neighbour's value once per edge *into* z; a dual mutation at z negates the value at z and adds it to each neighbour once per edge *out of* z; the numbers game allows only positive firings. The numbers game is credited to Mozes. The paper then builds the 7-dimensional representation of $\mathfrak{g}_2$ from a six-step numbers game and proves that the resulting operators span a 14-dimensional Lie algebra (Theorem 4.1).
- Caution: the introduction prints the dimension of F_4 as 56; the correct value is 52.
- Role on the site: the citation for the pair of definitions in section 2, and the worked G_2 example in module M8.

[W-sl3] N. J. Wildberger, "Quarks, diamonds, and representations of sl(3)", preprint (2003), https://web.maths.unsw.edu.au/~norman/papers/Diamonds.pdf. [S] read. Does not use either game. Listed only because the 2018 seminar on A_2 and quarks connects to it.

[ADLMPPW09] L. W. Alverson II, R. G. Donnelly, S. J. Lewis, M. McClard, R. Pervine, R. A. Proctor, N. J. Wildberger, "Distributive lattices defined for representations of rank two semisimple Lie algebras", *SIAM J. Discrete Math.* 23 (2009), 527-559. arXiv:0707.2421. [S] abstract and arXiv text read. Neither game is named in it.

Seminar abstracts (UNSW, 2018) [S] read:

- 4 June 2018, "The curious ubiquity of ADE graphs, and the Numbers and Mutation games": https://www.unsw.edu.au/science/our-schools/maths/engage-with-us/seminars/2018/curious-ubiquity-ade-graphs-and-numbers-and-mutation-games
- 18 June 2018, "Geometrical graph theory: root systems, Weyl groups and quadratic forms": https://www.unsw.edu.au/science/our-schools/maths/engage-with-us/seminars/2018/geometrical-graph-theory-root-systems-weyl-groups-and-quadratic-forms
- 2 July 2018, "ADE graphs and associated lattices, weights and quarks": https://www.unsw.edu.au/science/our-schools/maths/engage-with-us/seminars/2018/ade-graphs-and-associated-lattices-weights-and-quarks. This abstract introduces the numbers game as the dual game "which deals with weights instead of roots", and spells Mozes as "Moses"; the site uses "Mozes".

Video lectures [U unless marked]. Video pages could not be opened by the research tools, so titles and addresses below come from search listings and one third-party README. Every link, date and channel must be checked by a person before it is published.

- "Exceptional structures in mathematics and physics from dynamics on graphs", Insights into Mathematics: https://www.youtube.com/watch?v=peLRDMhWai8. Public introduction to the series.
- "DoG 1: The Mutation Game and a foundational exercise / challenge", Wild Egg Maths: https://www.youtube.com/watch?v=u7cUXZHZHEE. Online by 7 June 2021 [S, from the commit date of a repository that links it]. The series "Dynamics on Graphs" continues in a paid members' section of that channel; the episode list is not known.
- Recordings matching the first two 2018 seminars by title: https://www.youtube.com/watch?v=zPtnEZeuzH0 and https://www.youtube.com/watch?v=FgOGTGhhe48.
- Wildberger's blog reports that both channels were offline from October to December 2024 and that playlists were rebuilt, so older playlist links may be dead: https://njwildberger.com/2024/12/21/insights-into-mathematics-and-wild-egg-maths-are-both-now-back-in-business/ [S].

[vE] J. van Ekeren, "Lie algebras and some games", talk abstract, University of Waikato (supervisor: N. J. Wildberger), undated: https://www.math.auckland.ac.nz/mathwiki/images/f/fe/Lie_Algebras_and_Certain_Games.pdf. [S] abstract read. It describes positions of the mutation and numbers games as roots and weights, moves as raising and lowering operators, and an extension that builds the 26-dimensional representation of $\mathfrak{f}_4$ with integer coefficients. No fuller text was found.

[Gre07] R. M. Green, "Full heaps and representations of affine Kac-Moody algebras", arXiv:math/0605768. [S] read. Describes its constructions as modified versions of Wildberger's raising and lowering construction, and states that Theorem 4.1 of [W03a] appeared without proof. **[Gre08]** R. M. Green, "Full heaps and representations of affine Weyl groups", arXiv:math/0608123; citation only, not read [S2]. **[Gre13]** R. M. Green, *Combinatorics of Minuscule Representations*, Cambridge University Press, 2013. DOI 10.1017/CBO9781139207003. Introduction read [S]; the proofs behind module M8 live here. Journal data for the two papers and the book's series number are [U].

[Vie17] G. X. Viennot (who also signs as Xavier Viennot), *The Art of Bijective Combinatorics*, Part II, Chapter 5a (IMSc Chennai, 2017): https://viennot.org/abjc2-ch5.html. [S] page read: presents neighbourly heaps and "Wildberger's theorem" for simply laced Lie algebras, with slides and video.

Existing software and interactive work on the mutation game (details in Appendix F):

- `mutation-game`, a Python package by Andrea Cortis, MIT licence, versions 0.1.0 to 0.1.5 released 7 to 13 April 2026: https://pypi.org/project/mutation-game/, https://github.com/andreacortis/mutation_game, https://mutation-game.readthedocs.io/en/latest/. [S] source and documentation read. Its mutation rule agrees with Wildberger's. Several statements in its documentation are wrong and must not be copied (Appendix F lists them).
- GEOMETOR "Mutation Explorer" (2021, MIT, D3): https://geometor.github.io/graphs/, source https://github.com/geometor/graphs. [S] source read. Simple graphs only; random or round-robin mutations with a plot of the population total; no root sets.
- The Java applet "ADE Concurrent Game" on Wildberger's UNSW pages, in a directory named for Thai Son Hoang: https://web.maths.unsw.edu.au/~norman/ThaiSonHoang/numbers/ADEConcurrentGame.pdf. [S] documentation read, twice. It names no author and is dated "November 25, 2004", which is also the metadata date of the manuscript PDF; the two dates were read separately from the two files. A simultaneous-firing variant of the numbers game on A, D, E graphs, with the extra vertex of a D or E graph called V_0 and joined to V_2 or V_3, as in Wildberger's lists. Java applets do not run in current browsers.

11.2 The numbers game (Mozes) and its literature

Sign convention in each entry: "fires negative" means the source's legal move is at a negative number, so its positions are the negatives of this specification's positions (section 2.9 gives the full translation table).

[IMO86] 27th International Mathematical Olympiad (Warsaw, 1986), Problem 3. Official sheet: https://www.imo-official.org/assets/documents/problems/1986/1986_eng.pdf. Solutions: https://artofproblemsolving.com/wiki/index.php/1986_IMO_Problems/Problem_3, https://www.cut-the-knot.org/Curriculum/Algebra/IMO1986_3.shtml.

- Status: [S] statement read on the official sheet. Five integers with positive sum sit on a pentagon; a negative middle number y in a consecutive triple x, y, z may be replaced by $x + y, -y, z + y$; the task is to decide whether the process must stop (it must). Fires negative.
- Role on the site: a puzzle in module M4, played in tool T15. In this specification's language it is the numbers game on the 5-cycle, which is the affine board $\tilde{A}_4$, started at negative level. The problem prescribes no particular numbers; the site's example start is its own.
- Open: the proposer is not named in any source that could be read [U].

[Moz90] S. Mozes, "Reflection processes on graphs and Weyl groups", *J. Combin. Theory Ser. A* 53 (1990), no. 1, 128-142. DOI 10.1016/0097-3165(90)90024-Q. Abstract and record: https://cris.huji.ac.il/en/publications/reflection-processes-on-graphs-and-weyl-groups/.

- Access: free to read in Elsevier's open archive (Elsevier user licence, which is not a Creative Commons licence). The host refuses automated fetching, so the text was **not read** for this specification.
- Status: [S] bibliographic data and abstract. [S2] everything else: later authors (Donnelly 2008; Gashi, Schedler and Speyer 2012; Bugaenko et al. 2013) report that the paper introduces the game as a generalisation of the pentagon problem, works with integer weights coming from a symmetrizable generalized Cartan matrix, fires negative numbers, proves that the length and the end position of a terminating game do not depend on the play, and describes the terminating starts through the Weyl group of a Kac-Moody algebra. Theorem numbers, the index convention of his matrix, and whether the text uses the phrase "numbers game" are unknown [U].
- Action for the build: a human should open the paper once (it is 15 pages) and fill in theorem numbers before the history page is published.

[AKP89] N. Alon, I. Krasikov, Y. Peres, "Reflection sequences", *Amer. Math. Monthly* 96 (1989), no. 9, 820-823. DOI 10.1080/00029890.1989.11972287. Scan on Alon's page: https://www.tau.ac.il/~nogaa/PDFS/Publications/Reflection%20sequences.pdf.

- Status: [S] citation. [S2] content (the game on an n-cycle; the game length equals the number of negative arc sums). The free copy is an image scan without a text layer.

[Bjo88] A. Björner, "On a combinatorial game of S. Mozes", preprint, 1988. Unpublished; no public copy was found. Cited in this form by Proctor, Donnelly, Donnelly and Eriksson, and Wildberger (2020). Eriksson credits Björner with the notion of strong convergence [S].

[Eri92] K. Eriksson, "Convergence of Mozes's game of numbers", *Linear Algebra Appl.* 166 (1992), 151-165. DOI 10.1016/0024-3795(92)90274-E. Citation [S]; content [S2]: a graph-theoretic treatment of the unweighted game, including an A-D-E classification of the graphs on which every game ends. Not read (same host restriction as [Moz90]).

[Eri93] K. Eriksson, *Strongly convergent games and Coxeter groups*, PhD thesis, KTH Stockholm, 1993. Record with abstract: https://www.proquest.com/docview/304095210. Not online. Most later statements point here: Donnelly and Eriksson cite its Theorem 3.6 (strong convergence), Theorem 4.3 (comparison), Theorems 6.5 and 6.7 (classification) [S2].

[Eri94a] K. Eriksson, "Node firing games on graphs", in *Jerusalem Combinatorics '93*, Contemp. Math. 178, Amer. Math. Soc., 1994, 117-127. Citation [S2] (page range 117-127 as printed by Donnelly and Eriksson; two other papers print 117-128). The cited source for the affine termination and looping criteria.

[Eri94b] K. Eriksson, "Reachability is decidable in the numbers game", *Theoret. Comput. Sci.* 131 (1994), no. 2, 431-439. DOI 10.1016/0304-3975(94)90182-1. Citation only, not read [S2].

[Eri95a] K. Eriksson, "The numbers game and Coxeter groups", *Discrete Math.* 139 (1995), 155-166. DOI 10.1016/0012-365X(94)00131-2. Citation [S]; content [S2] through Donnelly: Propositions 4.1 and 4.2 (legal

play sequences are reduced words), Theorem 4.5 (comparison), section 4 (Tits cone). First presented at the 1992 Montréal conference, which is the earliest documented title containing the words "numbers game" [S]. Donnelly reports one error just before Proposition 4.4 that concerns asymmetric real weights only.

[Eri95b] K. Eriksson, "Polygon posets and the weak order of Coxeter groups", *J. Algebraic Combin.* 4 (1995), 233-252. DOI 10.1023/A:1022428111696. Free full text: https://emis.univie.ac.at/journals/JACO/Volume4_3/twlx522043172711.fulltext.pdf.

- Status: [S] **read**. Definition 1.1 (strong convergence) and Theorem 1 (the polygon property: a game is strongly convergent exactly when any two legal moves at a position can be continued to a common position in the same number of steps, or both can be continued forever), pp. 233-234; the numbers game with real weights in section 10. Fires negative.
- Role on the site: the source to cite for strong convergence, since it is the one Eriksson paper that is freely readable.

[Eri96a] K. Eriksson, "Strong convergence and a game of numbers", *European J. Combin.* 17 (1996), no. 4, 379-390. DOI 10.1006/eujc.1996.0031. Citation and abstract [S]; Theorem 3.1 is the usual pointer for the strong convergence theorem [S2].

[Eri96b] K. Eriksson, "Strong convergence and the polygon property of 1-player games", *Discrete Math.* 153 (1996), 105-122. DOI 10.1016/0012-365X(95)00131-F. Citation only, not read [S2].

[HEri94] H. Eriksson, *Computational and combinatorial aspects of Coxeter groups*, PhD thesis, KTH Stockholm, 1994. https://www.csc.kth.se/~viggo/henrik/he/thesis.pdf. [S] first 50 pages read: a compact account (section 3) with the statement that positions represent group elements faithfully and that reduced words are exactly the plays firing only negative numbers, which solves the word problem in time linear in the word length.

[BB05] A. Björner, F. Brenti, *Combinatorics of Coxeter Groups*, Graduate Texts in Mathematics 231, Springer, 2005. DOI 10.1007/3-540-27596-7. Section 4.3 "The numbers game" starts on p. 97 [S, table of contents].

- Status: section 4.3 itself was not read. Its definitions and main theorem are restated in [BCNS13, Theorem 3.3] [S2]: from a start with all entries positive, (1) two play sequences reach the same position exactly when their words are equal in the Coxeter group, (2) group elements correspond bijectively to reachable positions, (3) the descents of the element are the nodes carrying a negative number, (4) a play sequence fires only positive numbers exactly when its word is reduced. The book defines both positive and negative moves. The theorem number (believed to be 4.3.1) is [U].
- Role on the site: the standard textbook citation for module M5 and tool T8.

[Pro84] R. A. Proctor, "Bruhat lattices, plane partition generating functions, and minuscule representations", *European J. Combin.* 5 (1984), no. 4, 331-350. DOI 10.1016/S0195-6698(84)80037-2. Citation [S]; content [S2] (Donnelly): uses the firing process, with **positive** numbers fired, to compute Weyl group orbits of weights in fundamental-weight coordinates. This predates Mozes and is the origin of the sign convention used in this specification.

[Pro99a] R. A. Proctor, "Minuscule elements of Weyl groups, the numbers game, and d-complete posets", *J. Algebra* 213 (1999), 272-303. DOI 10.1006/jabr.1998.7648. Abstract read at https://proctor.math.unc.edu/Abstracts.html [S]; text not read.

[Pro99b] R. A. Proctor, "Dynkin diagram classification of λ-minuscule Bruhat lattices and of d-complete posets", *J. Algebraic Combin.* 9 (1999), 61-94. DOI 10.1023/A:1018615115006. Free: https://emis.univie.ac.at/journals/JACO/Volume9_1/j48700v65753066w.fulltext.pdf. [S] read: the restricted game (only nodes labelled -1 are fired, in a fire-negative convention) and the classification of d-complete posets into 15 classes.

[Ste01] J. R. Stembridge, "Minuscule elements of Weyl groups", *J. Algebra* 235 (2001), 722-743. Preprint: https://websites.umich.edu/~jrs/papers/FCminus.pdf [S] read. Peterson's λ-minuscule elements, characterised by reduced words and heaps; multiply laced types.

[**Don08**] R. G. Donnelly, "Eriksson's numbers game and finite Coxeter groups", *European J. Combin.* 29 (2008), no. 7, 1764-1781. DOI 10.1016/j.ejc.2007.06.029. Free: https://arxiv.org/abs/0710.0173.

- Status: [S] read (arXiv version). Fires **positive** numbers, following Proctor. Theorem 2.1 (strong convergence), Theorem 2.3 (comparison), Theorem 2.8 (legal sequences are reduced words; attributed to Eriksson), Remark 2.13 and section 5 (the fired number at each step is the pairing of the start with a positive root; Theorem 5.2: every positive root appears exactly once along a reduced word of the longest element when there is no "odd asymmetry", which integer weights never have), Proposition 3.2 and Corollaries 3.3, 3.4 (starts with zeros and parabolic quotients), Theorem 2.15 (Tits cone criterion, attributed to Eriksson), Theorems 6.1, 6.2 (classification). Mentions Wildberger's mutation game as the dual game and thanks him.
- Convention: his matrix M is the transpose of the matrix C used here ($C = M^{\mathsf{T}}$).
- Role on the site: the most useful single reference for modules M4 and M5, because it is free, uses the same sign convention as the site, and states the fired-numbers property.

[**DE08**] R. G. Donnelly, K. Eriksson, "The numbers game and Dynkin diagram classification results", arXiv:0810.5371 (2008). [S] read. No journal version was found.

- Theorem 2.1: a connected integer-weighted graph has a terminating game from some nonzero dominant start exactly when it is a Dynkin diagram of finite type, and then every game from every start terminates. The proof uses Perron-Frobenius theory and avoids the classification of finite Coxeter groups. Lemmas 4.3 to 4.7 give the spectral-radius criterion. Section 7 notes that Wildberger re-derives the classification in his preprint.
- Note: an earlier draft of this project's brief attached the arXiv number math/0610702 to this paper. That number belongs to [Don06].

[**Don06**] R. G. Donnelly, "The numbers game, geometric representations of Coxeter groups, and Dynkin diagram classification results", arXiv:math/0610702 (2006). [S] read. Its Theorem 1.2 is labelled "(Wildberger)".

[**Don07a**] R. G. Donnelly, "Eriksson's numbers game on certain edge-weighted three-node cyclic graphs", arXiv:0708.0880. [**Don07b**] R. G. Donnelly, "Roots of asymmetric geometric representations of Coxeter groups", arXiv:0707.3310 (later title "Root systems for asymmetric geometric representations of Coxeter groups"; journal data unverified [U]). [**Don08b**] R. G. Donnelly, "Convergent and divergent numbers games for certain collections of edge-weighted graphs", arXiv:0810.5362. All [S] read. Donnelly's annotated reading list: https://campus.murraystate.edu/academic/faculty/rdonnelly/CoxeterGroupsSeminar/references.htm.

[**GS11**] Q. R. Gashi, T. Schedler, "On dominance and minuscule Weyl group elements", *J. Algebraic Combin.* 33 (2011), 383-399. DOI 10.1007/s10801-010-0248-2. Free: https://arxiv.org/abs/0908.1091. [S] read. The numbers game "with a cutoff" on Dynkin and extended Dynkin diagrams. Fires negative.

[**GSS12**] Q. R. Gashi, T. Schedler, D. E. Speyer, "Looping of the numbers game and the alcoved hypercube", *J. Combin. Theory Ser. A* 119 (2012), no. 3, 713-730. DOI 10.1016/j.jcta.2011.11.012. Free: https://arxiv.org/abs/0909.5324.

- Status: [S] read. Theorem 1.2 collects the facts the affine module needs (in their fire-negative convention, with δ spanning the kernel of the Cartan form): the game on an extended Dynkin graph terminates exactly when $\delta \cdot v > 0$; when $\delta \cdot v = 0$ it visits finitely many positions and loops; otherwise it visits infinitely many. Theorem 3.1: in a looping orbit, legal play can reach every position from every other.
- Scope: mainly simply laced extended diagrams, with other types reached by folding. The statement for non-simply-laced and twisted affine boards used in section 2.12 is this specification's own formulation, checked by random play on 42 boards [V].

[**BLS91**] A. Björner, L. Lovász, P. W. Shor, "Chip-firing games on graphs", *European J. Combin.* 12 (1991), 283-291. DOI 10.1016/S0195-6698(13)80111-4; citation only, not read [S2]. [**BL92**] A. Björner, L. Lovász, "Chip-firing games on directed graphs", *J. Algebraic Combin.* 1 (1992), 305-328, free at https://emis.univie.ac.at/journals/JACO/Volume1_4/l31v67681vv718ll.fulltext.pdf [S] read. For the "how is this

different from chip-firing" sidebar.

[BKR16] G. Benkart, C. Klivans, V. Reiner, "Chip firing on Dynkin diagrams and McKay quivers", arXiv:1601.06849. [S] read: Remark 4.11 compares the two games and notes that they agree after the substitution $v = 1 - u$ when only numbers equal to -1 are fired (the minuscule situation). Its firing rule has the same shape as that of [GSS12], so the same translation applies (section 2.9).

[BCNS13] V. Bugaenko, Y. Cherniavsky, T. Nagnibeda, R. Shwartz, "Weighted Coxeter graphs and generalized geometric representations of Coxeter groups", arXiv:1304.0692. [S] read: restates the Björner-Brenti definitions and main theorem (its Theorem 3.3).

Expository. M. Baker, "The Pentagon Problem" (2014), https://mattbaker.blog/2014/02/25/the-pentagon-problem/, and "Firing games and greedoid languages" (2021), https://mattbaker.blog/2021/12/03/firing-games-and-greedoid-languages/ [S] read. C. Ji, R. Park, A. Song, "Combinatorial games of no strategy" (MIT PRIMES 2015), https://math.mit.edu/research/highschool/primes/materials/2015/Ji-Park-Song.pdf [S] read.

Existing software for the numbers game. One Sage repository, `adn1995/numbers-game` (GPL-3.0, described by its author as incomplete): https://github.com/adn1995/numbers-game. SageMath itself has no function named for the game; `simple_reflection(i)` on a weight-lattice element is the move. A Java-era interactive page for the pentagon game exists at cut-the-knot.

11.3 Related games and structures

Kostant's game (find the highest root). No publication by Kostant describing the game was found. The written record starts with teaching notes, and the attribution is by oral tradition; the site should say "attributed to Bertram Kostant, popularised by Allen Knutson and by Alexander Postnikov's 2017 MIT course".

- **[Ele16]** B. Elek, *Reflection Groups* (notes for the Ithaca High School Senior Math Seminar, 2016), section 5.4 "Kostant's find the highest root game": https://pi.math.cornell.edu/~bazse/reflection_groups.pdf. [S] read. Vocabulary: unhappy, happy, manic. Credits Allen Knutson. Proves the A-D-E classification through the affine diagrams.
- **[Che17]** E. Chen, notes of A. Postnikov's MIT course 18.218 (Spring 2017), Lecture 1 "Bert Kostant's game": https://web.evanchen.cc/notes/MIT-18-218.pdf; course page https://math.mit.edu/~apost/courses/18.218_2017/. [S] read (first half). Proposition 1.3 (order independence), Theorems 4.7 and 4.9 (finite exactly on A-D-E; equivalent conditions), Corollary 8.5 (generalized Cartan matrices), section 12.1 (a reduced word lists the inversions).
- **[CPS22]** A. Caviedes Castro, M. Pabiniak, S. Sabatini, arXiv:2206.00456, section 3.3. [S] read. The multiply laced version, with two possible endings.
- A. Knutson's MathOverflow answer in thread 171999 is cited by others as a source; it could not be opened [S2].
- Facts used on the site and checked by the reference script [V]: on A-D-E boards every play from a single simple root takes exactly $h - 2$ moves and ends at the highest root; on B, C, F, G it ends at the highest root from a long node and at the highest short root from a short node; and the final numbers are the road counts by which a new city makes the affine board, untwisted from a long start and twisted from a short one (section 2.12).

Quiver representations (module M8, sidebar).

- **[Gab72]** P. Gabriel, "Unzerlegbare Darstellungen I", *Manuscripta Math.* 6 (1972), 71-103. DOI 10.1007/BF01298413. Citation [S].
- **[BGP73]** I. N. Bernstein, I. M. Gel'fand, V. A. Ponomarev, "Coxeter functors and Gabriel's theorem", *Russian Math. Surveys* 28:2 (1973), 17-32. DOI 10.1070/RM1973v028n02ABEH001526. Free scan: https://www-users.cse.umn.edu/~reiner/ReadingSeminars/McKaySeminar2016/BGP.pdf. [S] read. Theorem 1.1: a reflection functor at a sink changes the dimension vector only at that vertex, to minus the old value plus the sum over the arrows into it, which is the mutation rule. Theorem 3.1 (Gabriel's theorem): on an A-D-E quiver the indecomposables correspond to the positive roots.

- **[Kac80]** V. G. Kac, "Infinite root systems, representations of graphs and invariant theory", *Invent. Math.* 56 (1980), 57-92. DOI 10.1007/BF01403155. Citation [S]; statement of Kac's theorem taken from W. Crawley-Boevey's lecture notes, https://www.math.uni-bielefeld.de/~wcrawley/dmvlecs.pdf [S].
- **[EGH+11]** P. Etingof et al., *Introduction to Representation Theory*, arXiv:0901.0827; in the MIT OpenCourseWare numbering, sections 5.4 to 5.8 (Theorem 5.23 is Gabriel's theorem; Proposition 5.31 is the dimension-vector rule). [S] read. The best free exposition to link.
- Caution for the text: a quiver arrow is an orientation, and a quiver only ever gives a symmetric Cartan matrix; Wildberger's unequal edge counts encode long and short roots. The two uses of arrows must not be merged.

Cluster algebras (the other "mutation"; section 2.16).

- **[FZ02]** S. Fomin, A. Zelevinsky, "Cluster algebras I: Foundations", *J. Amer. Math. Soc.* 15 (2002), 497-529. DOI 10.1090/S0894-0347-01-00385-X. arXiv:math/0104151. [S] read: Definition 4.2 and equation (4.3) give matrix mutation.
- **[FZ03a]** S. Fomin, A. Zelevinsky, "Cluster algebras II: Finite type classification", *Invent. Math.* 154 (2003), 63-121. DOI 10.1007/s00222-003-0302-y. arXiv:math/0208229. [S] read: Theorem 1.4 (finite type corresponds to finite-type Cartan matrices), Theorem 1.9 (cluster variables correspond to almost positive roots).
- **[FZ03b]** S. Fomin, A. Zelevinsky, "Y-systems and generalized associahedra", *Ann. of Math.* 158 (2003), 977-1018. DOI 10.4007/annals.2003.158.977. arXiv:hep-th/0111053. [S] read: the piecewise-linear maps $\tau_{\pm}$ built from a bipartite colouring.
- **[Kel08]** B. Keller, "Cluster algebras, quiver representations and triangulated categories", arXiv:0807.1960, section 3.2 (quiver mutation in words) [S]. Keller's quiver mutation applet (JavaScript and Java): https://webusers.imj-prg.fr/~bernhard.keller/quivermutation/ [S, page loaded; not run].

Heaps (module M5).

- **[Vie86]** G. X. Viennot, "Heaps of pieces, I: Basic definitions and combinatorial lemmas", in *Combinatoire énumérative*, Lecture Notes in Math. 1234, Springer, 1986, 321-350. DOI 10.1007/BFb0072524. Citation and abstract [S].
- **[Ste96]** J. R. Stembridge, "On the fully commutative elements of Coxeter groups", *J. Algebraic Combin.* 5 (1996), 353-385. DOI 10.1023/A:1022452717148. Free: https://websites.umich.edu/~jrs/papers/FC.pdf. [S] read: definition of the heap of a word (sections 2.1, 2.2), Proposition 2.2 (linear extensions are the words of the commutation class; the oracle function `commutation_class_size` counts them, and the count was checked against explicit enumeration [V]), Theorem 3.2 (fully commutative elements and distributive lattices).
- Dictionary to state on the site: Wildberger's "X-string" is a commutation class, his "frame" is the heap, a "free switch" is a commutation, and an "incremental frame" is a minuscule heap.

Reduced words (module M5).

- **[Sta84]** R. P. Stanley, "On the number of reduced decompositions of elements of Coxeter groups", *European J. Combin.* 5 (1984), 359-372. Citation [S, author's list]; the count for the symmetric group (1, 1, 2, 16, 768, 292864, ...) is OEIS A005118 [S].
- **[BH95]** S. Billey, M. Haiman, "Schubert polynomials for the classical groups", *J. Amer. Math. Soc.* 8 (1995), 443-482: https://math.berkeley.edu/~mhaiman/ftp/schubert/schubert.pdf. [S] read: Proposition 3.3 (type B reduced words of the longest element and shifted staircase tableaux). Counts 2, 42, 24024 are OEIS A039622 [S]. The script confirms that the number of complete games on B_n and C_n equals the number of standard Young tableaux of the $n \times n$ square for $n \leq 6$, and on A_n that of the staircase shape for $n \leq 7$ [V]; who first proved the square-shape count is [U] (Appendix E item 16).
- The counts for D_4 (2316), D_5 (12985968) and F_4 (2144892) were computed and are asserted by the script [V]; no published value was located.

Kac-Moody background.

- **[Kac90]** V. G. Kac, *Infinite-Dimensional Lie Algebras*, 3rd ed., Cambridge University Press, 1990. **Not opened in this research.** Everything attributed to it in this specification (symmetrizable matrices and Exercise 2.1, the Tits cone, the classification into finite, affine and indefinite type, marks and comarks, the statement that real roots are positive or negative) is standard, and its mathematical content was re-derived or checked by computation, but section and proposition numbers are [U] and must be looked up before they are printed on the site. One item has second-hand confirmation: Fomin and Zelevinsky cite Exercise 2.1 for the cycle-product criterion of symmetrizability [S2].

11.4 Real forms and Satake diagrams

Access warning for this whole subsection. The classical tables themselves (Helgason's table on pp. 532-534, Araki's on pp. 32-33, Onishchik and Vinberg's Table 9, Knapp's Appendix C) could not be opened by the research tools. The catalogue in section 6.4 was instead checked four ways: against a paper that states every diagram in words [DM22]; against the Satake codes in the source of the LaTeX package `dynkin-diagrams`, whose table reproduces Helgason's; by two independent computations; and by exhaustive enumeration of admissible pairs (section 2.15). The first two confirm black nodes and arrows; the multiplicities rest on the computations and on the partial published values listed below. A person should compare section 6.4 with Helgason's table before the catalogue is published; Appendix B is laid out for that comparison.

[Car14] É. Cartan, "Les groupes réels simples, finis et continus", *Ann. Sci. École Norm. Sup.* (3) 31 (1914), 263-355. First classification of the real simple Lie algebras. **[Car26]** É. Cartan, "Sur une classe remarquable d'espaces de Riemann", *Bull. Soc. Math. France* 54 (1926), 214-264 and 55 (1927), 114-134. Classification of Riemannian symmetric spaces and source of the labels A I to G. Citations and page ranges are [U] (the host could not be fetched); the historical role is [S2].

[Sat60] I. Satake, "On representations and compactifications of symmetric Riemannian spaces", *Ann. of Math.* (2) 71 (1960), 77-110. DOI 10.2307/1969880 (paywalled). The diagrams with black nodes, white nodes and arrows are reported to appear on p. 109 [S2]. Text not read.

[Ara62] S. Araki, "On root systems and an infinitesimal classification of irreducible symmetric spaces", *J. Math. Osaka City Univ.* 13 (1962), 1-34. Project Euclid: https://projecteuclid.org/journals/osaka-journal-of-mathematics/volume-13/issue-1/On-root-systems-and-an-infinitesimal-classification-of-irreducible-symmetric/ojm/1353055009.full (page exists; whether it is open access is [U]). Text not read. Three later papers cite its table of diagrams with restricted-root multiplicities as pp. 32-33 [S2]; Bulois cites section 2.14 for the theorem that the diagram determines the symmetric pair [S2].

[Tit66] J. Tits, "Classification of algebraic semisimple groups", in *Algebraic Groups and Discontinuous Subgroups*, Proc. Sympos. Pure Math. 9, Amer. Math. Soc., 1966, 33-62. DOI 10.1090/pspum/009/0224710. The index of a semisimple group over a general field; over the reals it carries the same information as the Satake diagram [S2]. Text not read.

[Hel01] S. Helgason, *Differential Geometry, Lie Groups, and Symmetric Spaces*, Graduate Studies in Mathematics 34, Amer. Math. Soc., 2001. DOI 10.1090/gsm/034. The standard table of Satake diagrams with multiplicities is on pp. 532-534 [S2, from the manual of the `dynkin-diagrams` package]. Not read. The table number ("Table VI of Chapter X") is [U].

[Kna02] A. W. Knapp, *Lie Groups Beyond an Introduction*, 2nd ed., Birkhäuser, 2002; free digital edition (2023): https://www.math.stonybrook.edu/~aknapp/download/Beyond2.pdf. [S] table of contents read: Chapter VI section 8 "Vogan diagrams" (p. 397), section 10 classification (p. 408), section 11 restricted roots in the classification (p. 422); Appendix C lists every real form (classical p. 693, exceptional p. 706). Knapp classifies with Vogan diagrams, so this is the free reference for the "other diagram" sidebar and for the per-form data sheets.

[OV90] A. L. Onishchik, E. B. Vinberg, *Lie Groups and Algebraic Groups*, Springer, 1990 (reference chapter, Table 9) [S2]. **[Oni04]** A. L. Onishchik, *Lectures on Real Semisimple Lie Algebras and Their Representations*, EMS, 2004 (appendix on Satake diagrams, pp. 73-76; tables pp. 77-82 [S, contents page]). **[Bum13]** D. Bump, *Lie Groups*, 2nd ed., Graduate Texts in Mathematics 225, Springer, 2013, chapters 28 "Symmetric Spaces" and 29 "Relative Root Systems" [S, chapter pages]. None of the three was read.

[Kol14] S. Kolb, "Quantum symmetric Kac-Moody pairs", *Adv. Math.* 267 (2014), 395-469. DOI 10.1016/j.aim.2014.08.010. arXiv:1207.6036.

- Status: [S] arXiv version read through a text extraction; **check formula numbers against the PDF before quoting them**. Definition 2.3 (admissible pair (X,τ): $\tau^2=\mathrm{id}$ and $\tau(X)=X$; τ acts on X as $-w_X$; for each white node j fixed by τ the number $\alpha_j(\rho_X^\vee)$ is an integer). Equation (2.10): on the root lattice the involution is $\Theta=-w_X\circ\tau$. Example 2.4 works the case of $\mathfrak{sl}_4$. Theorem 2.7: admissible pairs up to diagram automorphism correspond to conjugacy classes of involutions of the second kind. A remark after the definition says that for finite-dimensional simple Lie algebras the admissible pairs are the diagrams of Araki's table, and calls the check an instructive exercise.
- Role on the site: the definition behind the Satake Studio (T13). The site's "is this a real form?" checker is Kolb's three conditions, and the exhaustive enumeration in the reference script is the "instructive exercise" carried out by machine for every type of rank at most 8 [V]. What a failed third condition means is worked out for A_2 in section 2.15, with a matrix computation in the script [V].

[DK19] L. Dobson, S. Kolb, "Factorisation of quasi K-matrices for quantum symmetric pairs", *Selecta Math.* 25 (2019), article 63. arXiv:1804.02912. [S] read (same caveat about formula numbers): restricted roots as $(\alpha-\Theta\alpha)/2$ with multiplicity equal to the number of roots restricting to them; the restricted Weyl group generated by $\tilde{s}_i=w_{X\cup\{i,\tau(i)\}}\,w_X^{-1}$ (equation (2.8), Theorem 2.4, credited to Lusztig); Table 1 lists the rank-one Satake diagrams. This is the source for the "restricted game" of section 2.15.

[RV20] V. Regelskis, B. Vlaar, "Quasitriangular coideal subalgebras of $U_q(\mathfrak{g})$ in terms of generalized Satake diagrams", *Bull. London Math. Soc.* 52 (2020). DOI 10.1112/blms.12360. arXiv:1807.02388. **[RV16]** "Reflection matrices, coideal subalgebras and generalized Satake diagrams of affine type", arXiv:1602.08471. **[RV21]** "Pseudo-symmetric pairs for Kac-Moody algebras", arXiv:2108.00260. [S] read. They weaken Kolb's third condition to: the connected component of $X\cup\{j\}$ containing a τ-fixed white node j is not of type A_2. The larger family was first considered by Heck. For the site: an optional "advanced" switch in T13, with a warning that these diagrams need not give a restricted root system; the two generalized diagrams on G_2 are computed in the script as examples [V].

[Hec84] A. Heck, "Involutive automorphisms of root systems", *J. Math. Soc. Japan* 36 (1984), 643-658. Citation [U]; role [S2].

[DM22] C. Draper, J. Meulewaeter, "Inner ideals of real simple Lie algebras", arXiv:2202.09351. [S] read. It states the white-node set and arrows of every Satake diagram **in words**, which made it the one published source against which the black-node sets of section 6.4 could be checked line by line.

Other sources used to check the catalogue [S, all read as text]: A. Keurentjes, "The group theory of oxidation II", arXiv:hep-th/0212024 (maximal compact subgroup, character, real rank); M. Rösler, arXiv:0907.2447 (multiplicities for the Grassmannian families); S. Klein, arXiv:0809.1319 and arXiv:0801.4127 (E III, $\mathfrak{so}^*(10)$, $\mathfrak{so}(2,8)$; an algorithm that rebuilds the symmetric space from the diagram); J. Berndt, H. Tamaru, arXiv:math/0505490 (rank-one multiplicities, including $(8,7)$ for F II); M. Bulois, arXiv:1009.0688; the six-page "List of Satake and Vogan Diagrams of Real Simple Lie Algebras" at https://www.math.uni-duesseldorf.de/~kammeyer/satake-vogan.pdf; K. De Commer, S. Neshveyev, L. Tuset, M. Yamashita, arXiv:1712.08047 (a clean restatement of Kolb's definition).

Errors found in commonly used references, so that the site does not copy them [V for the corrections]:

- Wikipedia, "Satake diagram", symmetric-pair column: the row for A II gives the subalgebra as $\mathfrak{sp}(2p)$ where $\mathfrak{sp}(2p+2)$ is meant, and the row for C I gives $\mathfrak{gl}(2n)$ where $\mathfrak{gl}(n)$ is meant.

Two usages of the word "inner" [V for the two lists; `verify.py` section 21]. (i) A real form is called inner when its Cartan involution is an inner automorphism, which on the roots means that θ lies in the Weyl group. This happens exactly when τ equals $-w_0$ as a symmetry of the board [V on all 167 records]. For E_6 these forms are the compact form, E II and E III. (ii) A real form is called inner, or of type ${}^1E_6$, when it is an inner twist of the split form, which is the case of trivial τ. For E_6 these are E I and E IV only; the compact form is of type ${}^2E_6$, because $-w_0$ is the flip. The τ of this specification is usage (ii). Wikipedia,

"E6 (mathematics)", was recorded during this research as describing the compact form as inner; the page was not read again when this paragraph was revised, and two reviewers of this specification disagreed about which usage it follows [U]. Under usage (ii) the compact form does not belong with E I and E IV. The site should say which usage it means wherever it uses the word, and should not copy the grouping from that page without re-reading it.

Software.

- **CoReLG** (GAP package, version 1.57 of 7 July 2024; H. Dietrich, P. Faccin, W. de Graaf): https://gap-packages.github.io/corelg/. [S] manual and source read. Functions `SatakeDiagram` and `VoganDiagram`. Usable as an external oracle, with three cautions: it records τ only on white nodes (black nodes are shown as fixed), it uses GAP's node numbering, and for E_7 its index 3 is E VII and index 4 is E VI.
- **Atlas of Lie Groups and Representations**: https://www.liegroups.org. [S] scripts read: `restricted_roots.at` computes restricted roots without multiplicities; no Satake diagram function was found.
- **SageMath**: no Satake diagram or real-form classification was found in the root-system modules read [S]; that none exists anywhere in Sage is an inference [U].
- `dynkin-diagrams` (LaTeX, B. McKay; the current release on CTAN is version 3.14159265358979323846 of 4 December 2024): https://ctan.org/pkg/dynkin-diagrams. [S] manual read; the source of the release dated 2 June 2023 was read from a local TeX installation. It draws all 28 Satake diagrams by name, for example `\dynkin E{VI}`, and its manual says the table follows Helgason pp. 532-534. Its internal mark strings (for instance `o*oo*o*` for E VI, `oo***o` with a fold for E III, `*o*.o*.**` for C II) agree with section 6.4 for every one of the 28 patterns. Explicit mark strings such as `\dynkin[involutions={16}] E{oo***o}` were compiled and inspected by hand [S; not a check of `verify.py`]; the site's LaTeX export targets this syntax.
- No interactive web tool for Satake or Vogan diagrams was found. The search was cut short, so this is weak evidence [U].

11.5 Standard Lie theory, Coxeter groups, polytopes and the A-D-E pattern

Conventions. The decisive fact for the build is which index of the Cartan matrix carries the coroot. SageMath and Kac use $C_{ij} = \langle \alpha_i^\vee, \alpha_j \rangle$, as this specification does. Bourbaki and Humphreys print the transpose. SageMath follows Bourbaki's node numbering; Kac numbers the E types and G_2 differently.

- **SageMath** documentation and source [S, read; matrices also obtained by running Sage]: https://doc.sagemath.org/html/en/thematic_tutorials/lie/lie_basics.html, https://doc.sagemath.org/html/en/reference/combinat/sage/combinat/root_system/cartan_type.html, https://doc.sagemath.org/html/en/reference/combinat/sage/combinat/root_system/cartan_matrix.html. `CartanMatrix(['B',2]) = [[2,-1],[-2,2]]`, `['C',2] = [[2,-2],[-1,2]]`, `['G',2] = [[2,-3],[-1,2]]`, `['F',4] = [[2,-1,0,0],[-1,2,-1,0],[0,-2,2,-1],[0,0,-1,2]]`. These equal the matrices in `golden_data.json`, so **SageMath is an independent oracle that needs no translation**.
- **[Bou68]** N. Bourbaki, *Groupes et algèbres de Lie*, chapitres 4 à 6, Hermann, 1968 (English: *Lie Groups and Lie Algebras, Chapters 4-6*, Springer, 2002). Plates I to IX are the reference for node numbering. Not opened. That its matrices are the transposes of those used here is [S2] (SageMath documentation; the Lean mathlib file https://github.com/leanprover-community/mathlib4/blob/master/Mathlib/LinearAlgebra/Matrix/Cartan/Basic.lean, which cites the plates by page).
- **[Hum72]** J. E. Humphreys, *Introduction to Lie Algebras and Representation Theory*, Graduate Texts in Mathematics 9, Springer, 1972. DOI 10.1007/978-1-4612-6398-2. Not opened; all section numbers quoted from it are [U] (root strings 9.4, Weyl's dimension formula 24.3, the lift $\exp(\operatorname{ad} x)\exp(\operatorname{ad}(-y))\exp(\operatorname{ad} x)$ in 14.3, weights of a module 21.2 and 21.3, the list of minuscule weights in Exercise 13.13).
- **[Hum90]** J. E. Humphreys, *Reflection Groups and Coxeter Groups*, Cambridge University Press, 1990. [S] sections read: 1.7 (along a reduced expression the roots β_i are exactly the positive roots that w makes negative), 1.8 (longest element), Theorem 1.12 (stabiliser of a point of the closed chamber);

titles and pages of 3.16 “Coxeter elements”, 3.17 “Action on a plane”, 3.18 to 3.20 (Coxeter number, eigenvalues, exponents and degrees).

- **[FH91]** W. Fulton, J. Harris, *Representation Theory: A First Course*, Graduate Texts in Mathematics 129, Springer, 1991. Not opened; its Cartan matrix convention could not be settled and must not be cited for conventions [U].

Exponents, heights and the Coxeter element (modules M5 and M7).

- **[Ste59]** R. Steinberg, “Finite reflection groups”, *Trans. Amer. Math. Soc.* 91 (1959), 493-504. DOI 10.1090/S0002-9947-1959-0106428-2. Free at the AMS. [S] read (through text recognition; recheck item numbers). Statement 9.1: the numbers of positive roots of each height form a partition whose conjugate is the list of exponents (observed, not proved). The fact that the entries of the highest root sum to $h-1$ (F14 of section 2.7) is usually cited to this paper; the item was not located in the text read [U]. Statements 3.1 to 4.5: the two-colour factorisation of a Coxeter element and its rotation by $2\pi/h$. Statement 6.3: the periodic two-colour word lists every positive root exactly once in its first $nh/2$ letters.
- **[Kos59]** B. Kostant, “The principal three-dimensional subgroup and the Betti numbers of a complex simple Lie group”, *Amer. J. Math.* 81 (1959), 973-1032. Not read. First uniform proof of the heights-and-exponents theorem [S2, via Abe et al., arXiv:1304.8033, and Viswanath, arXiv:math/0609248]. **[Mac72]** I. G. Macdonald, “The Poincaré series of a Coxeter group”, *Math. Ann.* 199 (1972), 161-174. DOI 10.1007/BF01431421. Not read; second proof [S2].
- **[Cox51]** H. S. M. Coxeter, “The product of the generators of a finite group generated by reflections”, *Duke Math. J.* 18 (1951), 765-782. [S] read (scan): eigenvalues $e^{2\pi i m_j/h}$, $m_j + m_{n+1-j} = h$, the order of the group as $\prod(m_j+1)$.
- **[Cas17]** W. Casselman, “Coxeter elements in finite Coxeter groups” (2017): https://personal.math.ubc.ca/~cass/research/pdf/Element.pdf. [S] read: Proposition 3.7 (eigenvalues of the Gram matrix against those of the Coxeter element) and Theorem 3.11 (rotation through $2\pi/h$ on the plane of the extreme eigenvectors). The free reference for the Coxeter plane recipe of section 2.14. The numbers of roots on each ring of the projection, and the fact that they are multiples of h and not always equal to h, are from the script [V].
- The statement that h alternating colour classes give a reduced word of the longest element is Lemma 2.1 of [FZ03b], which cites Bourbaki, Chapter V, section 6, Exercise 2 [S for the lemma, S2 for Bourbaki].
- What the site states about the numbers game uses these theorems **for coroots**. Every source states them for roots; the coroot version is the same theorem applied to the dual root system, and it was checked for every type of rank at most 8 [V]. Individual roots and their coroots can have different heights (in B_2 the root $\alpha_1+\alpha_2$ has height 2 and its coroot has height 3), so the text must say “heights of coroots”.

Crystals and strings (module M8).

- **[Kas94]** M. Kashiwara, “Crystal bases of modified quantized enveloping algebra”, *Duke Math. J.* 73 (1994), 383-413. Not read. Credited by Kashiwara’s later papers for the Weyl group action on crystals; the action is stated in arXiv:math/0010293, section 2.2 [S]: S_i applies f_i exactly $\langle h_i, \mathrm{wt}(b)\rangle$ times when that number is non-negative and e_i otherwise. The section number in the 1994 paper is [U].
- SageMath implements the same reflection of a crystal element along its i-string as the method `s(i)` [S].

Folding (module M6). Two dual constructions share the name; section 2.13 fixes which is which.

- **[Ste08]** J. R. Stembridge, “Folding by automorphisms” (2008): https://websites.umich.edu/~jrs/papers/folding.pdf. [S] read. Orbit sums of simple roots form a simple system; the folded Weyl group is the subgroup fixed by the automorphism; every crystallographic root system arises this way.
- **[Lus93]** G. Lusztig, *Introduction to Quantum Groups*, Birkhäuser, 1993, chapters 12 and 14. Not read; admissible automorphisms and Proposition 14.1.2 are [S2] (Savage, arXiv:math/0406073).
- **[FSS96]** J. Fuchs, B. Schellekens, C. Schweigert, “From Dynkin diagram symmetries to fixed point structures”, *Comm. Math. Phys.* 180 (1996), 39-97. arXiv:hep-th/9506135. [S] read: the “orbit Lie algebra” ($A_{2n-1} \to B_n$, $D_{n+1} \to C_n$, $E_6 \to F_4$, $D_4 \to G_2$), with the explicit remark that it is not a

subalgebra and that the fixed-point subalgebra corresponds to folding the transposed matrix.
- **[KLP09]** S. Kumar, G. Lusztig, D. Prasad, "Characters of simplylaced nonconnected groups versus characters of nonsimplylaced connected groups", arXiv:math/0701615. [S] read.
- SageMath's folded Cartan types (`type_folded.py`) use the other convention (restriction): C_n in A_{2n-1}, B_n in D_{n+1} [S].

Polytopes and the E_8 lattice (module M7).

- **[Cox35]** H. S. M. Coxeter, "Wythoff's construction for uniform polytopes", *Proc. London Math. Soc.* (2) 38 (1935), 327-339. **[Cox73]** H. S. M. Coxeter, *Regular Polytopes*, 3rd ed., Dover, 1973; the three figures 4_{21}, 2_{41}, 1_{42} are cited to section 11.8, pp. 202-203. Neither was read; citations are [S2].
- **[CKPS95]** B. Champagne, M. Kjiri, J. Patera, R. T. Sharp, "Description of reflection-generated polytopes using decorated Coxeter diagrams", *Canad. J. Phys.* 73 (1995), 566-584. Not read [S2]. The face-counting rule of section 2.14 is of this kind; the rule as written there was checked against known f-vectors and against exact convex hulls of 48 polytopes in dimensions 3 and 4 [V], and should be compared with this paper before it is attributed.
- E_8 theta series 1, 240, 2160, 6720, 17520, ...: OEIS A004009 [S]. Lattice data: https://www.math.rwth-aachen.de/~Gabriele.Nebe/LATTICES/E8.html [S].

The A-D-E pattern (module M3).

- **[HHSV77]** M. Hazewinkel, W. Hesselink, D. Siersma, F. D. Veldkamp, "The ubiquity of Coxeter-Dynkin diagrams (an introduction to the A-D-E problem)", *Nieuw Arch. Wisk.* (3) 25 (1977), 257-307. Free: https://ir.cwi.nl/pub/10040/10040D.pdf. [S] read. The survey Wildberger cites; quotes Arnold's Problem VIII.
- **[BrH]** A. E. Brouwer, W. H. Haemers, *Spectra of Graphs*, section 3.1.1 "Graphs with largest eigenvalue at most 2": https://homepages.cwi.nl/~aeb/math/ipm/ipm.pdf [S, section title and page]. The underlying theorem is due to J. H. Smith (1970); that citation is [U].
- **[Pro93]** R. A. Proctor, "Two amusing Dynkin diagram graph classifications", *Amer. Math. Monthly* 100 (1993), 937-941. DOI 10.2307/2324217. Citation [S2]. The labelling criterion (twice each label equals the sum of the neighbours' labels) is the fixed-point condition of the mutation game.

11.6 Explorable explanations: design practice and evidence

All [S] read unless marked.

- B. Victor, "Explorable Explanations" (2011), https://worrydream.com/ExplorableExplanations/; "Up and Down the Ladder of Abstraction" (2011), https://worrydream.com/LadderOfAbstraction/; "Learnable Programming" (2012), https://worrydream.com/LearnableProgramming.
- N. Case, "Explorable Explanations" (2014), https://blog.ncase.me/explorable-explanations; "How I Make Explorable Explanations" (2017), https://blog.ncase.me/how-i-make-an-explorable-explanation/; "4 More Design Patterns" (2018), https://blog.ncase.me/explorable-explanations-4-more-design-patterns/.
- F. Hohman, M. Conlen, J. Heer, D. H. Chau, "Communicating with Interactive Articles", *Distill* (2020), DOI 10.23915/distill.00028.
- M. Conlen, A. Kale, J. Heer, "Capture & Analysis of Active Reading Behaviors for Interactive Articles on the Web", EuroVis 2019, DOI 10.1111/cgf.13720: https://idl.cs.washington.edu/files/2019-IdyllAnalytics-EuroVis.pdf. The measured evidence behind principles P1, P2 and P11 of section 8: central widgets were used by most readers, optional and late material was not, and phone readers spent a quarter to a half of the desktop reading time.
- A. Tse, "Why We Are Doing Fewer Interactives" (Malofiej 2016): https://github.com/archietse/malofiej-2016.
- A. Patel (Red Blob Games): "Little things", https://www.redblobgames.com/making-of/little-things/; "Draggable objects", https://www.redblobgames.com/making-of/draggable/; "Responsive design", https://www.redblobgames.com/making-of/responsive-design/; "Making of: circle drawing", https://www.redblobgames.com/making-of/circle-drawing/. The practical source for pointer handling, hit areas and "build the static figure first".

- L. Alfieri, P. J. Brooks, N. J. Aldrich, H. R. Tenenbaum, "Does discovery-based instruction enhance learning?", *J. Educational Psychology* 103 (2011), 1-18: https://www.fisme.science.uu.nl/publicaties/literatuur/2011_alfieri_discovery_learning.pdf. Unassisted discovery did worse than explicit instruction; guided discovery did better.
- P. A. Kirschner, J. Sweller, R. E. Clark, "Why minimal guidance during instruction does not work", *Educational Psychologist* 41 (2006), 75-86.
- Y.-S. Kim, K. Reinecke, J. Hullman, "Explaining the Gap", CHI 2017, DOI 10.1145/3025453.3025592: https://idl.uw.edu/papers/explaining-the-gap. Prediction before reveal.
- PhET research page (interview-based playtesting): https://phet.colorado.edu/en/research.

11.7 Related interactive work surveyed (prior art)

Appendix F gives the survey table. The items most worth opening before building: Keller's quiver mutation applet (click-to-mutate with history), https://webusers.imj-prg.fr/~bernhard.keller/quivermutation/; the Langlands Programs preview apps, which run a TypeScript port of PyCox in the browser, https://preview.langlandsprograms.com/apps/coxeter; Group Explorer 3, https://nathancarter.github.io/group-explorer/index.html; Stembridge's Coxeter plane pictures, https://websites.umich.edu/~jrs/coxplane.html; D. Vogan's explanation of the E_8 picture (each root joined to its 56 nearest neighbours), https://math.mit.edu/~dav/e8plane.html; SimpLie, https://github.com/teake/simplie; vZome Online, https://www.vzome.com/docs/online.html.

11.8 Technology and standards

Versions were read from the package registries on 9 October 2026 [S]. Official documentation: Astro 7 upgrade guide, https://docs.astro.build/en/guides/upgrade-to/v7/; Astro islands, https://docs.astro.build/en/concepts/islands/; sharing state between islands, https://docs.astro.build/en/recipes/sharing-state-islands/; Svelte 5, https://svelte.dev/docs/svelte/overview; KaTeX options and changelog, https://katex.org/docs/options.html, https://github.com/KaTeX/KaTeX/blob/main/CHANGELOG.md; Vitest, https://github.com/vitest-dev/vitest; fast-check, https://github.com/dubzzz/fast-check; Playwright accessibility testing, https://playwright.dev/docs/accessibility-testing; GitHub Pages limits, https://docs.github.com/en/pages/getting-started-with-github-pages/github-pages-limits.

Accessibility: WCAG 2.2, https://www.w3.org/TR/WCAG22/; ARIA Authoring Practices, keyboard interface, https://www.w3.org/WAI/ARIA/apg/practices/keyboard-interface/; MDN on live regions, https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Guides/Live_regions. Performance thresholds: https://github.com/GoogleChrome/web-vitals.

Licensing: Creative Commons FAQ, https://creativecommons.org/faq/; attribution practice, https://wiki.creativecommons.org/wiki/Best_practices_for_attribution; https://choosealicense.com/non-software/; arXiv licence information, https://info.arxiv.org/help/license/index.html.

11.9 Corrections to citations that circulated in the project's earlier notes

As first written	Correct
Donnelly and Eriksson, "The numbers game and Dynkin diagram classification results", arXiv:math/0610702	arXiv:0810.5371. The number math/0610702 is an earlier preprint by Donnelly alone, with a longer title.
Gashi and Schedler, "On Dynkin and extended Dynkin diagrams via the numbers game"	no paper with this title was found. Their numbers-game paper is "On dominance and minuscule Weyl group elements", arXiv:0908.1091.
Donnelly, "Roots of asymmetric geometric representations of Coxeter groups"	arXiv:0707.3310; the later version is titled "Root systems for asymmetric geometric representations of Coxeter groups".

As first written	Correct
Wildberger's manuscript dated 2003	undated; cited by its author as a 2001 preprint and then as "in preparation"; PDF metadata of the project copy 25 November 2004.